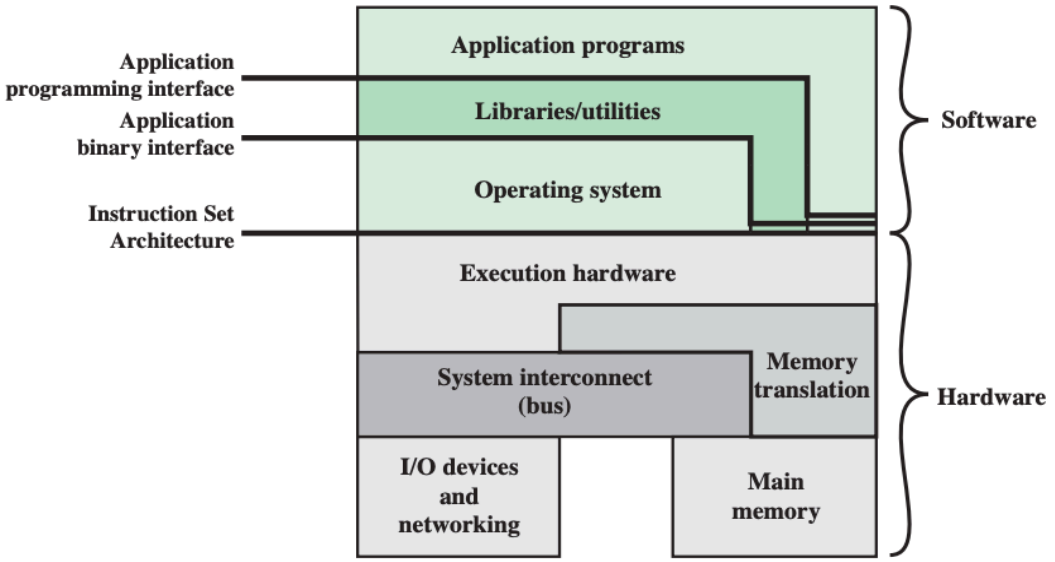
DISCLAIMER: questi appunti sono stati presi durante le lezioni e potrebbero essere soggetti a buchi/errori causati da assenze/distrazioni, in teoria i buchi sono stati riempiti, a volte pero' e' possibile che siano ancora presenti per causa di dimenticanze (sono un coglione), se c'e' qualcosa di mancante o di sbagliato siete liberi di aggiungerlo/correggerlo, man mano che studio anche io provvedero' a correggere e completare.

COMPITI DI UN SISTEMA OPERATIVO

La domanda cos'è un sistema operativo ha molte risposte, in generale il calcolatore (il computer) è solo una macchina di circuiti elettronici che eseguono operazioni basilari, queste operazioni però possono essere combinati in operazioni più complesse, che a loro volta possono essere combinati in operazioni ancora più complesse e così via, così arriviamo ai programmi con i quali possiamo fare una molteplicità di cose.
Il **sistema operativo** è quindi quel software che ha il compito di controllare gli altri programmi, e deve fare da interfaccia tra applicazioni e hardware, i suoi obiettivi sono quelli di convenienza (riuscire a fare più cose possibili con l'hardware a disposizione), efficienza (fare le cose in maniera efficiente senza ritardi inutili) e la capacità di evolversi.

Il sistema di elaborazione



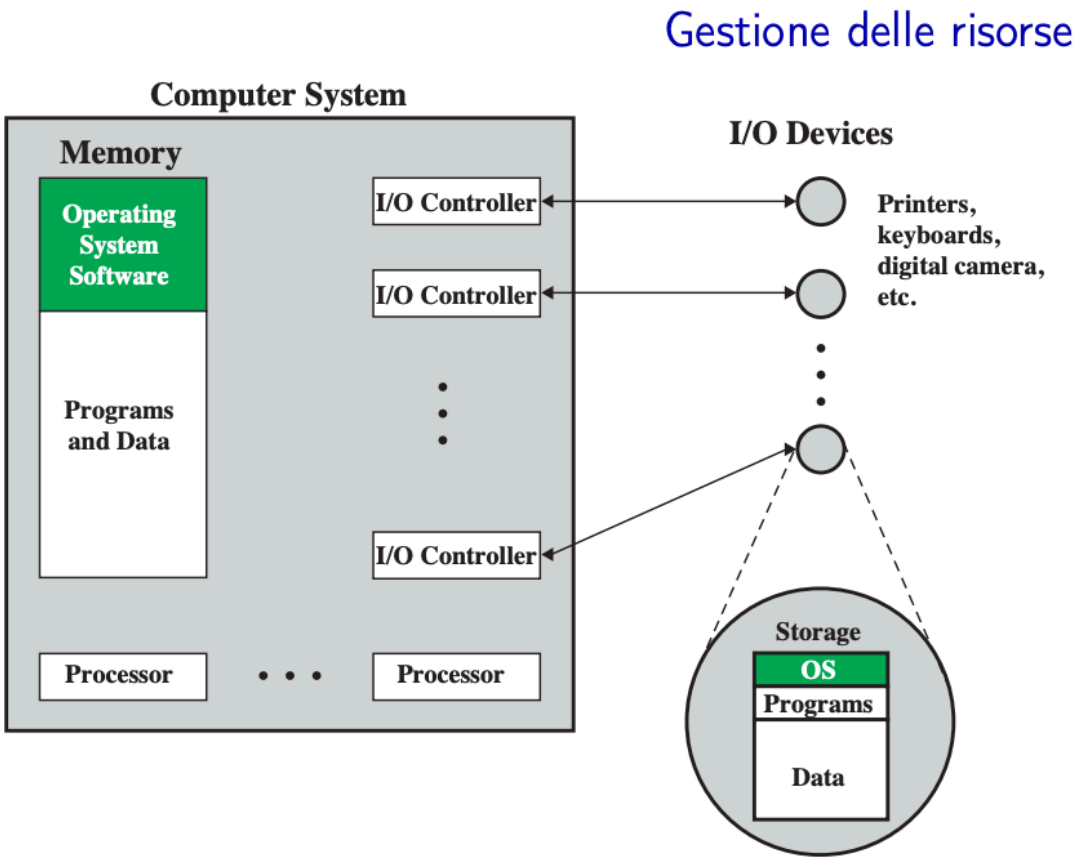
Il sistema operativo fornisce diversi servizi:

- La creazione dei programmi e la loro esecuzione;
- L'accesso ai dispositivi I/O;
- L'accesso controllato ai file;
- L'accesso al sistema;
- La rilevazione degli errori e la risposta;
- La contabilità.

Si dota di 3 interfacce, 3 livelli differenti di complessità:

- ISA, Instruction Set Architecture;
- ABI, Application Binary Interface;
- API, Application Programming interface.

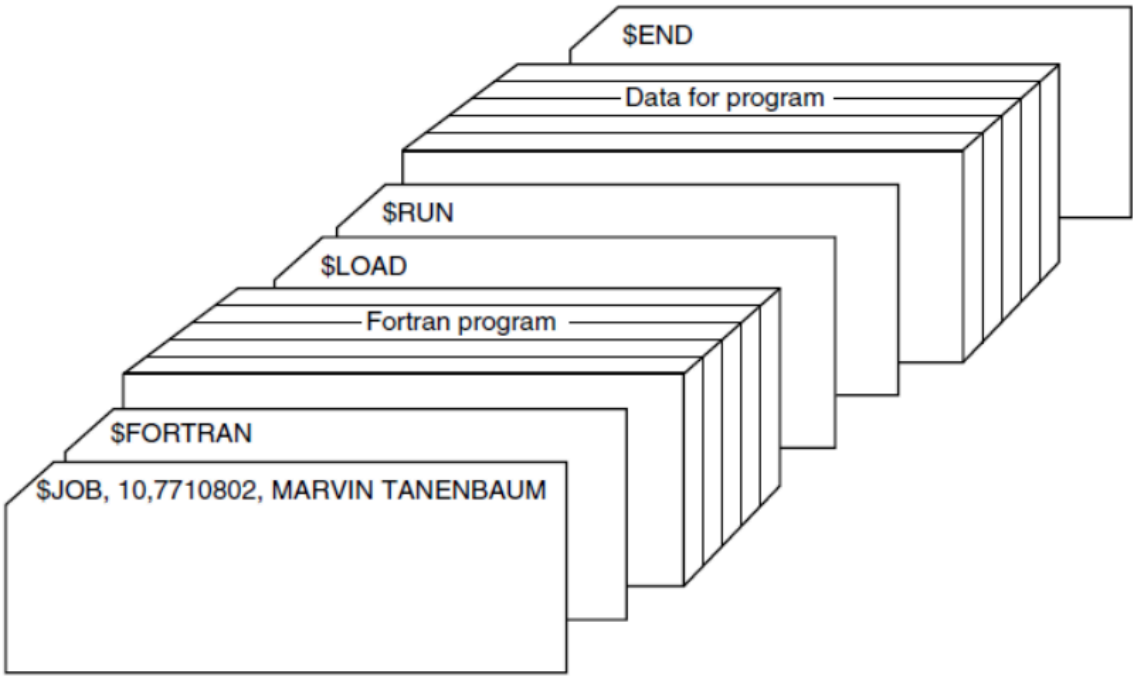
GESTIONE DELLE RISORSE



Il processore impiega pochissimo tempo a fare calcoli semplici che gli richiediamo noi, anche il sistema operativo è un programma gestito dal processore, quindi per quanto il sistema operativo "comandi" la CPU, egli comanda solo fin quando il processore gli permette di comandare, se il sistema vede che un altro programma va eseguito allora la CPU si sposterà su quell'altro finchè la situazione non cambierà.

EVOLUZIONE DEI SISTEMI OPERATIVI

Il sistema operativo si evolve nel tempo tramite aggiornamenti e nuovi tipi di hardware, richieste di nuovi servizi, correzione di nuovi errori.
La Prima Generazione di computer (1945-55) è stata realizzata in Gran Bretagna durante la Seconda Guerra mondiale per cercare di decodificare i messaggi segreti dei tedeschi, funzionava tramite delle valvole, senza sistemi operativi e veniva controllato tramite una console. Era una macchina però piuttosto problematica, richiede chissà quante valvole e metodi di raffreddamento di queste valvole e richiedeva un costo enorme. Tra gli anni 50 e 60 (Seconda generazione) si pensa quindi a un nuovo tipo di approccio, c'è l'innovazione dei transistor e dei sistemi a lotti gestiti da un operatore con i lavori raggruppati in lotti sul dispositivo di ingresso. Il **monitor** ordina il caricamento dei programmi e delle utility necessarie tramite il Job Control Language.

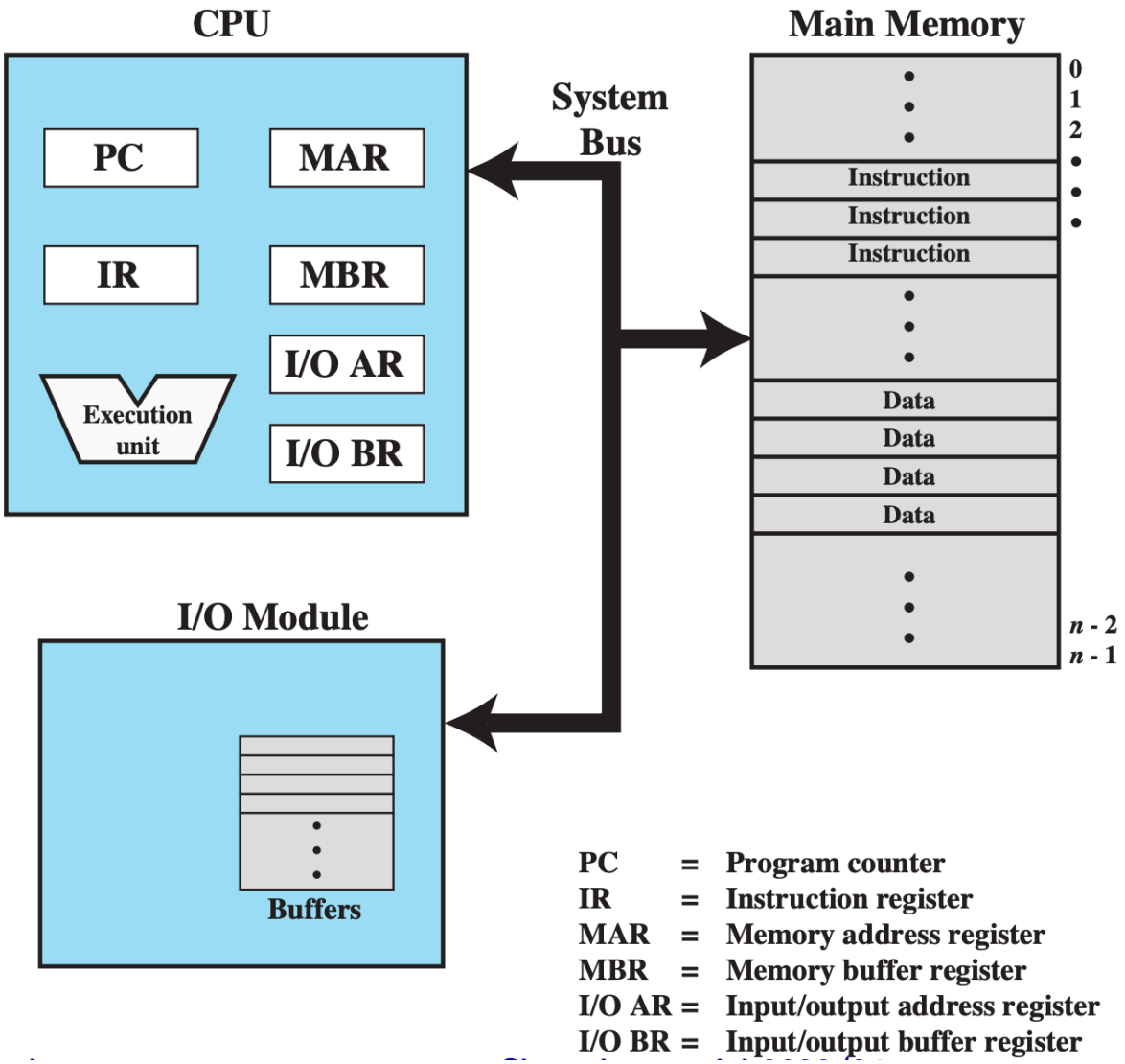


GESTIONE DELLA MEMORIA

Un'esecuzione di questo tipo tramite queste schede ci impone la creazione di nuove funzioni hardware, come la protezione della memoria, nella RAM una parte di memoria conterrà le istruzioni del programma da eseguire e un'altra che conterrà i gestori dei dispositivi, oppure le interruzioni, inoltre se si tenta di accedere al di là della memoria allocata per un programma si rischia di accedere alla memoria appartenente ad un altro programma, causando guai, perciò ci sono delle istruzioni hardware privilegiate che impediscono che ciò accada oppure a volte concedono

questi privilegi a dei programmi che devono accedere alla memoria di un altro programma.
Qui nascono dei problemi, ora parte della memoria è occupata dai monitor, che quindi non devono sprecare spazio tecnico e tempo macchina.
Così nella terza generazione (65-80) si pensa a metodi per ottimizzare i tempi e i costi di esecuzione di una macchina, come la multiprogrammazione, che consente di eseguire processi contemporaneamente quando possibile anziché uno ad uno e il time-sharing, che gestisce l'esecuzione di programmi da più utenti diversi.
Alcuni dei problemi della multiprogrammazione sono la gestione delle memoria e lo scheduling, poichè più utenti useranno terminali diversi per accedere al calcolatore, perciò il processore è condiviso da più utenti, si devono gestire quindi più lavori interattivi e il processore alterna l'esecuzione di ciascun programma per un breve periodo.
Oltre ai problemi di gestione della memoria e di scheduling ci sono i problemi di controllo degli accessi, poichè chiunque potrebbe accedere al calcolatore perciò c'è bisogno di verificare l'identità degli accessi.
Dal 1980 in poi è cominciata l'integrazione su larga scala dei computer, tramite i circuiti integrati le dimensioni dei calcolatori si sono ridotte e nascono i Personal Computer, che poi si evolveranno negli smartphone.

COMPONENTI DEL SISTEMA DI ELABORAZIONE

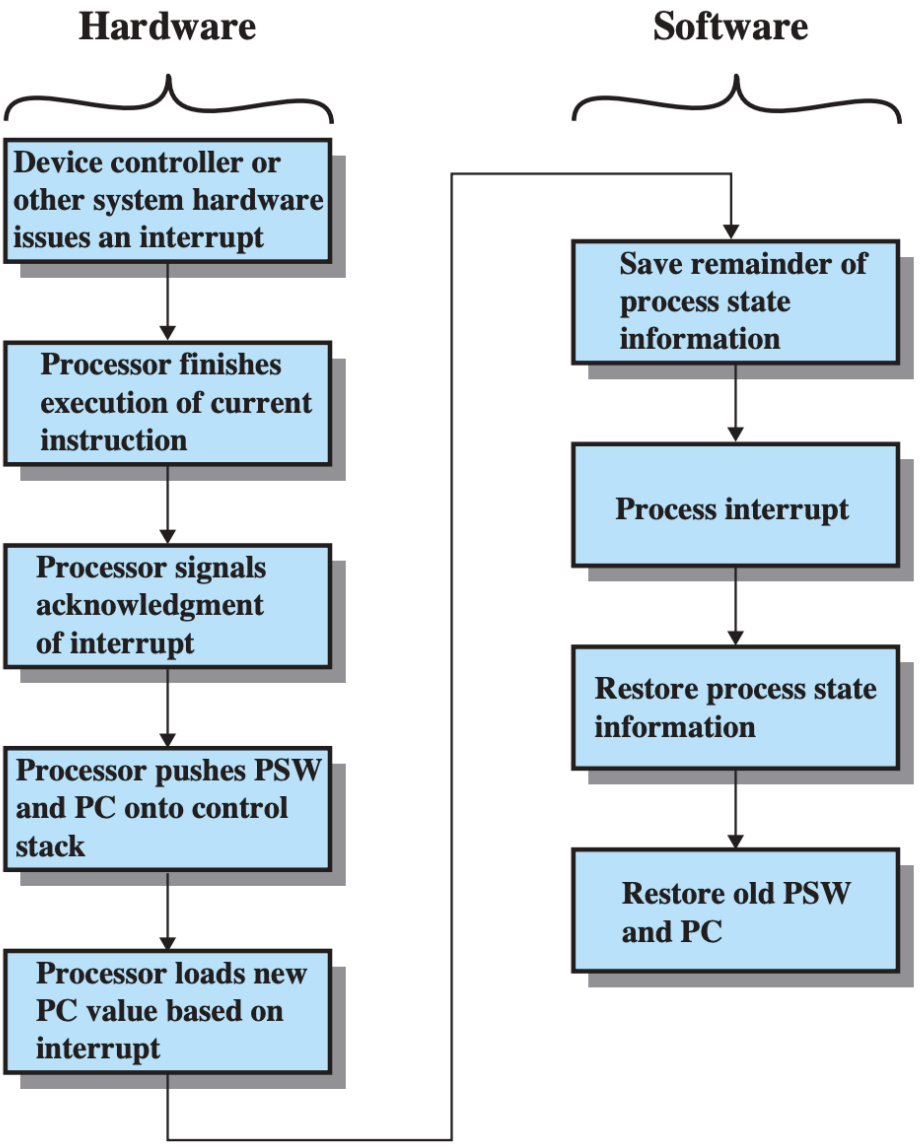


L'unità centrale del computer deve avere o un processore su un singolo chip o un multiprocessore, ossia più processori (core) su un chip (socket) ed è dotato di cache, può avere poi un processore grafico (GPU), che computa su array di dati con la tecnica Single-Instruction-Multiple-Data (SIMD), si usa in generale per le applicazioni grafiche, ma può essere utilizzato anche in altri ambiti.
Possiamo anche avere un processore di segnale digitale (DSP), che codifica e decodifica segnali audio e video e un SoC (principalmente su smartphone e tablet), ossia sistemi sviluppati sui chip.

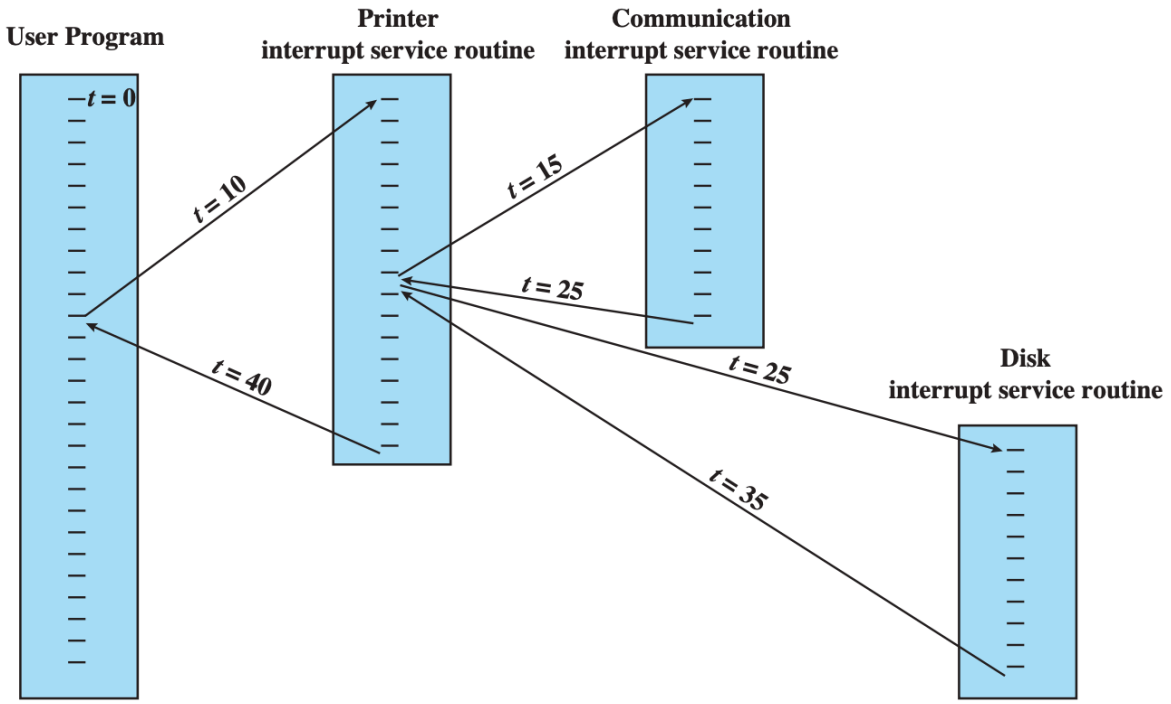
Il calcolatore in generale preleva l'istruzione dalla ram, la esegue(**ciclo fetch-execute**) e incrementa il **program counter** (che conta a quale linea siamo arrivati) di 1 per passare all'istruzione successiva e inserisce l'istruzione nell'**instruction register** per poi interrompere il programma alla fine delle istruzioni.
In un ciclo fetch-execute a meno che non sia disabilitato il sistema di interruzione si passa al **controllo delle interruzioni**, se c'è si salta da qualche altra parte del programma dove si gestisce l'interruzione e si mette il programma in pausa.

Possono essere interruzioni generate da errori di esecuzione del programma, oppure generate da timer che permettono al sistema operativo di eseguire delle funzioni a intervalli regolari, generate da un controller I/O per segnalare il completamento di un'operazione o un errore o generate da problemi hardware. Il vantaggio delle interruzioni è quello di impedire ad esempio di lasciare la CPU in attesa di I/O, oppure sospendere l'esecuzione del programma.

Gestione dell'interruzione



Può accadere di avere interruzioni multiple (interruzioni dentro le interruzioni); per questo esistono 2 approcci, o si elimina la possibilità di fare interruzioni all'interno di interruzioni, oppure si usa uno schema prioritario, in questo modo:



ARCHITETTURA DEI SISTEMI OPERATIVI

Architettura dei Sistemi Operativi

Aspetti principali nello sviluppo dei sistemi operativi:

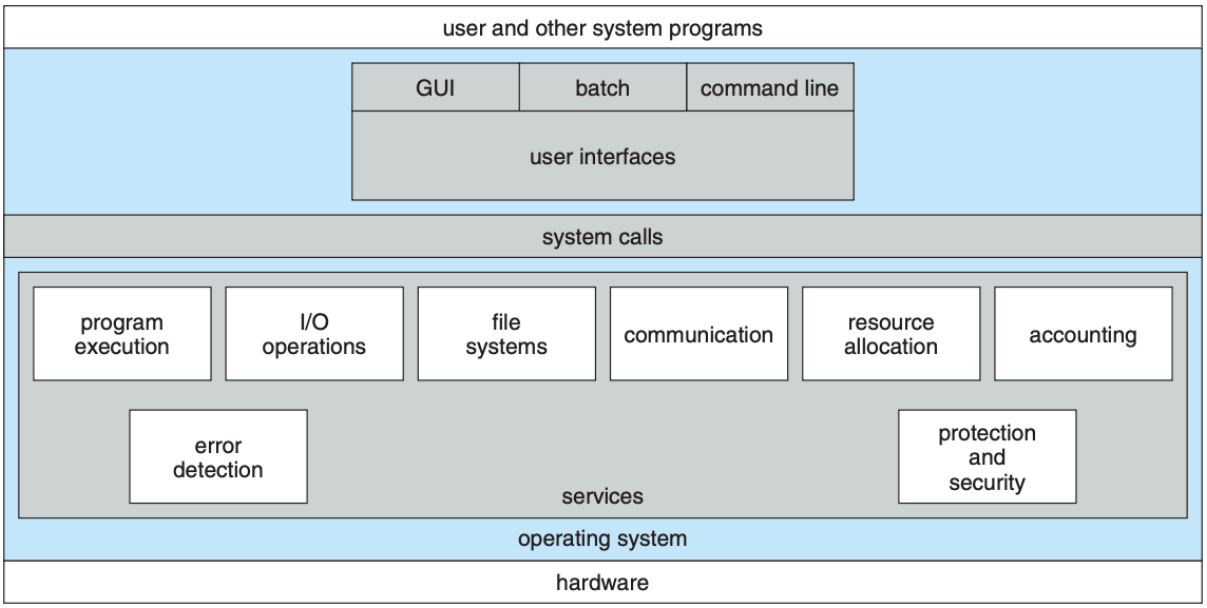
- Processi
- Gestione della memoria
- Protezione e sicurezza dell'informazione
- Schedulazione e gestione delle risorse
- Struttura del sistema

Approcci e elementi progettuali dei sistemi operativi:

- Architettura microkernel
- Multithread
- Multiprocessing simmetrico
- Sistemi operativi distribuiti
- Architettura orientata agli oggetti

Il sistema operativo realizza delle macchine virtuali, ciascuna con la sua memoria e il suo sistema operativo (eventualmente, il suo hardware), il virtual machine monitor

(VMM) è eseguito sul sistema operativo e gestisce le comunicazioni tra i singoli sistemi operativi virtuali e il processore.



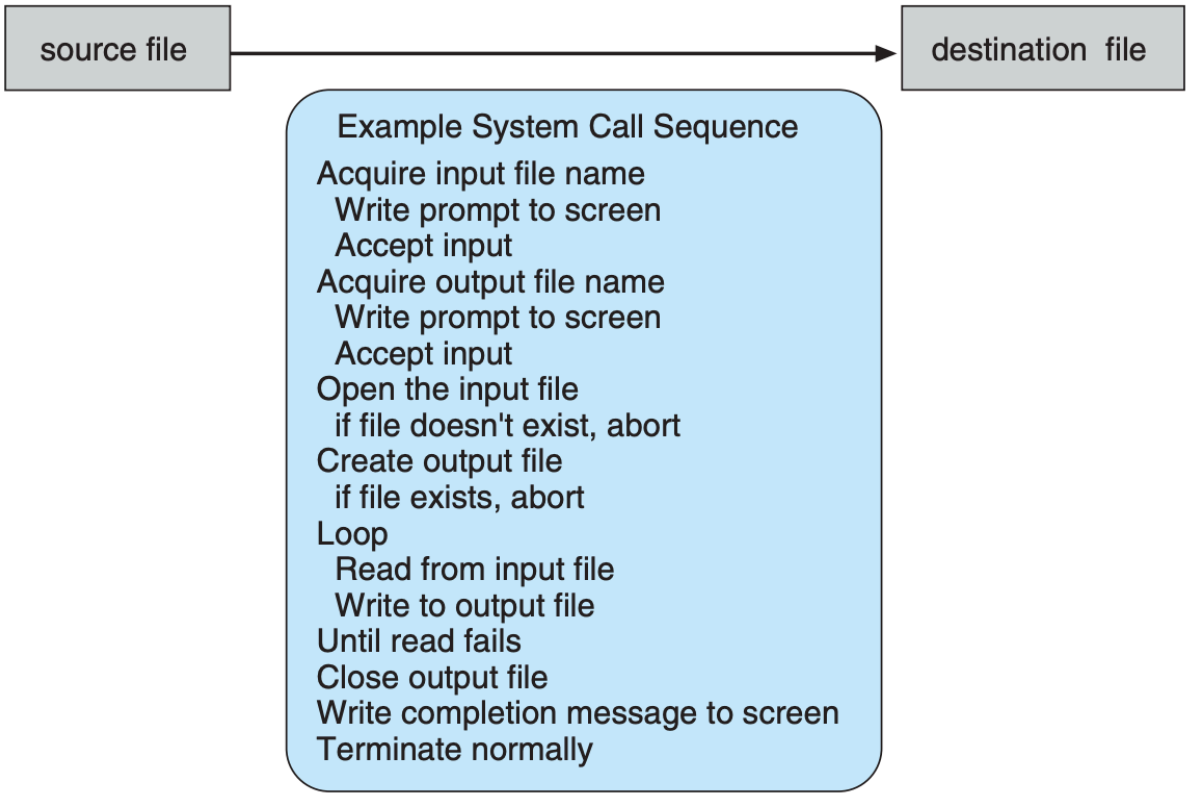
I servizi forniti dal sistema operativo per l'utente sono molti:

- L'interfaccia utente, come la linea di comando, la GUI o il batch, l'esecuzione;
- Il caricamento e la terminazione dei programmi;
  la gestione delle operazioni I/O (come la memoria che ha un aspetto centrale);
- Il file-system, senza il quale non si potrebbero registrare sequenze di caratteri o strutture più complesse in file, non si potrebbe neanche cercare file o informazioni e non si potrebbero gestire i permessi da assegnare ai file;
- La comunicazione fra processi, programmi in esecuzione svolgono compiti che devono essere coordinati tra di loro, possono condividere ad esempio la memoria, perché altrimenti la regola è che ogni programma dovrebbe lavorare indipendentemente e non potrebbe accedere alla memoria di un altro programma, tuttavia in casi particolari si può autorizzare l'accesso alla memoria di alcuni programmi;
- Rilevazione errori, che possono essere o di hardware, o di I/O, o dei programmi dell'utente;
- Funzioni per assicurare efficienza, come l'allocazione delle risorse, ossia il processore, la memoria, il disco e altri dispositivi I/O;
- L'accounting;
- La protezione, col controllo dell'uso e delle risorse, e la sicurezza, col controllo degli accessi al sistema.

INTERFACCE UTENTI

L'interprete dei comandi (**CLI**) esegue i comandi specificati dall'utilizzatore, può essere o implementato nel nucleo del sistema operativo, o può essere a sua volta un programma, nel secondo caso è possibile avere versioni multiple (**shells**), i comandi sono implementati nell'interprete o in programmi esterni.

Per l'utente medio la CLI risulta scomoda, perciò è disponibile un'interfaccia grafica (**GUI)**, che è un'interfaccia amichevole, anch'essa può essere o implementata nel nucleo o può essere un programma di sistema, si ha una "scrivania" dove è possibile trovare tutti i file messi in primo piano, con le icone che rappresentano file, directories (cartelle), programmi e funzioni, e menù che permettono di selezionare i comandi, in alcuni dispositivi è disponibile anche il touch-screen o i comandi vocali. Quale delle due è migliore? La CLI mi consente di automatizzare compiti ripetitivi tramite gli **shell script**, inoltre la GUI non consente l'accesso a tutte le funzioni a differenza della CLI, tuttavia la GUI è più intuitiva e quindi più adatta all'utente medio.
Le **system call** (chiamate di sistema) sono un'interfaccia per il programmatore ai servizi del sistema operativo, vi si accede tramite un'API che nasconde i dettagli al programmatore, i parametri sono passati tramite i registri, blocchi o tabelle di memoria, o nello stack da cui sono prelevati (pop che ritorna il primo elemento e lo rimuove) dal sistema operativo.



CATEGORIE DI CHIAMATE DI SISTEMA

CONTROLLO DEI PROCESSI:

- Creazione e arresto;
- Caricamento, esecuzione;
- Terminazione normale e anormale;
- Esame e impostazione degli attributi tramite funzioni apposite;
- Funzioni che mettono in pausa per un tempo indicato (ad esempio quando richiedono un input che scade dopo un tot di tempo);

- Attesa e segnalazione di un evento;
- L'assegnazione e il rilascio di memoria, il processo solitamente non ha a disposizione tutta la memoria del dispositivo, ma man mano si carica nella RAM il necessario.

GESTIONE DEI FILE:

- Creazione e cancellazione;
- Apertura, chiusura, lettura, scrittura e posizionamento;
- Esame e impostazione degli attributi (permessi di lettura, scrittura assegnati al proprietario, al gruppo e a tutti).

GESTIONE DEI DISPOSITIVI I/O:

- Richiesta e rilascio;
- Lettura, scrittura e posizionamento;
- Esame e impostazione degli attributi.
- Inserimento logico ed esclusione logica.

GESTIONE DELLE INFORMAZIONI

- Esame e impostazione dell'ora e della data;
- Esame e impostazione dei dati del sistema;
- Esame e impostazione degli attributi dei processi, file e dispositivi (come detto prima ad esempio, i permessi).

COMUNICAZIONE

- Creazione e chiusura di una connessione;
- Invio e ricezione di messaggi (interazione con messaggi);
- Creazione e accesso a regioni di memoria condivisa (interazione con condivisione);
- Informazioni sullo stato di un trasferimento;
- Inserimento ed esclusione di dispositivi remoti.

PROTEZIONE

- Controllo degli accessi alle risorse;
- Visualizzazione e impostazione dei permessi;
- Autorizzazione e divieto di accesso all'utente.

Un sistema operativo è fatto anche di programmi, detti **utilities**, servono a realizzare l'ambiente utile per lo sviluppo e l'esecuzione dei programmi, ce ne sono diverse categorie:

- Gestione files;
- Informazioni di stato;
- Modifica dei file (text editor, da quelli più semplici);
- Supporto alla programmazione (tutto ciò che serve per la compilazione, il linker, il debugger, ecc.);
- Caricamento ed esecuzione dei programmi;

- Comunicazioni;
- Servizi in background;
- Programmi applicativi.

PROGETTAZIONE DEL SISTEMA OPERATIVO

Come si progetta dunque un sistema operativo, bisogna decidere gli **obiettivi e le specifiche:**

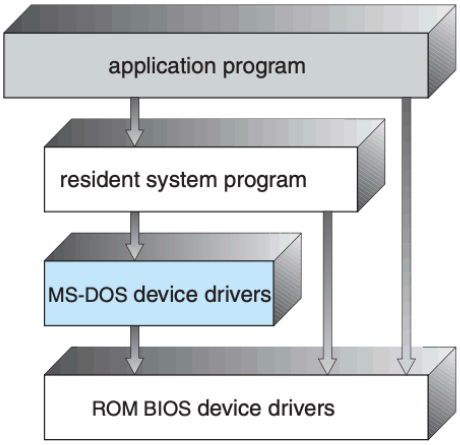
- Decidere l'hardware;
- Decidere quale tipo di sistema utilizzare;
- Obiettivi dell'utilizzatore;
- Obiettivi del sistema, (un obiettivo dell'utilizzatore può essere che i suoi programmi terminino nel minor tempo possibile, quello del sistema magari può essere di non lasciare il processore inutilizzato, quindi si deve scegliere quale dei due va valorizzato).
- Separazione delle politiche dai meccanismi, sarebbe scritto male un programma che mescola i due aspetti, dovremmo separare il meccanismo (come si accede al disco), dalla politica (ad esempio quando si ha un server si serve il client che manda per primo la richiesta, non l'ultimo, questa è una politica FIFO).

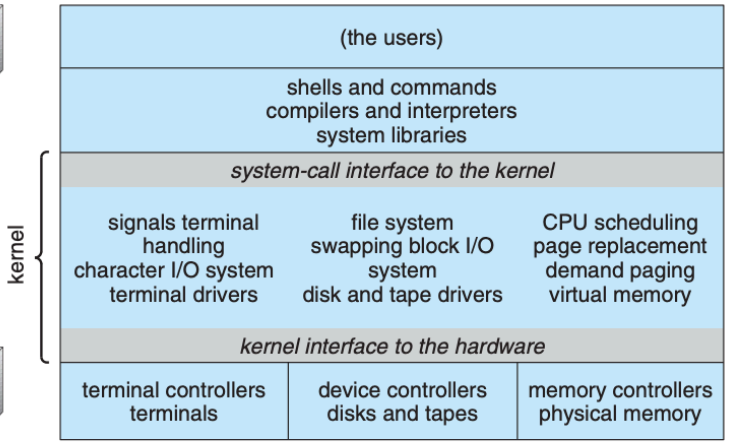
I moderni sistemi operativi sono scritti quasi interamente in linguaggi di alto livello (C, C++), facili per sviluppo e manutenzione, compatti e portabili, poichè devono essere efficienti, i computer ovviamente non sono fatti per mandare avanti il sistema operativo, ma sono fatti per far partire i programmi di sistema, che possono essere scritti anche in linguaggi più inefficienti e interpretati come Python e Perl. Alcune sezioni critiche dei sistemi operativi che richiedono altissima efficienza sono scritte in Assembler.

STRUTTURE DEL SISTEMA OPERATIVO

SISTEMI MONOLITICI:

Non divisi in moduli:





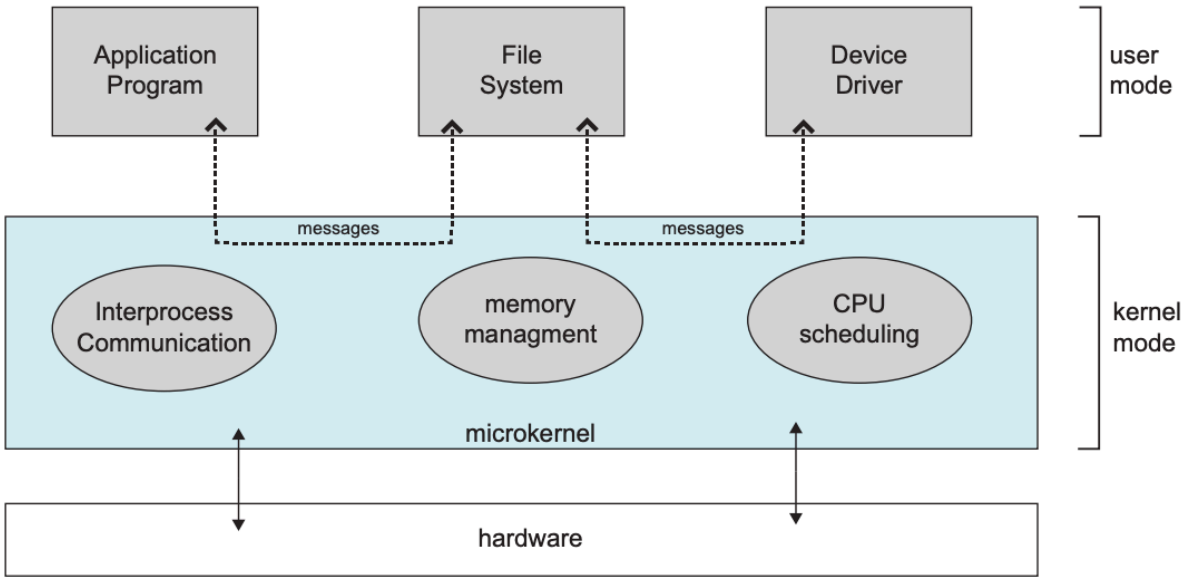
MS-DOS UNIX tradizionale

MS-DOS (primo sistema operativo per pc) e UNIX tradizionale sono **sistemi monolitici**, nonostante ciò sono comunque complessi e fatti tramite tantissime funzioni, il nucleo dello UNIX tradizionale ha un'interfaccia verso l'hardware per la gestione delle system call, questo all'interno di un programma (kernel) che gestisce i segnali in I/O, il file system, i driver, i dispositivi di archiviazione e lo **scheduling,** ossia la scelta del programma a cui assegnare il processore o la scelta di quali parti di un programma andrebbero caricati in memoria.

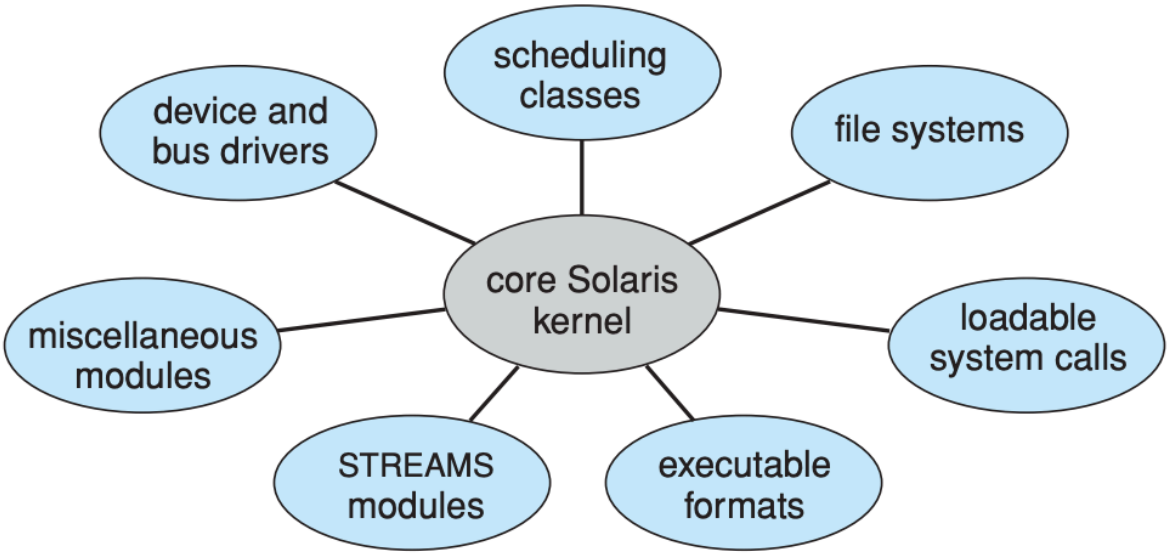
SISTEMI STRATIFICATI

Ogni strato (**layer)** è l'implementazione di un oggetto astratto fatto di dati e operazioni per manipolarli, ogni strato nasconde i dettagli dell'implementazione agli strati superiori, ha dei vantaggi, come la semplicità di scrittura, l'aggiornamento e il debugging e la portabilità, la difficoltà sta nel definire gli strati.

Si serve di un **Microkernel**, che fa il minimo necessario, il resto è intestato a programmi utente-layer (client-server), le richieste di servizi avvengono tramite scambio di messaggi appunto tra client e server. Uno degli svantaggi è l'overhead, ossia siccome il kernel affida molti più compiti ai programmi utente-server c'è bisogno di un maggiore tempo di esecuzione.



Non c'è differenza dal punto di vista del sistema tra questi programmi, ma ogni volta che un programma applicativo deve accedere a un file o un dispositivo invierà una richiesta tramite messaggio che sarà svolta da un altro programma che è il gestore del dispositivo e non dal kernel poiché esso contiene solo le funzioni fondamentali. Le altre funzionalità sono in moduli che possono essere caricati e collegati all'occorrenza, ogni modulo nasconde i dettagli dell'implementazione agli altri moduli.



Solaris ha questo tipo di architettura, con il kernel che contiene tutte le funzioni fondamentali e una serie di moduli da collegare quando si vuole per le loro specifiche funzioni.

DEBUGGING

Nella progettazione e nella gestione di un sistema operativo è il problema del debugging, gli OS sono sistemi complessi, quindi è normale che ci siano degli errori,

che però sono difficili da individuare poiché compaiono solo in situazioni particolari, non basta inoltre che il sistema non abbia errori, poichè deve anche fare il suo compito nel minor tempo possibile. Quindi valutare le prestazioni di un sistema operativo richiede una lunga sperimentazione tramite anche la generazione di file di log che registrano il comportamento del sistema e **core dump.**

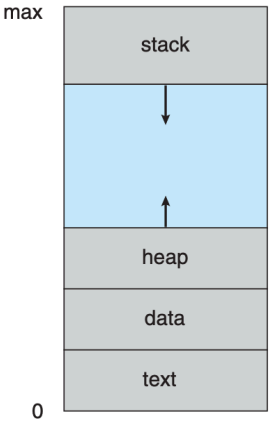
GENERAZIONE DEL SISTEMA OPERATIVO

Scritto il codice sorgente del sistema esso va compilato e configurato per il sistema su cui verrà eseguito (Windows, linux sono fatti per lavorare su diverse architetture, con parti dedicate a delle specifiche architetture), si installa il sistema e si avvia il computer e il nuovo sistema operativo.
La configurazione può realizzarsi modificando il codice prima della compilazione in base al sistema su cui lo vado a installare, oppure tramite la selezione di tabelle e moduli precompilati da collegare al momento dell'installazione, oppure tramite un sistema guidato da tabelle di dati che guidano il sistema in base all'hardware.
Per far partire il sistema l'esecuzione inizia in un predefinito indirizzo della ROM (questa parte di memoria non si modifica neanche a computer spento (o acceso)) contenente il **bootstrap loader**, le istruzioni necessarie a caricare il **bootstrap program** che a sua volta carica il kernel e avvia l'esecuzione, così il sistema entra in **esecuzione.**

PROCESSI

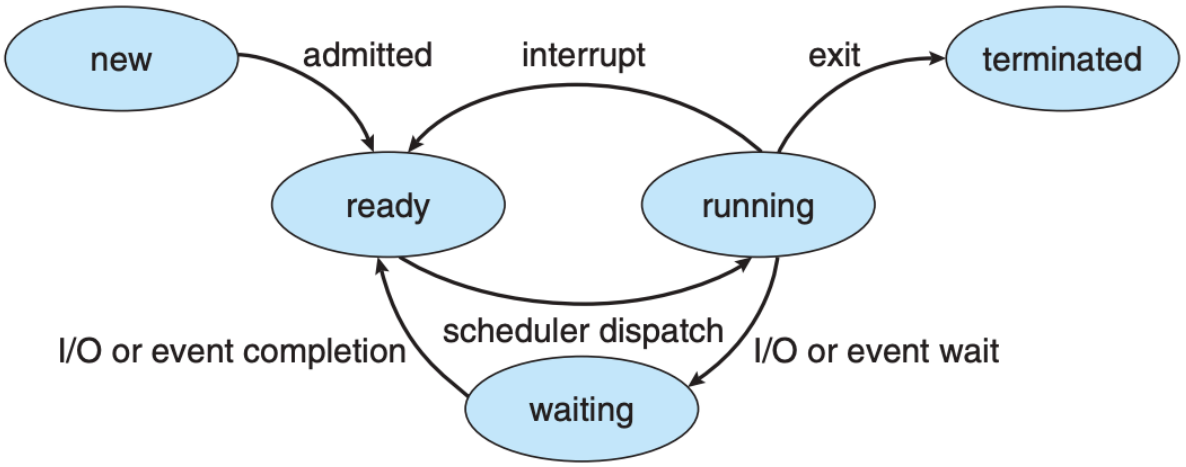
In un sistema moderno ci sono più attività simultaneamente sul nostro sistema, poiché sarebbe un peccato sprecare il processore lasciandolo inattivo anche perchè oggi i processori hanno più core.
Per realizzare un sistema moderno è molto importante la nozione di **processo,** (o **job)** che è un programma in esecuzione.
Esso è dotato di un **program counter**, con l'indirizzo dell'istruzione che deve essere eseguita e il contenuto dei registri (**immagine del processore**). Quando il processo è in esecuzione il processore preleva l'istruzione nel program counter e in base al contenuto nei registri procede all'esecuzione dell'istruzione, quando il processo non ha più possibilità di andare avanti a causa di un **evento** (ad esempio deve interagire con la tastiera o con un dispositivo a cui non ha accesso), a quel punto è inutile lasciare il processore con tutti i dati dei registri in attesa del processo, in quel caso l'immagine dei dati nei registri (**immagine del processore)** viene salvata a parte e il processore viene riassegnato per far avanzare altri processi.
Altri elementi del processo sono lo **stack,** che contiene variabili locali, o indirizzi di return o parametri delle funzioni.
Poi c'è il programma (**sezione testo)**, la **sezione dati** (le variabili globali) e lo **heap** (spazio riservato per l'allocazione dinamica di dati).

Il **programma** è l'entità passiva, il file eseguibile, invece il **processo** è l'entità vera e propria che esegue le istruzioni e usa le risorse associate, un **programma** può essere associato a più processi.



Nella memoria il processo occupa un certo spazio occupato dal **codice,** i **dati,** l'**heap** e lo **stack**, dove vengono eseguite le procedure.
L'heap si allarga verso l'alto (malloc aumenta lo stack), lo stack verso il basso (man mano che chiamiamo funzioni si allarga). Bisogna stare attenti che stack e heap non collidano.

Quando il processo viene creato esso può trovarsi o in **attesa** di un processore assegnato ad esso che esegue le istruzioni e termina, può capitare che l'esecuzione del processo sia temporaneamente interrotta, magari dal bisogno di ricevere dell'input, quindi in quel momento non è necessario che le risorse del processore siano ancora assegnate a questo progetto, quindi si salva l'immagine del processore e il processo rimane in **attesa** dell'evento che lo libera dallo stato di attesa, quando l'evento avviene saranno riassegnate al processo delle risorse con l'immagine salvata prima e si completerà l'esecuzione e terminerà il processo.



PCB
A ogni processo è associata una struttura dati detta **descrittore di processo (PCB, process control block),** contenente tutte le informazioni relative al processo:

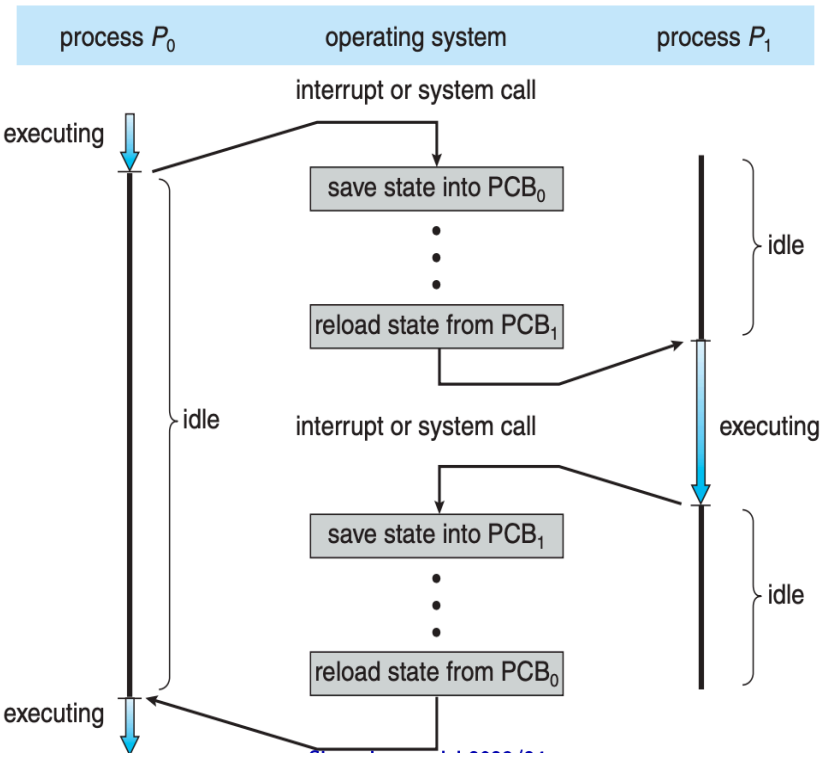
- Stato del processo;
- Program counter;
- Registri CPU (immagine del processore);
- Parametri per lo scheduling della CPU (come l' "importanza" del processo e quanta priorità va data al processo, processi di disco ad esempio sono più importanti dei processi accessori dell'utente);
- Informazioni per la gestione della memoria;

- Informazioni di accounting per sistemi multiutente;
- Informazioni su I/O;

Un processo può avere multiple sequenze di istruzioni (**THREADS**), processi più leggeri e semplici che collaborano fra di loro in un unico “processo” che possono essere eseguite in parallelo anche su core differenti per realizzare **task** differenti, in questo caso il PCB include informazioni su ciascun thread.

SALVATAGGIO DEL PCB

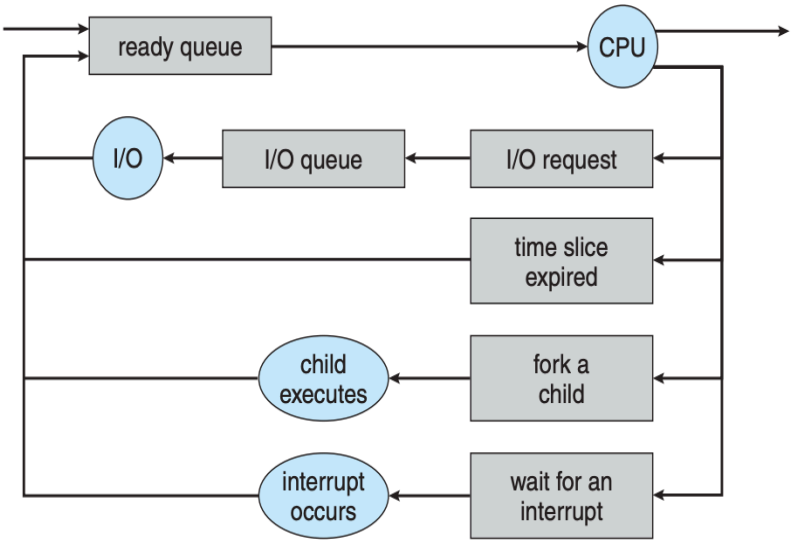
Supponiamo che venga creato il processo p0 che sta venendo eseguito (in stato di running), a un certo punto c’è un’interruzione e tutte le informazioni del processo sono salvate nel PCB0, che poi saranno ricaricate a cura del sistema di gestione delle interruzioni, a questo punto il processore passa ad un altro processo p1, di cui ricarica lo stato da PCB1 quando magari p1 era stato interrotto prima di p0, a quel punto può accadere un’altra interruzione, si ri-salva lo stato di p1 dentro PCB1 e si ricarica lo stato (l’immagine) contenuta in PCB0 e si riprende l’esecuzione di p0.



Questo ci permette di usare il sistema in maniera efficiente purchè le operazioni di salvataggio e ricarica degli stati del processore siano eseguite in maniera efficiente.

I PCB sono inseriti in varie code:

- Coda **job**, contenente tutti i processi;
- Coda **ready**, contenente tutti i processi pronti a partire;
- Coda dei **dispositivi,** una per dispositivo, contenente i processi in attesa di quel dispositivo.



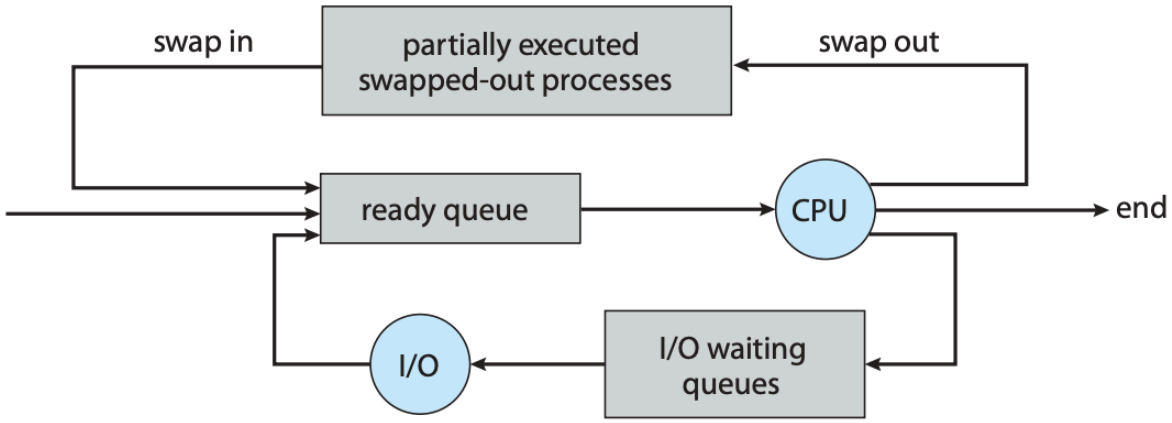
SCHEDULATORI

Il sistema operativo deve decidere a quali processi assegnare il processore, si può definire uno **schedulatore a lungo termine,** che prende i processi in coda job e li inserisce in coda ready, controlla quindi il grado di multiprogrammazione, è eseguito con poca frequenza, quando ad esempio si ha una lunga lista di processi pronti che può rallentare pesantemente il sistema, può applicare politiche complesse, ad esempio ci sono **processi CPU-bound**, processi che non rilasciano mai a meno che non forzati la CPU a loro assegnata, ci sono però alternative, ad esempio in UNIX se entro un certo periodo di tempo il processo non rilascia il processore viene costretto a rilasciarlo e viene rimesso in coda (**timebound**), oppure **processi I/O bound** che avranno un'interruzione dove dovranno attendere dell'input dall'utente o da qualche dispositivo.

Uno **schedulatore a breve termine** invece prende i processi dalla coda ready e gli assegna un processore, è infatti un **CPU scheduler** e deve essere eseguito con grande frequenza e perciò deve essere molto veloce (i processi running vengono anche messi in coda ready proprio perchè lo scheduler assegna la cpu a un processo, poi la toglie a quello per assegnarla ad un altro, così da far avanzare più processi contemporaneamente).

Uno **schedulatore a medio termine** invece è quello che assegna e revoca la memoria ai processi (**swapping)**, nei sistemi moderni c'è la **paginazione**, quindi in realtà tutti i processi hanno una memoria a loro assegnata, in cui c'è quella parte di programma che serve al momento.

Schedulazione a medio termine:

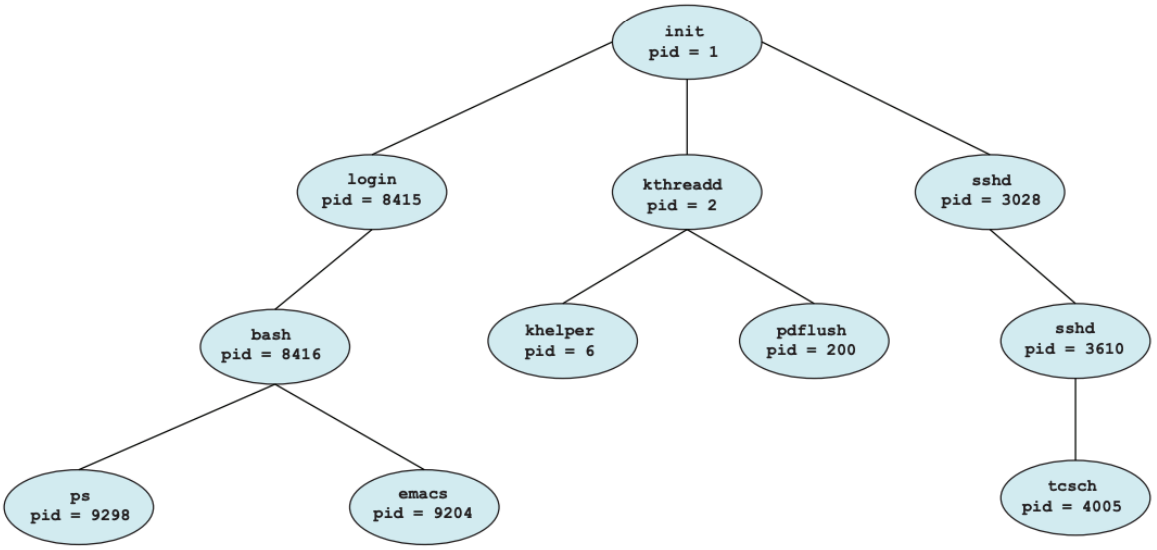


Per l'assegnazione della CPU ci sono delle politiche differenti, ad esempio si distingue tra processi I/O bound e processi CPU-bound che vengono gestiti in maniera differente.

Il problema dell'**overhead (cioè le risorse accessorie che sono ridondanti rispetto a quelle necessarie)** varia in base alla velocità della memoria, del numero dei registri e delle istruzioni speciali.

CREAZIONE DEI PROCESSI

I meccanismi di creazione e terminazione sono forniti dal sistema operativo, un **processo genitore** crea un **processo figlio** che a sua volta può creare altri processi, generando un **albero dei processi**, ogni processo è identificato da un numero detto **identificatore di processo** (o **pid**).



Il processo figlio può ottenere le **risorse** dal processo genitore, quindi le risorse possono essere **ripartite (assegna qualcuna delle sue risorse al figlio)** o **condivise (tutte le risorse sono condivise da figlio e genitore),** in alternativa ci pensa il sistema operativo ad assegnare le risorse al processo figlio.
Genitore e figlio possono essere eseguiti concorrentemente mentre il genitore attende la terminazione del figlio.
Per quanto riguarda la memoria il processo figlio può avere un duplicato di memoria del genitore e quindi avere lo stesso programma, oppure può averne uno nuovo.

```
#include < sys/types.h >
#include < stdio.h >
#include < unistd.h >
int main()
{
pid_t pid;
    /* fork a child process */
    pid = fork();
    if (pid < 0) { /* error occurred */
        fprintf(stderr, "Fork Failed");
        return 1;
    }
    else if (pid == 0) { /* child process */
        execlp("/bin/ls","ls",NULL);
    }
    else { /* parent process */
        /* parent will wait for the child to complete */
        wait(NULL);
        printf("Child Complete");
```

Il processo termina con la chiamata di sistema **exit()**, restituisce un valore di stato al processo genitore, via **wait()** e le risorse sono de-allocate dal sistema operativo oppure il genitore può terminare l'esecuzione del figlio con la chiamata di sistema abort(), magari per un eccesso di risorse allocate o per un compito non più richiesto, il pid però non scompare subito, il processo rimane in stato di **zombie** e scompare solo dopo che il genitore ha eseguito il wait(), può succedere che il genitore termini senza che vengano terminati i suoi processi figli, in tal caso quei processi si dicono **orfani.**

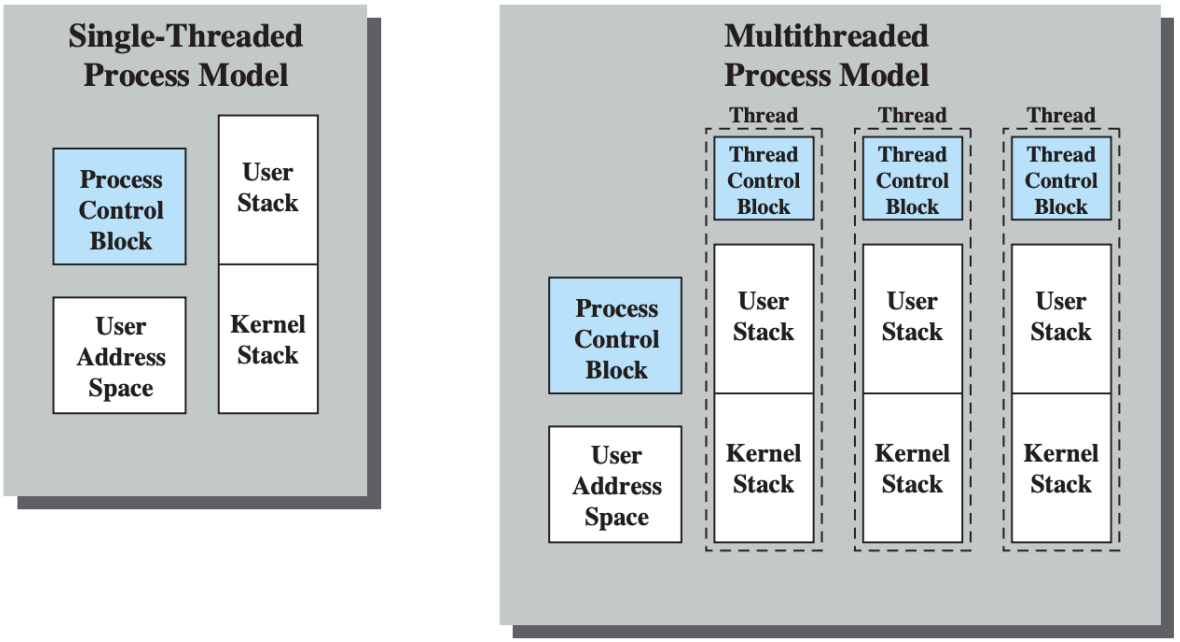
THREADS

I processi sono caratterizzati dalle **risorse allocate** (l'immagine in memoria, la RAM assegnata, i dispositivi I/O) e lo **scheduling** (la traccia di esecuzione, lo stato e la priorità).
Queste due caratteristiche però possono essere trattate in maniera indipendente, il processo **(task)** è quella parte che possiede le risorse, il **thread** (o processo leggero) è l'unità che viene assegnata al processore.
Il **multi-threading** è la capacità di un sistema operativo di permettere più tracce di esecuzione concorrenti in un singolo processo, quindi ad un processo o più processi vengono assegnati più di un singolo thread ciascuno, (In quel caso sarebbe **single-threading**)**.**
In un ambiente **multithread,** il processo è l'unità di allocazione delle risorse e un'unità di protezione, sono associati ad esso:

- lo **spazio di indirizzamento virtuale**, che contiene l'immagine del processo;
- l'accesso protetto al processore, ad altri processi, a files e dispositivi I/O;
- uno o più **thread**.

Al thread invece sono associati:

- lo **stato** (pronto, attesa, esecuzione);
- Il contesto del thread (**immagine del processore**);
- Lo **stack** di esecuzione;
- Lo spazio di memoria **statico** per le variabili locali;
- L'accesso a memoria e risorse del processo, condivise con gli altri thread;



I benefici del multi-threading sono molteplici:

- La creazione di un thread è più rapida della creazione di un processo, così come la sua terminazione;
- Lo switch tra due thread dello stesso processo è più veloce dello switch tra due processi;
- L'efficienza della comunicazione tra programmi.

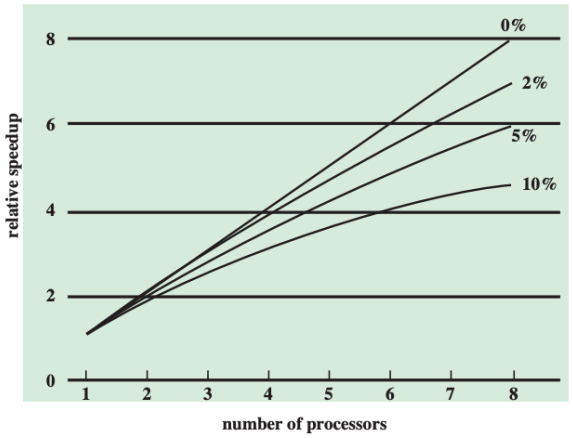
Nei sistemi a utente singolo consentono l'esecuzione in foreground e background dei processi, la loro elaborazione asincrona, maggiore velocità di esecuzione.
Si nota che la schedulazione è gestita principalmente nel thread, ma non la sospensione e la terminazione del processo.
Gli stati del thread sono gli stessi dei processi, così come le operazioni (creazione, block, unblock, terminazione).
I thread di un processo **condividono spazio di indirizzamento e altre risorse**, ogni modifica delle risorse influenza gli altri thread, il problema è che la modifica simultanea di un dato da parte di due thread può corrompere il dato stesso. È quindi necessario sincronizzare l'accesso dei thread alle risorse.
Thread a livello utente: la gestione dei thread è fatta dall'applicazione, il kernel non sa dell'esistenza dei thread, quindi lo scambio di thread non richiede privilegi di modalità kernel, la schedulazione inoltre può essere specifica per l'applicazione che è anche portabile.
Gli svantaggi sono ad esempio che l'uso di system call blocca l'intero processo (per risolvere questo si usa il **jacketing**, si trasforma una chiamata di sistema bloccante in una non bloccante), inoltre non si trae vantaggio dai

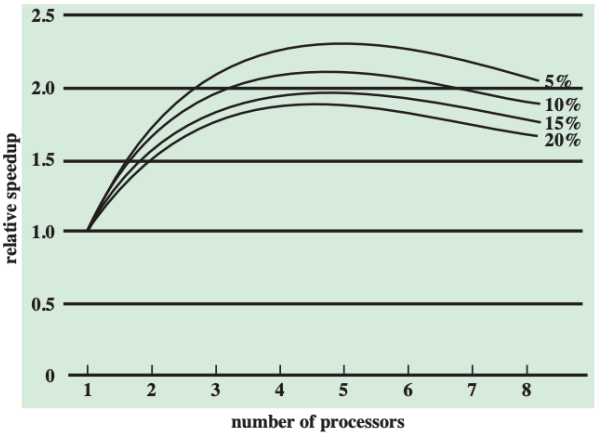
multiprocessori (per risolvere questo si può programmare un'applicazione in processi multipli).
Thread a livello kernel: La gestione dei thread è fatta dal kernel, l'applicazione usa un API per gestire i thread.
Il kernel quindi gestisce i PCB dei processi e dei thread e la schedulazione è quella del processore.
Si possono quindi eseguire thread differenti su diversi processori, il blocco di un thread non blocca il processo e si possono gestire le routine del kernel in multi-threading. I cambi di modalità utente/kernel però producono overhead.
Si possono usare quindi degli approcci misti:

- Creazione dei thread, schedulazione e sincronizzazione nello spazio utente
- Thread modalità utente mappati su thread modalità kernel (in numero minore o uguale)
- Permette esecuzione parallela dei thread e il blocco di un thread non blocca il processo
  - Solaris

LEGGE DI AMDAHL

$$\text{speedup} = \frac{\text{tempo di esecuzione su un processore}}{\text{tempo di esecuzione su } N \text{ processori}} = \frac{1}{(1-f) + \frac{f}{n}}$$





Si avvantaggiano del **multicore/multithreading**:

- Applicazioni multithread native
  - pochi processi con molti thread
- Applicazioni multiprocesso
  - molti processi single-thread
- Applicazioni Java
  - JVM è un processo multithread e assicura schedulazione e gestione della memoria per le applicazioni Java
- Multiple istanze di un'applicazione

In UNIX, **il kernel non distingue tra processi e thread**, per ogni **task** (che può essere o un processo o un thread), c'è un **task control block**, con **fork()** si crea una nuova task, con una copia del task control block, **clone()** invece crea una nuova task, ma nel TBC ci sono puntatori ai campi di quello del genitore (in base ai parametri passati a clone())

MUTUA ESCLUSIONE

I processi possono interferire tra di loro in maniera positiva o negativa, negativa ad esempio quando sottrae risorse ad altri processi, in maniera positiva quando due o più processi interagiscono e ci permettono di realizzare un'attività che sfrutta l'uso di diversi processi in un'unica applicazione.
La gestione dei processi può avvenire tramite:

- **Multiprogrammazione,** molti processi con un singolo processore, i processi avanzano attendendo che il processore si liberi;
- **Multiprocessing,** molti processi gestiti da un multiprocessore;
- **Processi distribuiti,** molti processi, gestiti da molti processori distribuiti (**cluster**), con memorie distribuite, rende le cose più complicate da gestire, ma sicuramente più veloci.

I differenti processi hanno bisogno di risorse che tipicamente vengono condivise tra i vari processi, si crea quindi una **concorrenza**, ad esempio nelle applicazioni multiple, o nelle applicazioni strutturate (con più processi o threads che devono accedere a delle risorse), la modifica dei dati disponibili da parte di un processo influenza l'andamento di un altro processo, oppure anche nella struttura del sistema operativo.
Nel caso dell'**uniprocessore multiprogrammato** l'esecuzione dei processi si alterna, interrompendo certi processi e riprendendone altri, nel **multiprocessore** invece le esecuzioni si alternano e si sovrappongono, questi

processi detti **concorrenti** magari ad un certo punto dovranno accedere ad una risorsa e non si può prevedere quale processo specifico entrerà per primo nella risorsa, poiché la velocità di esecuzione dei processi dipende da molti fattori (attività degli altri processi, gestione delle interruzioni, politiche di scheduling). Questo comporta un problema oggettivo nella condivisione delle risorse globali, rende difficile allocare le risorse nella maniera più opportuna e

```
void echo()
{
   chin = getchar();
   chout = chin;
   putchar(chout);
}
```

Nell'uniprocessore:

```
Processo P1              Processo P2          Esito:
     •                        •               stampa due volte
chin = getchar();             •               il secondo
  <prerilascio>          <esecuzione>         carattere letto
     •                chin = getchar();
     •                chout = chin;
     •                putchar(chout);
   <esecuzione>         <prerilascio>
chout = chin;                 •
putchar(chout);               •
```

al momento del debugging può comparire un errore specifico dovuto a una serie di eventi sfortunati che si verifica raramente e sarà difficile da ri-individuare.
Cioè l'uniprocessore prende il processo P1, esegue chin=getchar(), sospende l'esecuzione di P1 passando a P2, rifà getchar() e cambia chin, stampa chout, poi si ritorna al processo P1, dove chout = chin, ma chin è stato cambiato da P2, perciò verrà stampato nuovamente il secondo carattere.
Se un solo processo alla volta potesse eseguire la procedura.

```
Processo P1              Processo P2          Esito:
     •                        •               stampa prima il
chin = getchar();             •               carattere richiest
  <prerilascio>          <esecuzione>         da P1, poi quello
     •                   <sospensione>        richiesto da P2
     •                        •
  <esecuzione>                •
chout = chin;                 •
putchar(chout);          <attivazione>
     •                        •
  <sospensione>          <esecuzione>
     •                chin = getchar();
     •                chout = chin;
     •                putchar(chout);
     •                        •
```

P2 va in stato di block perchè la risorsa (in questo caso la procedura) non è disponibile poiché usata da un altro processo, P2 viene quindi sospeso, la computazione va avanti finché viene ripreso P1, viene conclusa la procedura e si ritorna a P2 con la risorsa ora disponibile, i due caratteri ora sono stampati in ordine.
Anche nel multiprocessore ci sarebbe questo problema:

```
void echo()
{
   chin = getchar();
   chout = chin;
   putchar(chout);
}
```

Nel multiprocessore:

Processo P1	Processo P2	Esito:
•	•	stampa due volte
chin = getchar();	•	il secondo
•	chin = getchar();	carattere letto
chout = chin;	chout = chin;	
putchar(chout);	•	
•	putchar(chout);	
•	•	

Perciò in sostanza quando ci sono variabili condivise è necessario **proteggerle,** si deve controllare il codice che accede alla variabile.
Quando parliamo di concorrenza e dati condivisi si parla anche di **race condition**, più di un processo accede a dati e il risultato finale dipende dall'ordine di esecuzione delle istruzioni sui dati, che è ciò che avveniva negli esempi di prima, se i processi P1 e P2 assegnano rispettivamente 1 e 2 alla variabile condivisa, il valore finale sarà quello del secondo arrivato.
Esempio:
I processi P1 e P2 condividono le variabili b e c con valori iniziali $b = 1$, $c = 2$. P1 esegue $b = b + c$, P2 esegue $c = b + c$.
Se arriva primo P1, avremo $b = 3$, $c = 5$. Nel caso opposto, $b = 4$, $c = 3$.

La concorrenza quindi può portare a problemi, è compito del sistema operativo fare in modo che la concorrenza si svolga nella maniera corretta:

- Deve tenere traccia dei processi (col Process Control Block);
- Allocare e deallocare risorse per i processi attivi (tempo di elaborazione, memoria, files, dispositivi I/O);
- Proteggere dati e risorse da interferenze involontarie di altri processi;

- Il risultato di ogni processo deve essere indipendente dalla sua velocità relativa a quella dei processi concorrenti.

Degree of Awareness	Relationship	Influence that One Process Has on the Other	Potential Control Problems
Processes unaware of each other	Competition	• Results of one process independent of the action of others • Timing of process may be affected	• Mutual exclusion • Deadlock (renewable resource) • Starvation
Processes indirectly aware of each other (e.g., shared object)	Cooperation by sharing	• Results of one process may depend on information obtained from others • Timing of process may be affected	• Mutual exclusion • Deadlock (renewable resource) • Starvation • Data coherence
Processes directly aware of each other (have communication primitives available to them)	Cooperation by communication	• Results of one process may depend on information obtained from others • Timing of process may be affected	• Deadlock (consumable resource) • Starvation

-

Cosa cambia quindi quando abbiamo processi indipendenti e processi cooperanti?
La relazione più semplice è la **competizione**, i processi non sanno nulla (unaware) degli altri processi in corso, il risultato di un processo deve quindi essere indipendente dall'azione degli altri, ma il tempo di elaborazione potrebbe essere influenzato poiché una risorsa potrebbe non essere disponibile (come prima quando si blocca l'accesso a una procedura già usata) e si deve attendere che si liberi, i problemi di controllo possibili possono essere la **mutua esclusione,** cioè si impedisce l'accesso alla stessa risorsa da parte di più di un processo, la **deadlock**, cioè tanti processi bloccati perché richiedono le stesse risorse già usate da altri processi e quindi al momento inaccessibili, la **starvation,** un processo ha bisogno di una risorsa, ma gli altri processi la prendono prima di lui e esso non riesce mai ad accedere alla risorsa.

Mutua esclusione:

```
     /* PROCESS 1 */
void P1
{
  while (true) {
    /* preceding code */;
    entercritical (Ra);
    /* critical section */;
    exitcritical (Ra);
    /* following code */;
  }
}
```

```
     /* PROCESS 2 */
void P2
{
  while (true) {
    /* preceding code */;
    entercritical (Ra);
    /* critical section */;
    exitcritical (Ra);
    /* following code */;
  }
}
```

Una situazione più complessa è una **collaborazione tramite la condivisione di risorse**, in questo caso i processi non si conoscono, ma condividono dati, il risultato di un processo può dipendere dalle azioni di un altro e anche qui i tempi di elaborazione ne sono influenzati, i problemi sono sempre gli stessi, la **mutua esclusione,** poichè il meccanismo di controllo deve assicurare l'integrità dei dati (tranne per le operazioni di lettura)**,** la **deadlock** e la **starvation**, ma anche un problema di **data coherence**, i processi modificano dei dati condivisi e se uno dei processi sbaglia a modificare i dati essi diventano incoerenti e portano a errori anche gli altri processi.
Situazione ancora più complessa è l'**interazione tra processi (collaborazione per comunicazione),** ci sono delle primitive che consentono a un processo di mandare informazioni a un altro processo o di riceverle fornite dal kernel o dal linguaggio di programmazione, quindi in questo caso il risultato è sicuramente influenzato da ciò che il processo riceve, così come il tempo di elaborazione che è limitato a quando il processo riceve l'informazione necessaria, in questo caso non c'è problema di **mutua esclusione** poiché i processi non condividono dati, si hanno però sempre problemi di **deadlock e starvation.**
Quindi per la mutua esclusione dobbiamo far sì che un solo processo alla volta è ammesso alla sezione critica per ciascuna risorsa condivisa, un processo che si ferma fuori dalla sua sezione critica non deve interferire con altri processi, si deve evitare deadlock e starvation, quindi se nessun processo è nella sezione critica, deve essere concesso a ogni processo di entrare senza ritardo.

Per assicurare che un processore non possa uscire da una sezione critica di un processo su uniprocessore si possono **disabilitare le interruzioni** tramite delle primitive definite dal kernel, ciò però **rallenta il sistema ed è inapplicabile per multiprocessori**:

```
while (true) {
    /* disabilita interruzioni */;
    /* sezione critica */;
    /* abilita interruzioni */;
    /* resto del programma */;
}
```

In alternativa si possono usare delle **istruzioni speciali**, un solo processore alla volta può accedere a una locazione di memoria, con speciali istruzioni macchina che eseguono due operazioni su un dato in maniera atomica.

Ad esempio la **Compare and swap.**

```
int compare_and_swap ( int *word, int testval, int newval)
{
    int oldval;
    oldval = *word
    if (oldval == testval) *word = newval;
    return oldval;
}

/* program mutualexclusion */
const int n = /* number of processes */;
int bolt;
void P( int i)
{
   while (true) {
      while (compare_and_swap(bolt, 0, 1) == 1)
         /* do nothing */;
      /* critical section */;
      bolt = 0;
      /* remainder */;
   }
}

void main()
{
   bolt = 0;
   parbegin (P(1), P(2), ... ,P(n));
}
```

Un altro tipo di istruzione speciale è la exchange, una procedura che scambia il valore di 2 variabili.

```
/* program mutualexclusion */
int const n = /* number of processes */;
int bolt;
void P( int i)
{
   int keyi = 1;
   while (true) {
      do exchange (&keyi, &bolt)
      while (keyi != 0);
      /* critical section */;
      bolt = 0;
      /* remainder */;
   }
}

void main()
{
  bolt = 0;
  parbegin (P(1), P(2), ..., P(n));
}
```

Vantaggi e svantaggi delle istruzioni macchina speciali:

Vantaggi

- applicabile a qualsiasi numero di processi su uniprocessore o multiprocessore
- semplicità
- può supportare multiple sezioni critiche

Svantaggi

- attesa attiva (busy-waiting)
- possibilità di stallo
- possibilità di starvation

ALGORITMO DI DEKKER

L'obiettivo è permettere a due processi di assicurare la mutua esclusione:

- senza supporto hardware (salvo impossibilità di accesso simultaneo allo stesso indirizzo di memoria);
- senza stallo;
- senza starvation.

Primo tentativo:

```
/* PROCESS 0 /*
...
while (turn != 0)
   /* do nothing */ ;
/* critical section*/;
turn = 1;
...
```

```
/* PROCESS 1 /*
...
while (turn != 1)
   /* do nothing */ ;
/* critical section*/;
turn = 0;
...
```

Apparentemente questo primo approccio funziona, c'è però un problema, supponiamo che il processo 0 esegua la sua sezione critica e che poi setti turn = 1, ma poi process 0 va in attesa, magari si vuole eseguire la sezione critica solo una volta, ma così le esecuzioni dei due processi si alternano.
Secondo tentativo:

```
/* PROCESS 0 */
...
while (flag[1])
  /* do nothing */;
flag[0] = true;
/*critical section*/;
flag[0] = false;
...
```

```
/* PROCESS 1 */
...
while (flag[0])
  /* do nothing */;
flag[1] = true;
/*critical section*/;
flag[1] = false;
...
```

Si usano due flag che dovrebbero segnalare che la sezione critica sta venendo eseguita da uno dei due processi, questo non funziona poiché se i due processi arrivano quasi contemporaneamente con entrambe le flag false, entrano entrambi nella sezione critica poiché pensano che essa sia libera.

Terzo tentativo:

```
/* PROCESS 0 */
...
flag[0] = true;
while (flag[1])
  /* do nothing */;
/* critical section*/;
flag[0] = false;
...
```

```
/* PROCESS 1 */
...
flag[1] = true;
while (flag[0])
  /* do nothing */;
/* critical section*/;
flag[1] = false;
...
```

Scambiamo l'ordine delle operazioni di scrittura e lettura, prima occupo, poi aspetto che l'altra sia liberata, ma anche qui se i due processi partissero quasi insieme le due variabili sarebbero inizializzate entrambe a true, così entrambi i loop while sarebbero infiniti e i due processi non avanzeranno mai, è un esempio palese di stallo.
Quarto tentativo:

```
/* PROCESS 0 */
...
flag[0] = true;
while (flag[1]) {
  flag[0] = false;
  /*delay */;
  flag[0] = true;
}
/*critical section*/;
flag[0] = false;
...
```

```
/* PROCESS 1 */
...
flag[1] = true;
while (flag[0]) {
  flag[1] = false;
  /*delay */;
  flag[1] = true;
}
/*critical section*/;
flag[1] = false;
...
```

Se un processo rileva che un altro processo ha flag true blocca se stesso, poi fa un delay che permette all'altro processo di avanzare e di entrare nella sezione critica per poi impostare flag a false.

La soluzione corretta:

```
boolean flag [2];
int turn;

void P0()                                          void P1()
{                                                  {
   while (true) {                                    ...
      flag [0] = true;
      while (flag [1]) {
         if (turn == 1) {
            flag [0] = false;
            while (turn == 1) /* do nothing */;
            flag [0] = true;
         }
      }
      /* critical section */;
      turn = 1;
      flag [0] = false;
      /* remainder */;
   }
}
```

Con questa soluzione però paghiamo l'attesa attiva che è un costo non indifferente.
L'algoritmo di Dekker si può scrivere in maniera più elegante con l'algoritmo di Peterson:

```
boolean flag [2];
int turn;

void P0()
{
   while (true) {
      flag [0] = true;
      turn = 1;
      while (flag [1] && turn == 1) /* do nothing */;
      /* critical section */;
      flag [0] = false;
      /* remainder */;
   }
}

void P1() { . . . }
```

SEMAFORO

I semafori sono i meccanismi utilizzati per sincronizzare l'accesso alle sezioni critiche dei processi.

SEMAFORO GENERICO

Formati da una variabile con un valore intero, una coda e tre operazioni, inizializzato a un valore non negativo, l'operazione **semWait** decrementa il valore del semaforo, se il valore diventa negativo, il processo è sospeso e inserito nella coda, **semSignal** invece incrementa il valore del semaforo, se il valore non diventa positivo, uno dei processi della coda è riattivato.

Per la situazione di prima potremo utilizzare un semaforo inizializzato a 1, con la prima semWait il valore diventa 0, così tutti gli altri processi che tenteranno di accedere facendo la semWait renderanno il valore -1, sospendendo il processo e inserendolo nella coda, quando il processo iniziale manderà un semSignal si tornerà a un valore positivo e il valore precedentemente in coda entrerà nella sezione critica.

In pratica quando la variabile è positiva si indica quanta disponibilità c'è, quanti processi possono accedere, quando è negativo mi dice quanti processi sono in coda in attesa di entrare nella sezione critica.

Osservazione: Il processo non ha modo di sapere in anticipo se con l'esecuzione di semWait sarà sospeso, se c'è un processo che attende l'esecuzione di semSignal, quale fra i processi in attesa sarà riattivato da semSignal (di solito i processi vengono accodati, quindi ordine First In First Out).

Semaforo generico

```
struct semaphore {
   int count;
   queueType queue;
};

void semWait(semaphore s)
{
   s.count--;
   if (s.count < 0) {
      /* place this process in s.queue */;
      /* block this process */;
   }
}

void semSignal(semaphore s)
{
   s.count++;
   if (s.count<= 0) {
      /* remove a process P from s.queue */;
      /* place process P on ready list */;
   }
}
```

SEMAFORO BINARIO

La variabile ora è semplicemente un valore binario, le tre operazioni:

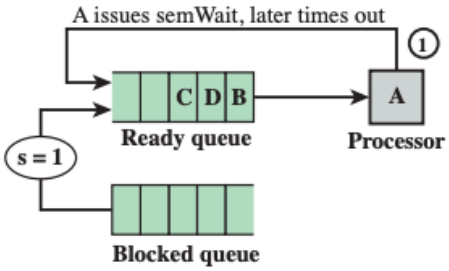
- Inizializzazione a zero o uno;
- semWaitB verifica il valore del semaforo, se è 1 allora è portato a 0, altrimenti il processo è sospeso e inserito nella coda;
- semSignalB verifica se ci sono processi in attesa nella coda. In caso affermativo, uno dei processi della coda è riattivato, nel caso opposto il valore del semaforo è portato a 1.

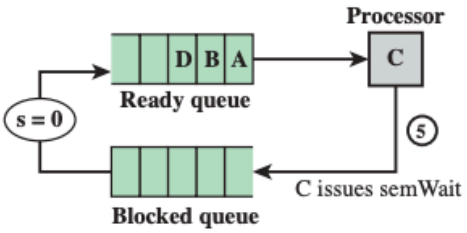
Così facendo ora non posso dire che ci possono essere X programmi running e X in coda, ma il programmatore deve gestire la coda, ad esempio incrementando un contatore di processi in coda ogni volta che si esegue la semSignal, si deve quindi evitare la starvation.

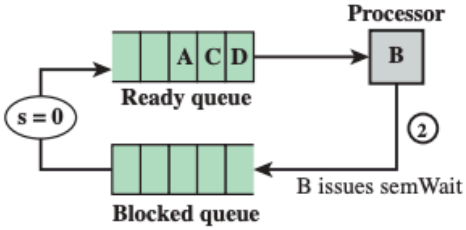
Semaforo binario

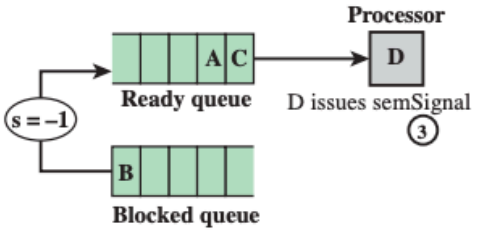
```
struct binary_semaphore {
   enum {zero, one} value;
   queueType queue;
};
void semWaitB(binary_semaphore s)
{
   if (s.value == one)
      s.value = zero;
   else {
      /* place this process in s.queue */;
      /* block this process */;
   }
}
void semSignalB(semaphore s)
{
   if (s.queue is empty())
      s.value = one;
   else {
      /* remove a process P from s.queue */;
      /* place process P on ready list */;
   }
}
```

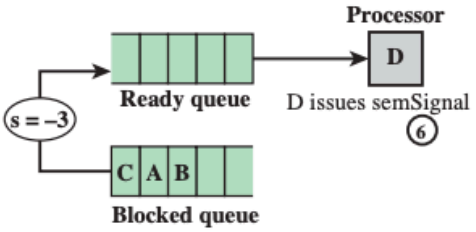
Come funziona un semaforo:à

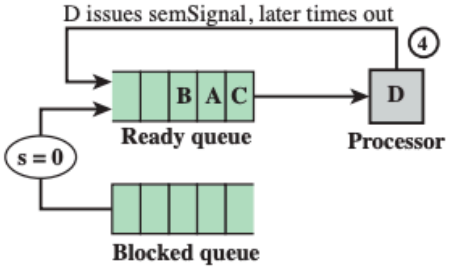

A issues semWait, later times out
1
C D B
A
Ready queue
Processor
s = 1
Blocked queue

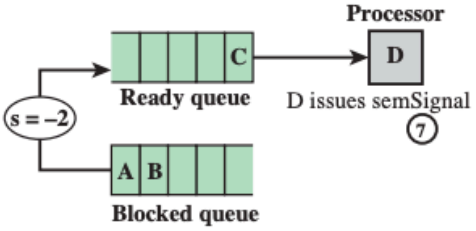

Processor
D B A
C
Ready queue
s = 0
5
C issues semWait
Blocked queue


Processor
A C D
B
Ready queue
s = 0
2
B issues semWait
Blocked queue


Processor
A C
D
Ready queue
D issues semSignal
3
s = –1
B
Blocked queue


Processor
D
Ready queue
D issues semSignal
6
s = –3
C A B
Blocked queue


D issues semSignal, later times out
4
B A C
D
Ready queue
Processor
s = 0
Blocked queue

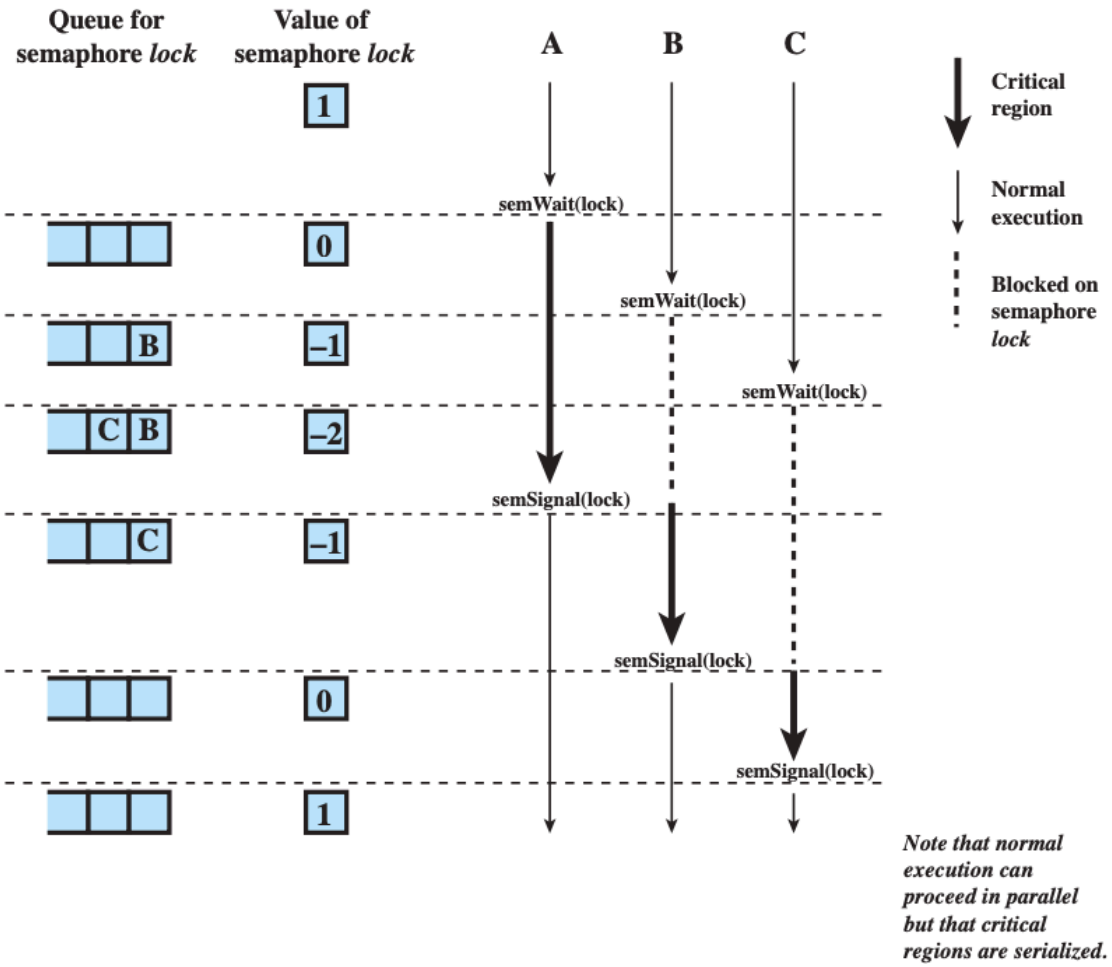

Processor
C
D
Ready queue
D issues semSignal
7
s = –2
A B
Blocked queue

```
/* program mutualexclusion */
const int n = /* number of processes */;
semaphore s = 1;

void P(int i)
{
   while (true) {
      semWait(s);
      /* critical section */;
      semSignal(s);
      /* remainder */;
   }
}

void main()
{
   parbegin (P(1), P(2), ..., P(n));
}
```

Si usa nella programmazione parallela.



semWait e semSignal devono essere primitive atomiche, con soluzioni hardware o firmware, o istruzioni speciali, o con l'algoritmo di Peterson.
L'alternativa alla condivisione di dati è lo scambio di **messaggi**, un meccanismo molto più sicuro poiché i processi non accedono allo spazio degli altri processi.

MESSAGGI

Anche il meccanismo dei messaggi consente la sincronizzazione per realizzare la mutua esclusione e la comunicazione per lo scambio di informazioni tra processi.
Perché utilizzare i messaggi invece della condivisione di memoria? In sistemi distribuiti i processi hanno difficoltà a leggere la memoria di altri processi in sistemi diversi, perciò il loro utilizzo è naturale nei sistemi distribuiti, ma anche nei sistemi a memoria condivisa questo meccanismo è utile.
Hanno due primitive:

- **send(destination, message)**, un processo invia informazioni in forma di messaggio a un altro processo designato come destinatario;

- **receive (source, message)**, un processo riceve informazioni indicando il mittente e l'indirizzo in cui salvare il messaggio.

La sincronizzazione per la send è bloccante se il processo a cui si vuole mandare non attende un messaggio, quindi si blocca l'esecuzione del send, è non bloccante quando il messaggio viene mandato correttamente, il receive allo stesso modo è bloccante quando si tenta di ricevere un messaggio indiretto senza che sia stato ancora mandato, non bloccante quando il messaggio già c'è, oppure si testa per la presenza di messaggi.
Per la send tipicamente l'indirizzamento è **diretto**, il processo specifica a quale processo inviare il messaggio, la receive può avere un indirizzamento **esplicito** (dal processo X prelevo il messaggio) o implicito, quando il processo esegue una receive e se c'è un messaggio per questo processo prende il messaggio e l'identificativo del processo che l'ha mandato.
Il formato dei messaggi può essere di lunghezza fissa o variabile, inoltre l'organizzazione delle code dei messaggi è FIFO, oppure può esserci una gestione tramite priorità.

SINCRONIZZAZIONE

Il destinatario non può ricevere il messaggio prima che sia stato inviato, quando si esegue la receive:

- Se c'è un messaggio, il processo lo riceve e continua l'esecuzione (non bloccante);
- In caso contrario, due possibilità:
    1. Si sospende finchè il messaggio non è inviato;
    2. Continua l'esecuzione senza più ricevere il messaggio.

Quando si ha un mittente e un destinatario bloccanti, essi lo sono fino al completamento dell'operazione, finchè poi si incontrano e vengono sincronizzati i processi.
Quando si ha un send non bloccante e un receive bloccante, il mittente procede, ma il destinatario è bloccato fino alla ricezione del messaggio, è la combinazione più comune poiché ad esempio il mittente invia più messaggi a tanti destinatari senza perdita di tempo (esempio: processo server che fornisce servizi a più processi).
Quando si ha un send non bloccante e un receive non bloccante semplicemente nessuno dei due processi attende.

INDIRIZZAMENTO

Diretto:

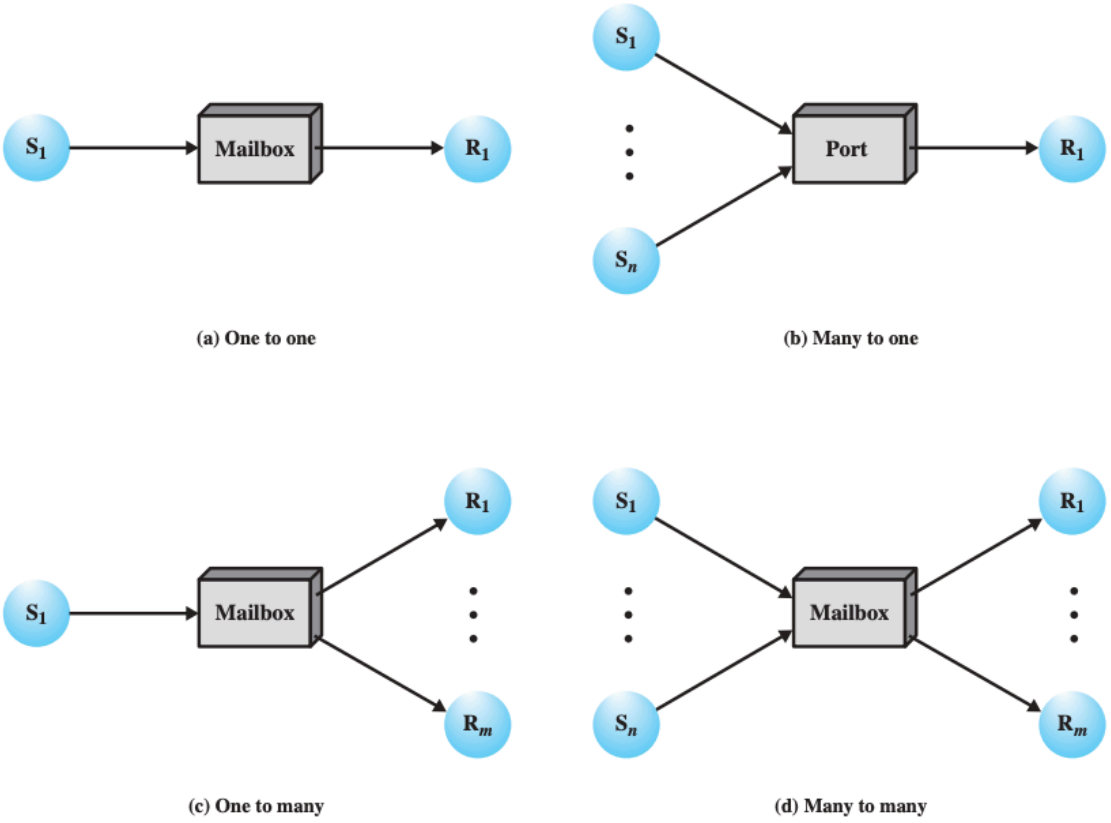
Come detto la primitiva send deve includere **l'identificatore** del processo destinatario.

La receive invece può o specificare il mittente (utile per processi concorrenti cooperanti), oppure può usare l'indirizzamento implicito, quindi ricevere un messaggio in generale e il parametro **sender** conterrà l'identificatore del mittente.

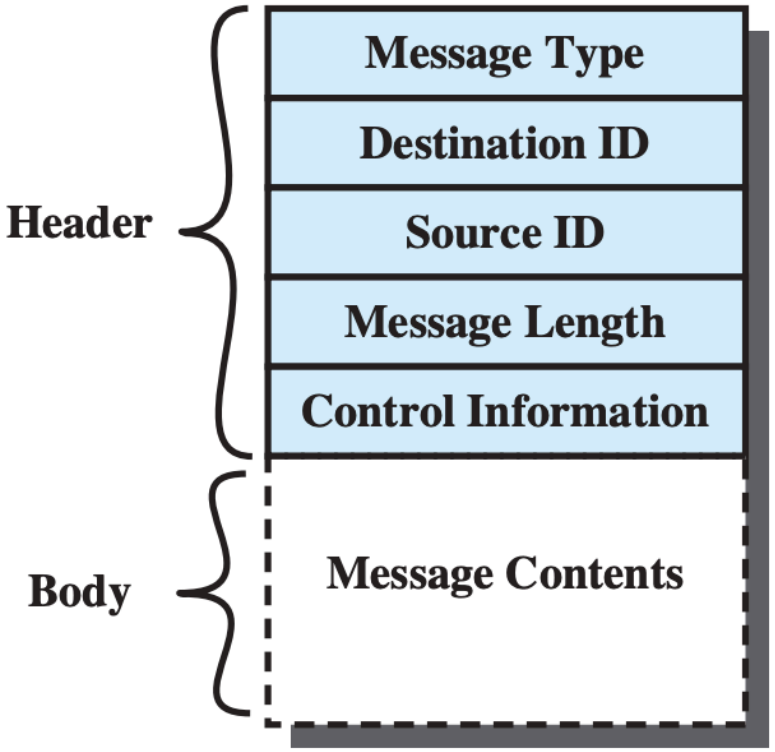
Indiretto:

I messaggi vengono inviati a una struttura dati condivisa fatta di code che contengono temporaneamente i messaggi (**mailbox**)**,** quindi il mittente invia un messaggio a una mailbox, il destinatario la preleva.

Indirizzamento indiretto.



FORMATO DEL MESSAGGIO



GESTIONE DELLE CODE

Il meccanismo più semplice è il FIFO, il primo messaggio in coda viene trasmesso, oppure si può fare una coda prioritaria per tipo di messaggio o designata dal mittente, alternativamente il destinatario potrebbe ispezionare la coda e scegliere un messaggio particolare.

Avendo quindi a disposizione una receive bloccante, la mutua esclusione diventa molto più semplice degli algoritmi visti in precedenza, si crea una mailbox e si inviano ad essa dei messaggi, i processi prima di iniziare la sezione critica eseguono un receive, se c'è il messaggio nella mailbox entrano nella sezione critica e terminata la sezione mandano un messaggio alla mailbox, se un processo esegue la receive mentre l'altro processo è nella sezione critica troverà la mailbox vuota e perciò si bloccherà finchè l'altro processo non uscirà dalla sezione critica e manderà un messaggio nella mailbox.

```
/* program mutualexclusion */
const int n = /* number of process */
void P(int i)
{
    message msg;
    while (true) {
        receive (box, msg);
        /* critical section */;
        send (box, msg);
        /* remainder */;
    }
}

void main()
{
    create mailbox (box);
    send (box, null);
    parbegin (P(1), P(2),..., P(n));
}
```

PROBLEMA PRODUTTORE/CONSUMATORE

Ci sono processi che consumano e processi che producono, ho uno o più produttori che inseriscono dati in un buffer, mentre un consumatore estrae i dati dal buffer, uno alla volta, un solo produttore o consumatore alla volta può accedere al buffer.

Bisogna quindi assicurare la mutua esclusione ed evitare che un produttore aggiunga dati a un buffer pieno e che un consumatore cerchi di estrarre dati da un buffer vuoto.
Come si può procedere? Soluzione errata (dato un buffer infinito)

```
/* program producerconsumer */
int n;
binary_semaphore s = 1, delay = 0;

void producer()
{
  while (true) {
    produce();
    semWaitB(s);
    append();
    n++;
    if (n==1) semSignalB(delay);
    semSignalB(s);
  }
}
```

```
void consumer()
{
  semWaitB(delay);
  while (true) {
    semWaitB(s);
    take();
    n--;
    semSignalB(s);
    consume();
    if (n==0) semWaitB(delay);
  }
}
```

Abbiamo un semaforo binario che esegue la wait per realizzare la mutua esclusione, append() scrive l'evento prodotto nel buffer, poi n++ incrementa il contatore del numero di elementi nel buffer, se n == 1, esegue un signal sull'altro semaforo, che prenderà la risorsa, decrementerà il contatore n e segnalerà con delay che il buffer è vuoto. Questo non funziona perchè terminato semSignalB(s) il produttore potrebbe aver già prodotto qualcos'altro, quindi blocca l'accesso ad altri processi, n non sarà zero, il problema è che la semWaitB() non può stare nel while true, ma altrimenti non funzionerebbe.

Soluzione corretta (sempre dato un buffer infinito)

```
/* program producerconsumer */
int n;
binary_semaphore s = 1, delay = 0;

void producer()
{
  while (true) {
    produce();
    semWaitB(s);
    append();
    n++;
    if (n==1) semSignalB(delay);
    semSignalB(s);
  }
}
```

```
void consumer()
{
  int m; /* a local variable */
  semWaitB(delay);
  while (true) {
    semWaitB(s);
    take();
    n--;
    m = n;
    semSignalB(s);
    consume();
    if (m==0) semWaitB(delay);
  }
}
```

La procedura del producer non cambia, il consumer invece crea m, copia in m il contatore n, consuma la risorsa e poi il check viene fatto non con n che potrebbe essere stato modificato da producer, ma con m che ha salvato il valore di n prima che il consumer abbia consumato la risorsa, così il problema è risolto.
Nel caso reale con un buffer finito e l'uso di semafori generici:
Il produttore produce, poi fa wait sul semaforo e, che prenderà valori negativi, quando il buffer sarà pieno, e wait su s per la mutua esclusione sull'accesso al buffer, poi fa il signal a s per segnalare che ora il buffer è libero e segnala a n, incrementa quindi n di 1 dicendo che ora c'è un elemento in più nel buffer, il consumer invece fa esattamente il contrario, facendo lo wait su n che se positivo indica che ci sono degli elementi nel buffer, altrimenti il consumer si mette in coda, se c'è la risorsa prende la risorsa, segnala che ora e si è

liberato di un elemento.

```
/* program boundedbuffer */
const int sizeofbuffer = /* buffer size */;
semaphore s = 1, n= 0, e= sizeofbuffer;

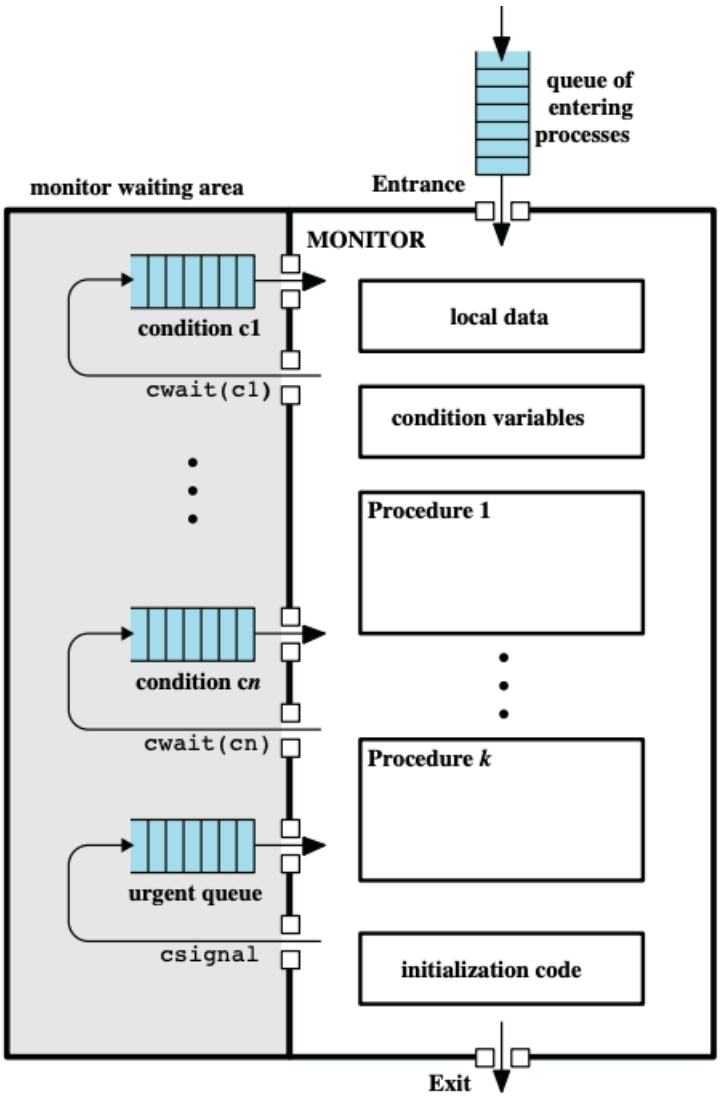
void producer()
{
  while (true) {
    produce();
    semWait(e);
    semWait(s);
    append();
    semSignal(s);
    semSignal(n);
  }
}
```

```
void consumer()
{
  while (true) {
    semWait(n);
    semWait(s);
    take();
    semSignal(s);
    semSignal(e);
    consume();
  }
}
```

MONITOR

Quando si hanno programmi complessi si possono utilizzare dei programmi costruiti apposta per la concorrenza, questi programmi hanno degli oggetti fatti apposta per la gestione dei dati condivisi, il monitor è uno dei costrutti usati per la programmazione concorrente, è un modulo software usato per gestire i dati condivisi, definisco in esso le operazioni da fare su questi dati condivisi, il resto dei programmi che usano la concorrenza sono chiamate alle procedure definite nel modulo. I singoli processi quindi non toccano direttamente i dati condivisi, ma devono passare per le procedure del modulo. I dati condivisi quindi sono accessibili **solo tramite le procedure del monitor,** i codici dei processi possono accedervi solo tramite le procedure, inoltre un solo processo alla volta può eseguire le procedure del monitor.
All'interno del monitor stanno le **variabili di condizione**, variabili accessibili solo all'interno del monitor attraverso due funzioni:

- **cwait(c)**, sospende l'esecuzione del processo che l'ha eseguita sulla condizione c;
- **csignal(c)**, riattiva uno dei processi in attesa sulla condizione c, se non ce ne sono non fa nulla.


queue of
entering
processes
monitor waiting area
Entrance
MONITOR
condition c1
cwait(c1)
local data
condition variables
Procedure 1
condition cn
cwait(cn)
Procedure k
urgent queue
csignal
initialization code
Exit

Problema produttore/consumatore

```
/* program producerconsumer */
monitor boundedbuffer;
char buffer [N];          /* space for N items
int nextin, nextout;      /* buffer pointers
int count;                /* number of items in buffer
cond notfull, notempty;   /* condition variables for synchronization

void append (char x)
{
   if (count == N) cwait(notfull);    /* buffer is full; avoid overflow */
   buffer[nextin] = x;
   nextin = (nextin + 1) % N;
   count++;                           /* one more item in buffer */
   csignal(notempty);                 /* resume any waiting consumer */
}
void take (char x)
{
   if (count == 0) cwait(notempty);  /* buffer is empty; avoid underflow */
   x = buffer[nextout];
   nextout = (nextout + 1) % N;
   count--;                          /* one fewer item in buffer */
   csignal(notfull);                 /* resume any waiting producer */
}
{                                       /* monitor body */
   nextin = 0; nextout = 0; count = 0;  /* buffer initially empty */
}
```

PROBLEMA DEI LETTORI E SCRITTORI

Alcuni processi leggono soltanto (**lettori**), altri scrivono soltanto (**scrittori**), imponiamo come condizioni affinchè tutto vada bene che:

- Qualsiasi numero di lettori può leggere simultaneamente i dati;
- Un solo scrittore alla volta può scrivere i dati;
- Durante la scrittura nessun lettore può leggere.

Ci sono due possibili strade:

Dare precedenza ai lettori:

Si ha un contatore, due semafori, il semaforo x per la mutua esclusione, e un semaforo wsem per controllare l'accesso degli scrittori. Se readcount == 1, cioè se c'è un lettore esegue una wait sul semaforo wsem. Fatto ciò esco dalla mutua esclusione. La x in sostanza serve per la mutua esclusione sulla variabile readcount, devo incrementare readcount, e per fare questa operazione ho tre istruzioni elementari, cioè leggi readcount, aggiungici 1 e aggiorna. A questo punto il lettore legge e quando ha terminato decrementa readcount (la variabile in mutua esclusione). Quando verifica che lui è l'ultimo lettore, esegue la signal su wsem, quindi abilita o un processo scrittore o un prossimo lettore. Il processo scrittore può scrivere solo quando non ci sono lettori, quindi esegue semwait su wsem, scrive ed esegue la signal. Può scrivere il suo comando solo se è l'unico processo che sta accedendo. C'è un

semWait all’interno di una sezione critica, sarebbe buona regola non lasciare dei wait all’interno delle sezioni critiche perché altrimenti se il processo si blocca nessuno può più accedervi, ma qui non fa danni perché la wait blocca il processo quando c’è un processo che sta scrivendo, quindi finché il processo scrive è vero che tutti i lettori non possono accedere alla sezione critica, ma non potrebbero comunque accedere alla lettura.

```
/* program readersandwriters */
int readcount;
semaphore x = 1, wsem = 1;
void reader()                                   void writer()
{                                               {
  while (true) {                                  while (true) {
    semWait (x);                                    semWait (wsem);
    readcount++;                                    WRITEUNIT();
    if (readcount == 1) semWait (wsem);             semSignal (wsem);
    semSignal (x);                                }
    READUNIT();                                 }
    semWait (x);
    readcount--;
    if (readcount == 0) semSignal (wsem);
    semSignal (x);
  }
}

void main() { readcount = 0; parbegin (reader, writer); }
```

Dare precedenza agli scrittori:
ho 3 semafori e non uno solo, semsignal(x) esco dalla mutua esclusione per questo tratto, poi semsignal(rsem), semsignal(z), a questo punto finita la lettura, segnalo semwait(x) per la mutua esclusione, decremento il contatore, e se il contatore è 0 eseguo una signal su wsem, e ciò significa che tutti i lettori hanno finito di leggere. Alla fine semsignal(x) conclude la mutua esclusione. Passando a void writer(), si ha una semWait(y) sul semaforo y per la mutua esclusione, poi incrementiamo la writecount, se writecount == 1 significa che questo è l’unico processo che vuole scrivere, c’è una wait su rsem (semaforo dei lettori), dopodiché si esce dalla sezione critica e si esegue una wait sul semaforo wsem che segnala se si può scrivere o meno. A questo punto si scrive, terminato di scrivere si segnala wsem, cioè un altro processo può accedere. Finiti gli scrittori, per cui questo era l’ultimo scrittore, segnala ai processi lettori di partire. Wsem inizialmente è 1, viene decrementato o quando accede un primo lettore o quando accede uno scrittore, quindi wsem sarà a 0 quando almeno un processo sta leggendo o un processo sta scrivendo, ciò dunque assicura che non vi siano sovrapposizioni tra due processi scrittori, e un processo scrittore e uno lettore. Il semaforo x serve per la mutua esclusione sulla variabile readcount. Semwait e semsignal devono

essere accessibili all'interno della mutua esclusione, cioè non posso mettere semwait(x) e basta o semwait(z) e basta, perché succederebbe che mentre un processo è fermo sulla semsignal, non sarebbe accessibile la lettura che invece deve essere accessibile perché altrimenti il processo in uscita non potrebbero aggiornarla e quindi far ripartire il processo.

```
/* program readersandwriters */
int readcount, writecount;
semaphore x = 1, y = 1, z = 1, wsem = 1, rsem = 1;

void reader()
{
  while (true) {
    semWait (z);
      semWait (rsem);
        semWait (x);
          readcount++;
          if (readcount == 1)
            semWait (wsem);
        semSignal (x);
      semSignal (rsem);
    semSignal (z);
    READUNIT();
    semWait (x);
      readcount--;
      if (readcount == 0)
        semSignal (wsem);
    semSignal (x);
  }
}

void writer ()
{
  while (true) {
    semWait (y);
      writecount++;
      if (writecount == 1)
        semWait (rsem);
    semSignal (y);
    semWait (wsem);
      WRITEUNIT();
    semSignal (wsem);
    semWait (y);
      writecount--;
      if (writecount == 0)
        semSignal (rsem);
    semSignal (y);
  }
}

void main() { readcount = writecount = 0; parbegin (reader, writer); }
```

SCAMBIO DI MESSAGGI

Un server, chiamato anche **controller** gestisce le informazioni che andrebbero condivise. A sx si ha il programma del processo lettore e scrittore, il **processo lettore** ha una variabile rmsg con un messaggio che deve leggere il lettore, invia questo messaggio a una mailbox chiamata **readrequest**, un contenitore che conterrà tutte le richieste da parte dello scrittore, fatto questo si mette in attesa sulla sua porta, mailbox indice i e quando avrà ricevuto il messaggio di risposta dal controllo, legge e poi invia il messaggio lettura terminata in un'altra mailbox denominata finish. Il **processo scrittore** funziona allo stesso modo, con la differenza che invece di readrequest avrà writerequest. Se c'è una richiesta di lettura viene prelevato il messaggio, decrementato il contatore, e in risposta al messaggio ricevuto si invia un messaggio di ricezione. Quando il contatore arriva a 0 si manda allo scrittore il messaggio OK e ci si mette in attesa del messaggio di terminata scrittura, dopodiché il contatore viene portato a 100. Al termine si ha un while(count <0), finché il count è <0 ci si mette in attesa dei messaggi di lavoro terminato e mano mano si incrementa il contatore. Quindi in questo modo il diritto di accesso alla lettura e alla scrittura la dà un processo ben preciso, ovvero il

controllore, e non c'è più la necessità di realizzare la mutua esclusione poiché è il controllore stesso che gestisce il tutto. In questo senso i meccanismi operativi basati su meccanismi client server sono più sicuri, un solo processo che sbaglia le operazioni di mutua esclusione rendono il dato condiviso incoerente.

```
void reader(int i)
{
  message rmsg;
  while (true) {
    rmsg = i;
    send (readrequest, rmsg);
    receive (mbox[i], rmsg);
    READUNIT ();
    rmsg = i;
    send (finished, rmsg);
  }
}

void writer(int j)
{
  message rmsg;
  while(true) {
    rmsg = j;
    send (writerequest, rmsg);
    receive (mbox[j], rmsg);
    WRITEUNIT ();
    rmsg = j;
    send (finished, rmsg);
  }
}
```

```
void controller ()
{
  while (true)
  {
    if (count > 0) {
      if (!empty (finished)) {
        receive (finished, msg);
        count++;
      }
      else if (!empty (writerequest)) {
        receive (writerequest, msg);
        writer_id = msg.id;
        count = count - 100;
      }
      else if (!empty (readrequest)) {
        receive (readrequest, msg);
        count--;
        send (msg.id, "OK");
      }
    }
    if (count == 0) {
      send (writer_id, "OK");
      receive (finished, msg);
      count = 100;
    }
  }
  while (count < 0) {
    receive (finished, msg);
    count++;
  }
}
```

STALLO

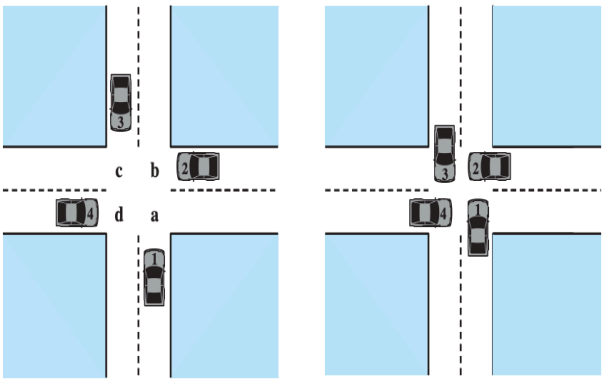
Un insieme di processi è in **stallo (deadlock)** se ogni processo è in attesa di un evento che può essere generato solo da un altro processo dell'insieme, nel caso generale non c'è una soluzione efficiente per la deadlock.

```
Process P

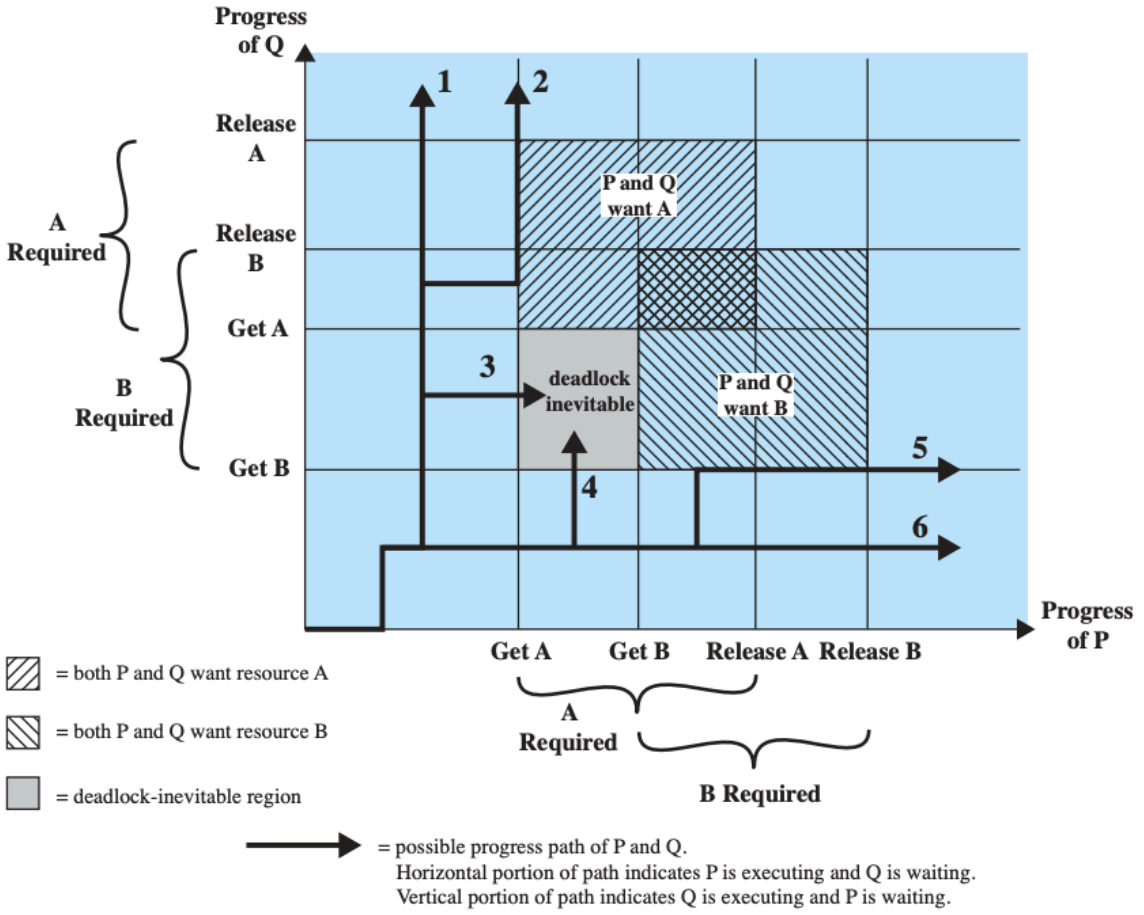
• • •
Get A
• • •
Get B
• • •
Release A
• • •
Release B
• • •
```

```
Process Q

• • •
Get B
• • •
Get A
• • •
Release B
• • •
Release A
• • •
```

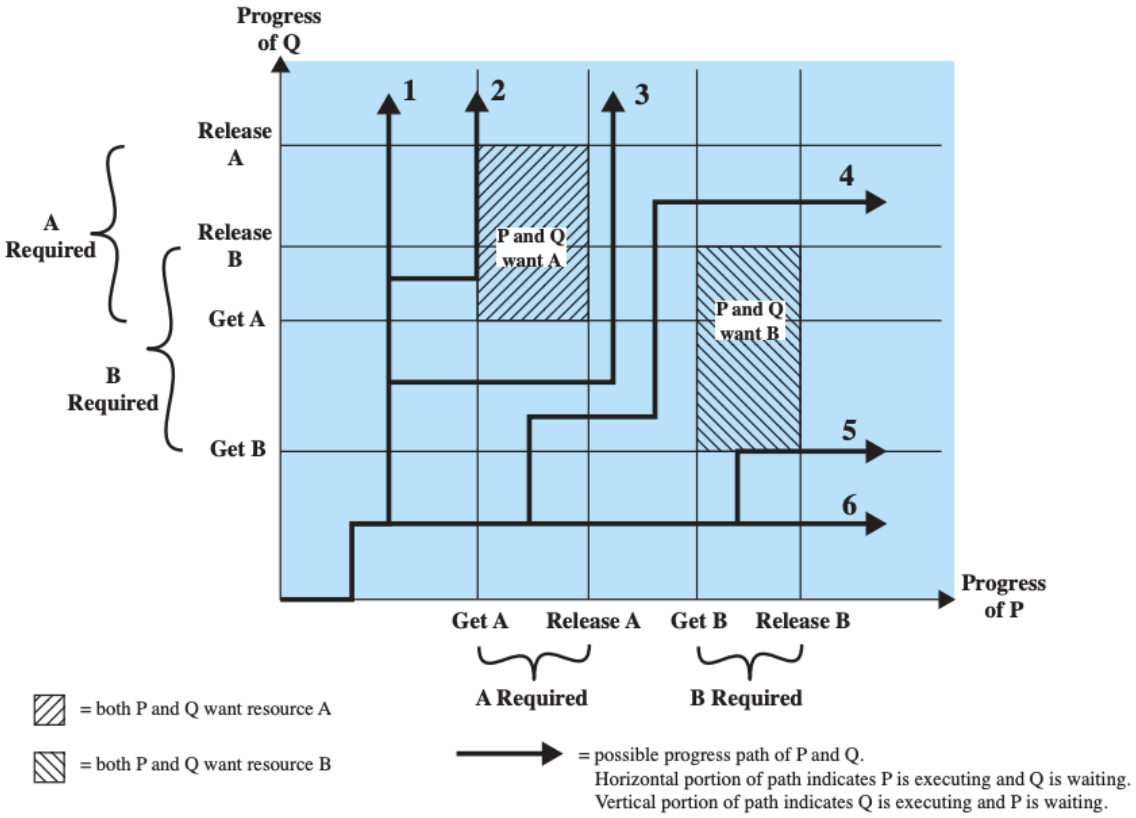


Se siamo sfortunati il processo P preleva A mentre il processo Q preleva B, quando poi i due cercano di prelevare rispettivamente B e A entrambe le risorse sono già occupate, ma per essere rilasciate dai processi si deve passare il punto dove si prendono B e A, perciò oltre ai processi che rimarranno in stallo all'infinito, anche le risorse A e B saranno occupate all'infinito creando un gran problema.



La deadlock avviene se una delle due frecce si interseca con una delle aree tratteggiate, da questa situazione come scritto la deadlock è inevitabile se si

entra nell'area grigia poichè poi le frecce andranno per forza nelle aree tratteggiate.



In questa foto invece è sempre possibile evitare la deadlock.

Per i processi concorrenti non c'è modo di prevedere chi avanzerà per primo e chi per secondo, se la deadlock succede ci si ritrova in una situazione complicata.

CATEGORIE DI RISORSE
Risorse riutilizzabili: Utilizzabili da un solo processo alla volta, non vengono distrutte dal loro utilizzo (come il processore, i canali I/O, la memoria, ecc).
Risorse consumabili: Possono essere create (prodotte) e distrutte (consumate), come le interruzioni, i segnali, i messaggi, i dati in buffer ecc…
Esempio risorse utilizzabili:
Due processi competono per l'uso di un file D e un dispositivo T, p1 richiede prima D e lo blocca, p2 richiede prima T e lo blocca, poi però i processi richiedono subito rispettivamente T e D, che però ora sono bloccati dall'altro processo, perciò ora ci troviamo nella situazione di stallo descritta prima.

P1	P2
Request (D)	Request (T)
Lock (D)	Lock (T)
Request (T)	Request (D)
Lock (T)	Lock (D)
Perform function	Perform function
Unlock (D)	Unlock (T)
Unlock (T)	Unlock (D)

Se però si invertisse l'ordine in cui sono richieste T e D in modo tale che entrambi i processi richiedono le risorse T e D nello stesso ordine, così p1 prende subito sia T che D, fa ciò che deve fare e sblocca le risorse a p2 che aveva trovato T subito occupato ed era rimasto perciò in attesa.

Esempio risorse consumabili:
Due processi si scambiano messaggi:

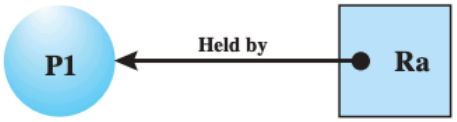
P1	P2
• • •	• • •
Receive(P2)	Receive(P1)
• • •	• • •
Send(P2,M1)	Send(P1,M2)

Receive è una funzione bloccante, p1 attende che p2 mandi un messaggio,ma allo stesso tempo p2 attende che p1 mandi un messaggio, quindi attendono messaggi che non arriveranno mai poichè entrambi in attesa.

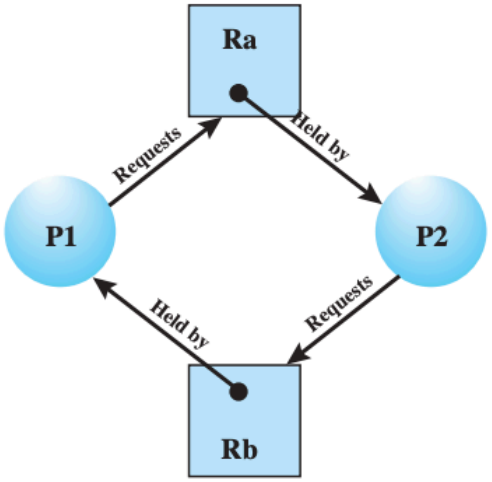
Si può fare un **grafo di allocazione delle risorse**.



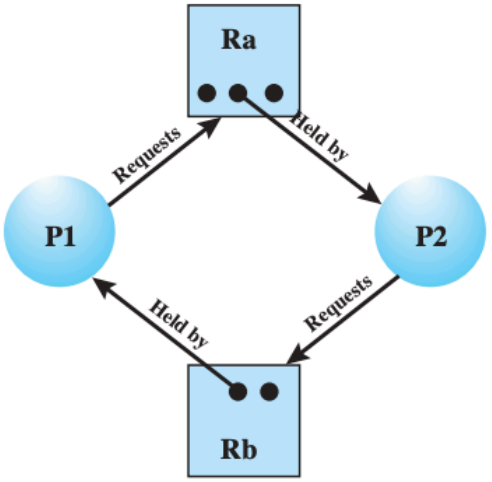
(a) Resouce is requested



(b) Resource is held

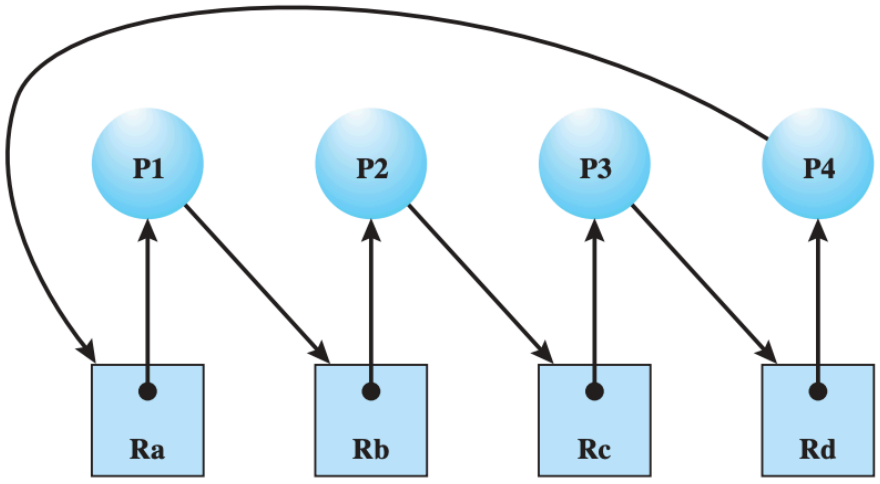


(c) Circular wait



(d) No deadlock

L'esempio dell'incrocio delle auto che non sanno chi debba passare per primo può essere rappresentato così:



CONDIZIONI PER LO STALLO

Le condizioni necessarie sono:

- La mutua esclusione, le risorse possono essere assegnate a un solo processo alla volta;
- Il possesso della risorsa e l'attesa che essa si liberi;
- L'assenza di prerilascio;

La condizione **determinante** per lo stallo è l'**attesa circolare**, cioè che esista una catena chiusa di processi tale che ogni processo possiede una risorsa richiesta dal processo successivo (quindi che si abbia un ciclo nel grafo di Holt, il grafo delle risorse).
Se si hanno solo le condizioni necessarie, lo stallo è **possibile,** se si ha anche la quarta lo stallo è **esistente**.
Per prevenire lo stallo si possono fare diverse azioni:

- **Prevenzione**, far si che i processi si comportino in maniera tale da evitare lo stallo;
- **Esclusione**, meccanismi che permettono al sistema operativo di controllare se assegnare una risorsa a un dato processo può causare lo stallo, nel qual caso questa assegnazione viene rifiutata o posticipata.
- **Rilevamento,** non lo prevediamo, non lo escludiamo, ma quando temiamo che esso si sia verificato cerchiamo di rilevarlo e prendere provvedimenti.

PREVENZIONE

Si progetta il sistema in modo da escludere lo stallo.
Ci sono **metodi indiretti**, che riguardano l'esclusione di una delle tre condizioni necessarie per lo stallo, i **metodi diretti** sono metodi atti a prevenire il presentarsi dell'attesa circolare.
Osserviamo i metodi indiretti e quali delle condizioni necessarie è possibile escludere:

- La **mutua esclusione** è piuttosto impossibile da disabilitare per risorse seriali, è possibili solo in rari casi, come la lettura dei file dove tutti i processi possono accedere contemporaneamente al file;
- Per il **possesso e l'attesa** si può imporre ai processi un'unica richiesta globale di tutte le risorse, sospendendolo finché non le ha ottenute tutte simultaneamente (**assegnazione statica**), funziona,

ma è **inefficiente**, poichè molti processi restano sospesi in attesa delle risorse e possono esserci risorse assegnate e non utilizzate.

- Per l'**assenza di prerilascio**, si impone ai processi in attesa di una risorsa di rilasciare quelle che possiedono, oppure il sistema pre rilascia il processo che usa la risorsa richiesta e gli impone di rilasciare la risorsa utilizzata (**priorità**), anche questo però non è sempre realizzabile, è realizzabile solo se lo stato della risorsa può essere salvato e ripristinato.

L'alternativa sono i metodi diretti, si può evitare l'attesa circolare tramite l'**assegnazione ordinata:**

- Si definisce un ordine totale sulle categorie di risorse;
- Un processo cui siano state già assegnate delle risorse, può richiedere solo risorse di categorie successive a quelle delle risorse assegnate;

Poiché in caso di attesa circolare, un processo ha una risorsa Ri e chiede una risorsa Rj, un altro processo ha la risorsa Rj e chiede Ri, allora uno dei due sta violando il protocollo, il primo se $j < i$, il secondo se $i < j$. Anche il metodo diretto dell'assegnazione ordinata funziona, ma è anch'esso inefficiente, tuttavia se l'ordinamento è coerente e tiene conto dell'ordine in cui le risorse vengono usate (ad esempio scrivere dati su disco, prima ho bisogno della tastiera e poi ho bisogno del disco, raramente viceversa) è molto meno inefficiente dell'assegnazione statica.

ESCLUSIONE

L'esclusione dello stallo può avvenire tramite due approcci:

- il **rifiuto di esecuzione** per i processi le cui richieste potrebbero determinare stallo;
- Il **rifiuto di allocazione** per le richieste di risorse che potrebbero determinare stallo;

In entrambi i casi è necessario conoscere le future richieste dei processi.
C'è bisogno quindi di un algoritmo detto **algoritmo del banchiere** che usa una serie di informazioni:

- Il vettore **risorse**, la quantità totale di ciascuna risorsa nel sistema;
- Il vettore **disponibilità**, indica quante di quelle risorse sono disponibili;
- Una matrice di **richieste**, che per ogni riga (che rappresenta un processo), mi dice quante risorse sta usando quel processo (le colonne sono il tipo di risorsa);
- Una matrice **allocazione**, dove ciascuna riga è sempre un processo che contiene l'allocazione attuale della risorsa della colonna per il processo della riga in cui ci troviamo.

Resource $= \mathbf{R} = (R_1, R_2, \ldots, R_m)$	Total amount of each resource in the system
Available $= \mathbf{V} = (V_1, V_2, \ldots, V_m)$	Total amount of each resource not allocated to any process
Claim $= \mathbf{C} = \begin{pmatrix} C_{11} & C_{12} & \cdots & C_{1m} \\ C_{21} & C_{22} & \cdots & C_{2m} \\ \vdots & \vdots & \vdots & \vdots \\ C_{11} & C_{12} & \cdots & C_{1m} \end{pmatrix}$	$C_{ij} =$ requirement of process i for resource j
Allocation $= \mathbf{A} = \begin{pmatrix} A_{11} & A_{12} & \cdots & A_{1m} \\ A_{21} & A_{22} & \cdots & A_{2m} \\ \vdots & \vdots & \vdots & \vdots \\ A_{11} & A_{12} & \cdots & A_{1m} \end{pmatrix}$	$A_{ij} =$ current allocation to process i of resource j

$$R_j = V_j + \sum_{i=1}^{n} A_{ij}\,, \qquad C_{ij} \leq R_j\,, \qquad A_{ij} \leq C_{ij}\,.$$

Le 3 equazioni/disequazioni mostrano le relazioni fra questi dati definiti sopra, se io sommo le risorse attualmente usate in un processo e ci aggiungo le risorse disponibili rimaste inutilizzate ottengo Rj, cioè la quantità disponibile di quella risorsa, la seconda indica che chiaramente un processo non può richiedere più di quanto c'è al massimo di una risorsa, la terza indica chiaramente che in ogni momento un processo deve avere una quantità di risorse di ogni tipo minore di quella che ha dichiarato necessaria.
Questi quindi sono i dati del problema, allora un possibile approccio è il seguente:
Un nuovo processo Pn+1 inizierà se e soltanto se:

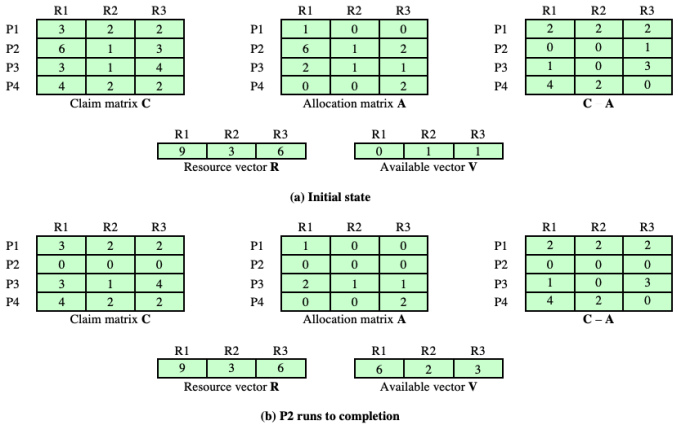
$$\text{per ogni } j, \quad R_j \geq C_{n+1,j} + \sum_{i=1}^{n} C_{ij}$$

, cioè se ci sono risorse sufficienti per soddisfare tutti i processi.
Questo però è inefficiente poichè assume che tutte le richieste di tutti i processi debbano essere soddisfatte simultaneamente.
È molto più efficiente invece l'**algoritmo del banchiere**

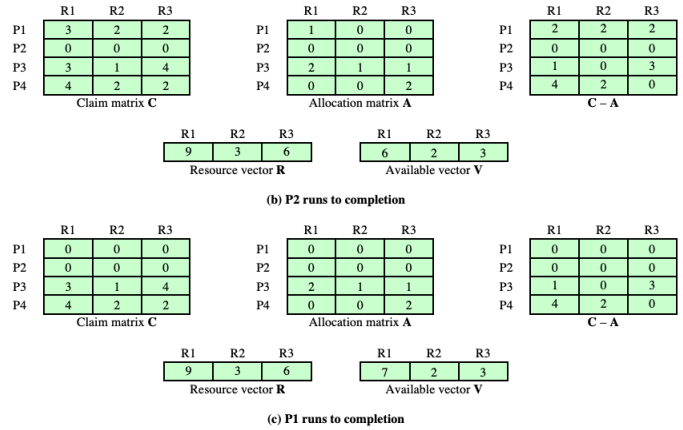
ALGORITMO DEL BANCHIERE

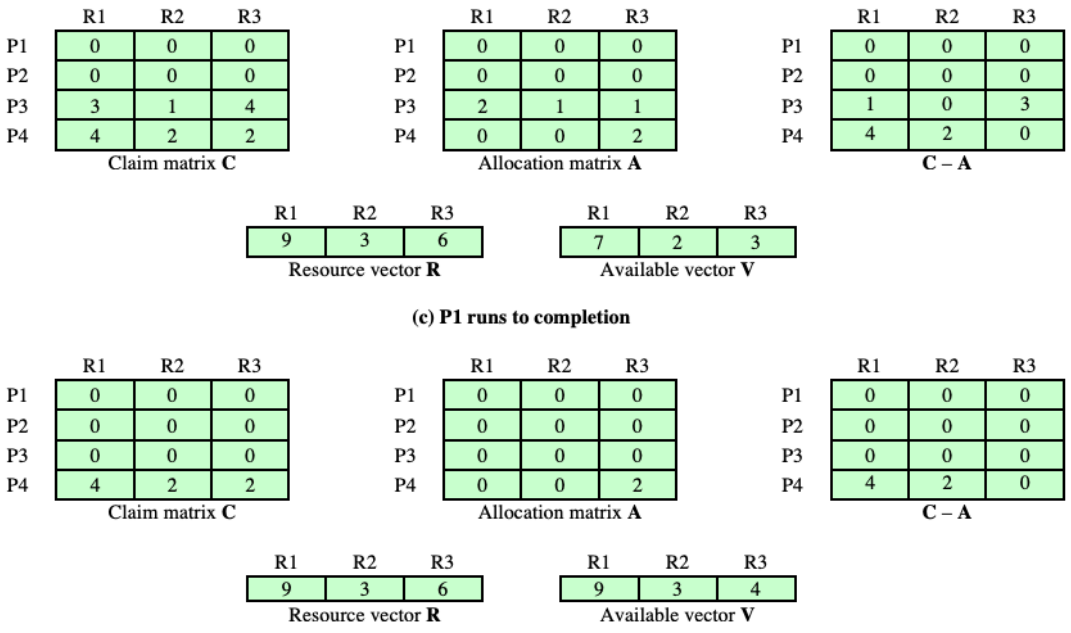
L'algoritmo tiene conto dello stato del sistema, quindi i vettori sopra definiti R e V e le matrici C e A, l'algoritmo controlla se esiste almeno una sequenza di allocazioni che porta a termine tutti i processi e rifiuta una richiesta se porta in uno stato insicuro per evitare lo stallo.
Come si determina se uno stato è sicuro?

Abbiamo 4 processi con le loro specifiche matrici A, C, la matrice C-A sono le risorse aggiuntive di cui avrà bisogno ogni processo per terminare, dopo questo nel vettore delle risorse disponibili mi saranno rimaste 0 1 1, la domanda è con queste risorse rimaste posso terminare un processo già esistente? SI, in questo caso il processo p2 a cui manca solo 1 della risorsa R3 per terminare, perciò l'assegnamo, termina p2 e le risorse allocate a p2 si azzerano e tornano nel vettore V.

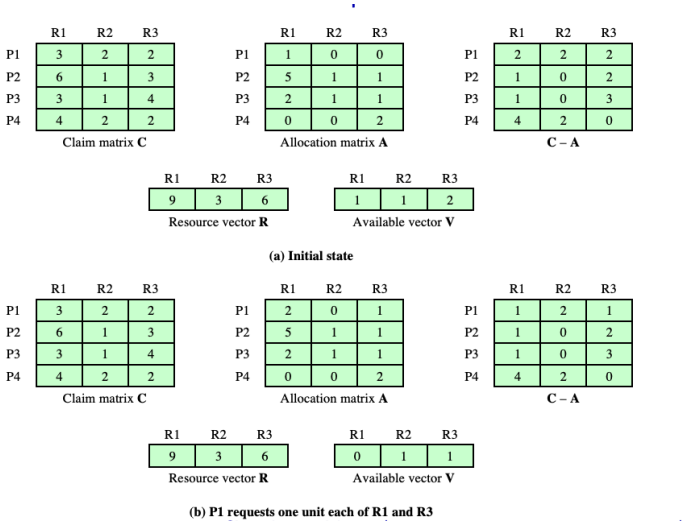


Con le nuove risorse ottenute posso chiudere anche il processo 1 e così via andrò poi a chiudere anche gli altri processi, lo stato è quindi **sicuro.**





Esempio di un'allocazione rifiutata:
In questo caso ad esempio le risorse di V verrebbero allocate, 1 di R1 e di R3 a p1, ma così non termineremmo né il processo p1, né sarebbe poi possibile con le risorse disponibili rimaste terminare gli altri processi, quindi quest'allocazione viene rifiutata.



	R1	R2	R3
P1	3	2	2
P2	6	1	3
P3	3	1	4
P4	4	2	2

Claim matrix **C**

	R1	R2	R3
P1	1	0	0
P2	5	1	1
P3	2	1	1
P4	0	0	2

Allocation matrix **A**

	R1	R2	R3
P1	2	2	2
P2	1	0	2
P3	1	0	3
P4	4	2	0

C – A

R1	R2	R3
9	3	6

Resource vector **R**

R1	R2	R3
1	1	2

Available vector **V**

(a) Initial state

	R1	R2	R3
P1	3	2	2
P2	6	1	3
P3	3	1	4
P4	4	2	2

Claim matrix **C**

	R1	R2	R3
P1	2	0	1
P2	5	1	1
P3	2	1	1
P4	0	0	2

Allocation matrix **A**

	R1	R2	R3
P1	1	2	1
P2	1	0	2
P3	1	0	3
P4	4	2	0

C – A

R1	R2	R3
9	3	6

Resource vector **R**

R1	R2	R3
0	1	1

Available vector **V**

(b) P1 requests one unit each of R1 and R3

```
1  W ← V
2  while c'è un processo P_i non spuntato, tale che C_ij − A_ij ≤ W_j per ogni j
3        spunta P_j
4        for all j
5              W_j ← W_j + A_ij
6  if tutti i processi sono spuntati
7        return SICURO
8  else
9        return INSICURO
```

Algoritmo del banchiere:

```
struct state {
   int resource[m];
   int available[m];
   int claim[n][m];
   int alloc[n][m];
}

if (alloc [i,*] + request [*] > claim [i,*])
   <error>;       /* total request > claim*/
else if (request [*] > available [*])
   <suspend process>;
else {            /* simulate alloc */
   <define newstate by:
    alloc [i,*] = alloc [i,*] + request [*];
    available [*] = available [*] - request [*]>;
}
if (safe (newstate))
   <carry out allocation>;
else {
   <restore original state>;
   <suspend process>;
}
```

Lo stato è la struttura dei dati disponibili sui processi e le risorse allocate, il primo if controlla se c'è una richiesta di risorse maggiore a quelle veramente necessarie dichiarate nel claim.
Il secondo if controlla che la richiesta non sia maggiore della quantità disponibile di quella risorsa.
Passati questi due controlli definisco un nuovo stato cercando di allocare ad un processo le risorse necessarie e se il nuovo stato è safe mando a buon fine l'allocazione, altrimenti torno allo stato originale e sospendo il processo.

Come controllo se uno stato è safe?

```
boolean safe (state S) {
   int currentavail[m];
   process rest[<number of processes>];
   currentavail = available;
   rest = {all processes};
   possible = true;
   while (possible) {
      <find a process Pk in rest such that
         claim [k,*] - alloc [k,*]<= currentavail;
      if (found) {     /* simulate execution of Pk */
         currentavail = currentavail + alloc [k,*];
         rest = rest - {Pk};
      }
      else possible = false;
   }
   return (rest == null);
}
```

La procedura safe cerca nello stato che gli viene fornito se c'è un processo Pk che può essere completato tramite le risorse disponibili attualmente, se lo trova simula l'esecuzione di Pk e lo toglie dall'insieme dei processi.

Quali sono i vantaggi e i limiti dell'algoritmo del banchiere per evitare lo stallo?

Vantaggi:

- Non richiede prerilascio e ripristino del processo;
- È meno restrittivo della prevenzione;

Limitazioni:

- I processi devono stabilire in anticipo le risorse necessarie;
- I processi devono essere indipendenti;
- Risorse allocabili in un numero fisso;
- Nessun processo può terminare prima di aver rilasciato le risorse.

RILEVARE LO STALLO

Nei casi in cui si previene o si evita del tutto lo stallo l'accesso alle risorse è limitato a causa delle restrizioni imposte a processi e assegnazioni per evitare lo stallo, quando invece lo stallo si **rileva** solo quando si verifica (**deadlock detection**) le risorse sono assegnate

senza restrizioni, sono periodiche le verifiche dell'assenza di attesa circolare.
I vantaggi della rilevazione sono che la rilevazione dell'attesa circolare permette un ripristino precoce e l'algoritmo è molto semplice, tuttavia le verifiche vanno fatte frequentemente e consumano molto tempo di elaborazione.

Algoritmo di rilevazione dello stallo

I dati necessari all'algoritmo sono sostanzialmente quelli necessari all'algoritmo del banchiere:

- La matrice di allocazione A = A_{ij};
- Il vettore delle risorse disponibili V = V_j;
- Una matrice delle **richieste** Q = Q_{ij};

```
1  W ← V
2  spunta tutti i processi P_i tali che A_ij = 0 per ogni j
3  while c'è un processo P_i non spuntato, tale che Q_ij ≤ W_j per ogni j
4        spunta P_i
5        for all j
6              W_j ← W_j + A_ij
```

Al termine dell'algoritmo gli eventuali processi non spuntati sono in stallo.
Nell'esempio prima controlliamo se ci sono alcuni processi Pi tali che Aij = 0 in tutte le colonne, in questo caso P4 ha 0 risorse allocate e quindi viene spuntato, poi vediamo che sulla riga di P3, per ogni j gli elementi Qij sono <= Wj, quindi possiamo spuntare anche P3, le cui risorse allocate vengono liberate e assegnate a W che diventa (0,0,0,1,1), poi però guardando P2 e P1 entrambi non possono essere spuntati, dunque alla fine rimangono P1 e P2 non spuntati e quindi c'è lo stallo.

	R1	R2	R3	R4	R5
P1	0	1	0	0	1
P2	0	0	1	0	1
P3	0	0	0	0	1
P4	1	0	1	0	1

Request matrix Q

	R1	R2	R3	R4	R5
P1	1	0	1	1	0
P2	1	1	0	0	0
P3	0	0	0	1	0
P4	0	0	0	0	0

Allocation matrix A

R1	R2	R3	R4	R5
2	1	1	2	1

Resource vector

R1	R2	R3	R4	R5
0	0	0	0	1

Available vector

- **W** = (0 0 0 0 1)
- spunta P4
- **W** = (0 0 0 0 1)
- spunta P3
- **W** = (0 0 0 1 1)
- nessun altro processo da spuntare: P1 e P2 in stallo

Approach	Resource Allocation Policy	Different Schemes	Major Advantages	Major Disadvantages
Prevention	Conservative; undercom-mits resources	Requesting all resources at once	• Works well for processes that perform a single burst of activity • No preemption necessary	• Inefficient • Delays process initiation • Future resource requirements must be known by processes
		Preemption	• Convenient when applied to resources whose state can be saved and restored easily	• Preempts more often than necessary
		Resource ordering	• Feasible to enforce via compile-time checks • Needs no run-time computation since problem is solved in system design	• Disallows incremental resource requests
Avoidance	Midway between that of detection and prevention	Manipulate to find at least one safe path	• No preemption necessary	• Future resource requirements must be known by OS • Processes can be blocked for long periods
Detection	Very liberal; requested resources are granted where possible	Invoke periodically to test for deadlock	• Never delays process initiation • Facilitates online handling	• Inherent preemption losses

Preempts = interruzioni forzate dei processi.

STRATEGIE DI RIPRISTINO

- Terminare tutti i processi in stallo;
- Ripristinare tutti i processi in stallo a uno stato precedente (**checkpoint**), questo però necessita l'uso di meccanismi di salvataggio e riattivazione che rallentano l'esecuzione e che inoltre rischiano di riproporre lo stallo;
- Terminare un processo alla volta finchè lo stallo non scompare (applicando una politica di costo minimo elencate sotto);
- Revocare una risorsa alla volta finchè lo stallo non scompare (sempre applicando una politica di costo minimo).

Alcuni tipi di **politiche** seguono:

- Minimo tempo di elaborazione consumato (che usa meno CPU);
- Minimo output prodotto;
- Massimo tempo di elaborazione rimanente (stimato);
- Minima quantità di risorse allocate;
- Priorità.

STRATEGIA INTEGRATA

Una strategia non deve per forza prendere uno dei metodi e usarlo per tutto, l'idea è di raggruppare le risorse in classi, usare l'assegnazione ordinata per prevenire l'attesa circolare fra classi di risorse e in ogni classe utilizzare il metodo più appropriato.

Esempio

Classi di risorse:

- Spazio di swap
- Risorse dei processi (dispositivi)
- Memoria principale
- Risorse interne (canali I/O)

assegnate nell'ordine indicato.

Spazio di swap assegnazione statica (o rifiuto di esecuzione)
Risorse dei processi algoritmo del banchiere (o assegnazione ordinata)
Memoria principale prevenzione tramite prerilascio
Risorse interne assegnazione ordinata

GESTIONE DELLA MEMORIA

Un programma per essere eseguito va caricato in memoria, quali sono i principali compiti del gestore della memoria?

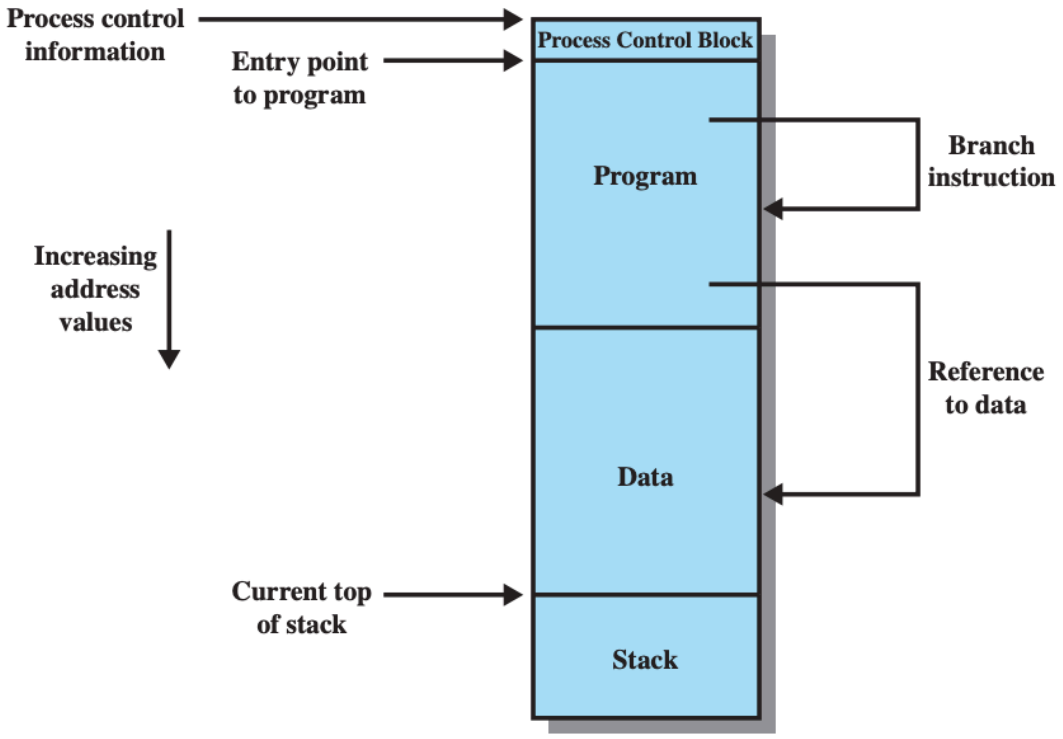
- La **rilocazione**, il nostro programma viene copiato nella memoria centrale, ma a priori non sappiamo in quale parte venga caricato, quindi nel nostro eseguibile gli indirizzi in cui andare a prelevare le istruzioni e i dati non ci possono stare, saranno indirizzi simbolici (**logici**), una volta caricati, questi indirizzi simbolici devono essere convertiti in indirizzi fisici;
- La **protezione**, il processo preleva l'istruzione, la esegue, scrive i risultati in memoria e passa all'istruzione successiva, quando fa queste operazioni è fondamentale che non esca dalla memoria assegnata al processo.
- La **condivisione**.
- L'**organizzazione logica**.
- L'**organizzazione fisica**.

RILOCAZIONE

Il programmatore non può sapere nel momento in cui esegue il programma cosa ci sarà nella memoria principale del calcolatore.
Deve essere possibile trasferire i programmi dalla memoria principale alla memoria secondaria e viceversa (**swapping**).
Pertanto è impossibile:

- Specificare in che parte della memoria sarà caricato il processo;
- Richiedere che a ogni swap sia caricato nella stessa posizione;

Quindi il processo può essere **rilocato** in un'area di memoria differente.
L'**entry point** è la prima istruzione del programma da cui parte, poi c'è il programma, poi c'è una parte riservata ai dati dove si salta quando nel programma ci si riferisce a quei dati, l'istruzione che fa il riferimento conterrà un indirizzo simbolico al dato richiesto.
Questo è il modello più semplice di programma, dopodichè questo oggetto sarà trasferito in memoria principale, dove l'istruzione 0 corrisponderà a un certo indirizzo fisico della ram, l'indirizzo 1 a quella successiva e così via.



PROTEZIONE

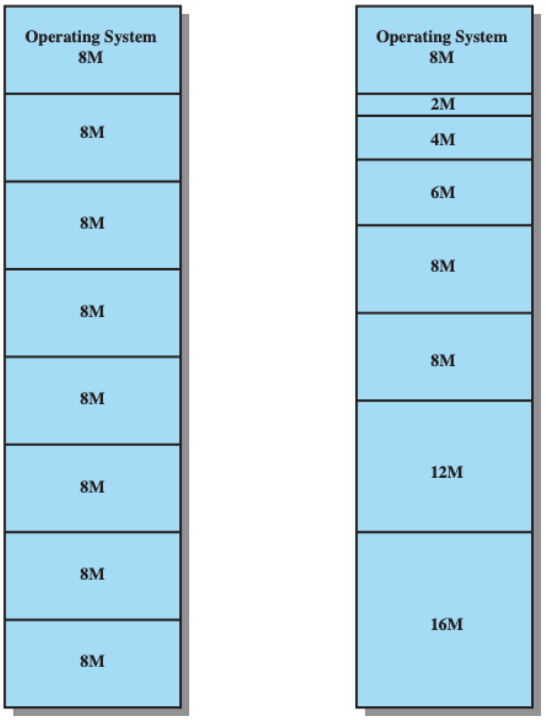
I riferimenti di cui parlavamo prima alla memoria servono per leggere e scrivere i dati, come detto non sappiamo l'effettiva locazione del programma in memoria, quindi solo quando tentiamo di accedere un indirizzo possiamo realmente vedere se quell'indirizzo fa parte del processo, quindi i meccanismi di rilocazione devono anche contenere un supporto di sicurezza.

CONDIVISIONE
In alcuni sistemi è fortemente limitata, in altri è supportata.
Ci sono programmi che vengono eseguiti da più processi, quindi in questo caso è utile permettere a questi processi di accedere entrambi alla stessa copia del codice.
Il gestore della memoria quindi deve permettere un accesso controllato delle aree di memoria condivisa, senza compromettere la protezione. I meccanismi che supportano la rilocazione supportano anche la condivisione.

ORGANIZZAZIONE LOGICA
La memoria principale è organizzata linearmente, i programmi sono scritti in moduli.
L'organizzazione logica modulare permette che i moduli vengano scritti e compilati indipendentemente e che possano avere differenti gradi di protezione (alcuni solo per la lettura, altri solo per la scrittura). Dal punto di vista dell'utilizzatore quindi la condivisione a livello di modulo è più naturale, ed è anche la forma più elementare di organizzazione.
Lo strumento che meglio soddisfa questi requisiti è la **segmentazione**.
ORGANIZZAZIONE FISICA
Un punto fondamentale è la differenza sostanziale tra la memoria principale (RAM) del sistema e la memoria secondaria (Il disco rigido nel PC), la differenza sostanziale è che la memoria principale è volatile, veloce e costosa, la memoria secondaria invece è permanente, lenta, ma più economica quindi più accessibile a tutti.
Il programma da eseguire deve stare almeno parzialmente nella RAM, perchè altrimenti sarebbe troppo lento accedervi istruzione per istruzione.
Il problema sta nel fatto che potrebbe non esserci RAM sufficiente nel programma, il primo motivo è la multiprogrammazione, potremmo avere tanti processi in avanzamento, il secondo è il fatto che alcuni programmi potrebbero richiedere più spazio di quello che abbiamo disponibile.
Allora se il nostro programma è più grande della RAM che abbiamo a disposizione? Una possibilità è quella di realizzare **l'overlaying:** la stessa regione è assegnata a vari moduli sotto la responsabilità del programma, ad esempio un programma che produce 10 giga di output, potremmo pensare di mettere tutto in un buffer e trasferirlo alla fine nel disco, però 10 giga di ram sono tanti, magari non li abbiamo, allora alcune tecniche lasciate fare al sistema aggiungono delle istruzioni che permettono di spostare dati e a volte anche istruzioni del programma stesso da RAM a disco fisico e viceversa.

Il gestore della memoria quindi carica i processi nella memoria principale per essere eseguiti, i metodi principali per fare ciò sono 3, **partizionamento, segmentazione e paginazione**.

PARTIZIONAMENTO

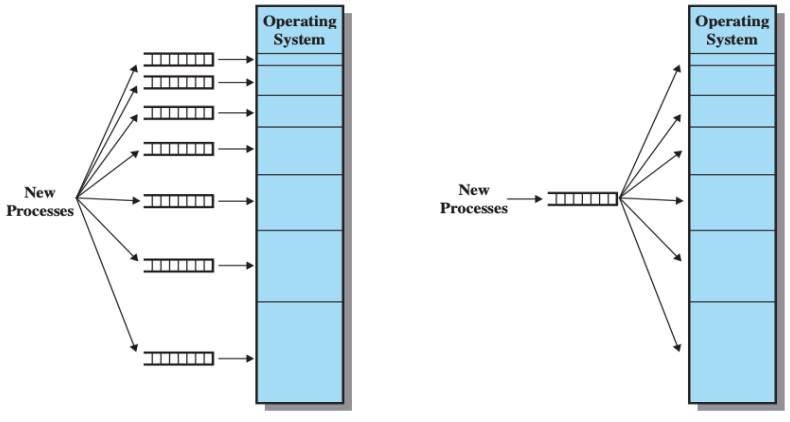


È un metodo perlopiù obsoleto e che non prevede memoria virtuali, usa le **partizioni fisse** e le **partizioni variabili.**

Si carica il sistema operativo nella RAM e ciò che resta di RAM viene diviso in partizioni uguali, poi carico il programma che occupa 4 megabyte e si cercherà una delle partizioni con grandezza minore o uguale a quella richiesta dal programma. Nell'esempio sulla prima colonna abbiamo tutte partizioni da 8 mega, quindi possiamo far partire solo programmi da 8 mega, nella seconda invece abbiamo quantità differenti.

I problemi di questo metodo stanno proprio nel fatto che un programma potrebbe essere troppo lungo per entrare in una partizione e quindi si userebbe fer forza l'overlay, inoltre comporta un utilizzo inefficiente della memoria principale poichè ogni programma occupa un'intera partizione e quindi si limita anche il numero di processi attivi al numero di partizioni che abbiamo, si crea anche una **frammentazione interna**, cioè uno spreco di spazio poichè il programma può richiedere meno memoria di quella della partizione.

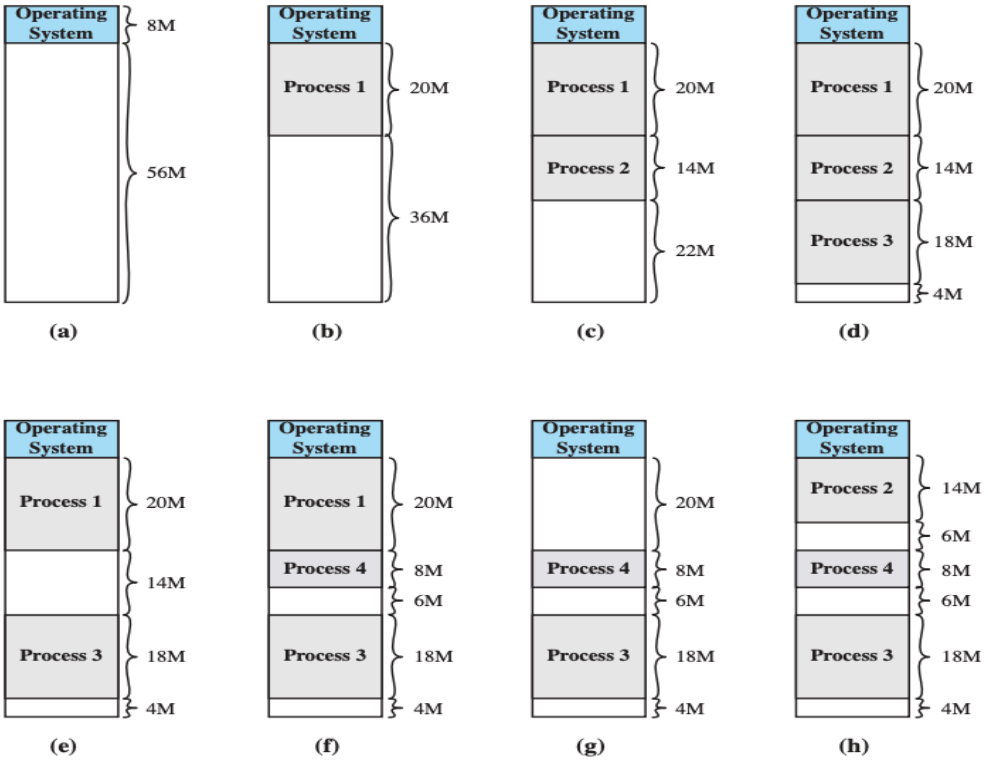
Per allocare un processo ad una partizione o si crea una coda per ogni processo, oppure metto il processo in una singola coda che guarda tutte le partizioni.



(a) One process queue per partition (b) Single queue

SEGMENTAZIONE

Un'alternativa sono le **partizioni dinamiche,** cioè le partizioni vengono create dinamicamente quando un processo è caricato in memoria, la ram che non è occupata dal sistema operativo dunque è un blocco unico che viene allocato man mano.



Questo però come si vede nella foto provoca una **frammentazione esterna** che può essere risolta però con una compattazione della ram che viene liberata man mano che i processi finiscono.

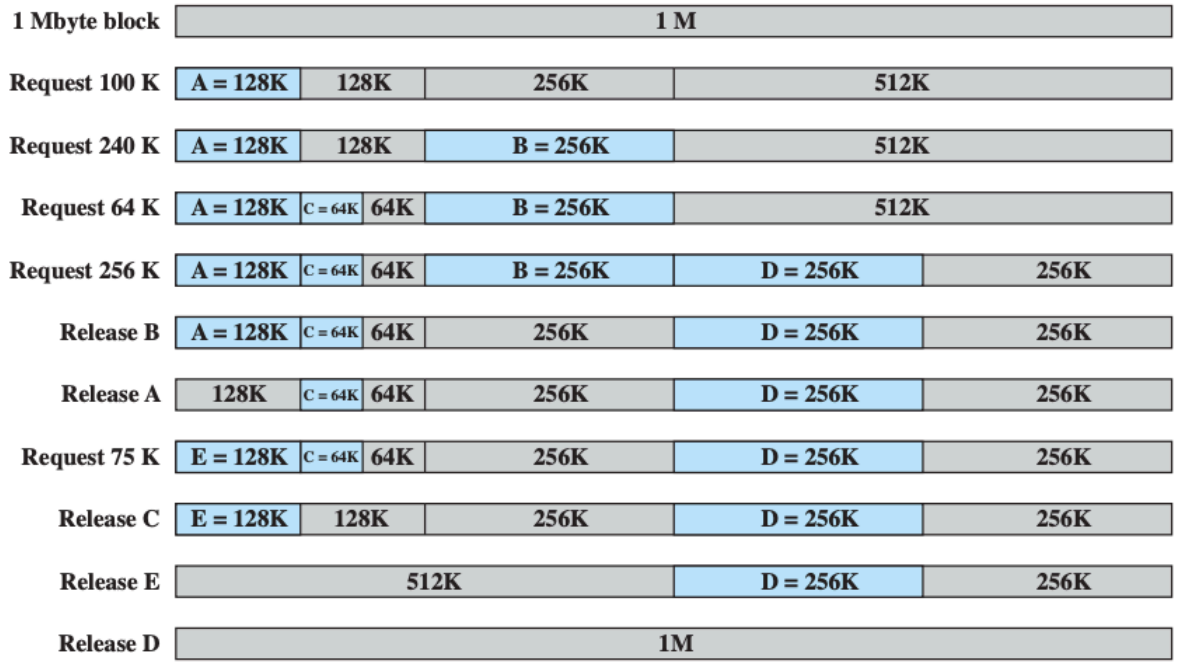
Per il metodo di allocazione si possono avere 3 politiche di scelta del blocco:

- **Best-fit**, sceglie il blocco disponibile che ha la dimensione più simile a quella richiesta;
- **First-fit**, sceglie il primo blocco di dimensione sufficiente, a partire dall'inizio della memoria;
- **Next-fit**, sceglie il primo blocco sufficiente a partire dall'ultima partizione creata.

Il best-fit può sembrare il metodo migliore però con esso si creano un sacco di partizioni molto piccole, la first-fit invece tutto sommato funziona meglio, però con questa si cerca di usare sempre la prima parte della memoria, si potrebbe creare quindi una frammentazione concentrata sulla prima parte della memoria.

BUDDY SYSTEM

C'è però un meccanismo, il **buddy system**, un compromesso tra partizioni fisse e dinamiche che è più facile da gestire.
Si hanno blocchi di dimensione 2^K, L <= K <= U, dove 2^U è lo spazio allocabile, 2^L è la dimensione dei blocchi allocabili.
Un programma di dimensione I con 2^{K-1} < I <= 2^K viene allocato in un blocco libero di dimensione 2^K, se non ce ne sono, un blocco di dimensione 2^{K+1} è spezzato in due **buddies** di dimensione 2^K.
Non appena due buddies sono contemporaneamente liberi sono riuniti in un unico blocco di dimensione doppia.

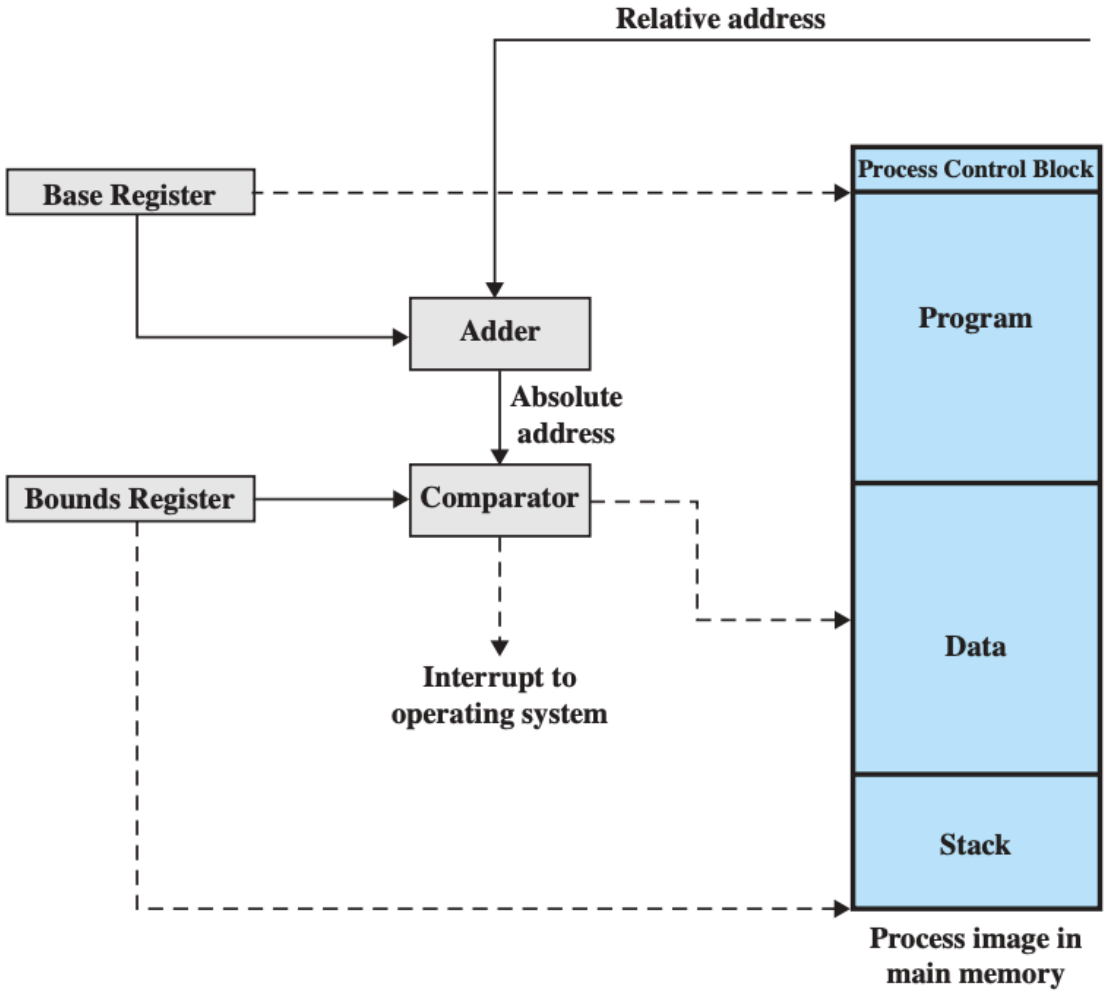


Gli indirizzi all'interno del nostro programma possono essere **logici**, ossia riferimenti a una locazione di memoria indipendente dall'assegnazione della memoria principale al processo, quindi il vero indirizzo del dato non è conosciuto al programmatore, l'indirizzo **fisico** invece è l'effettiva posizione del dato nella memoria principale, l'indirizzo **relativo** è l'indirizzo logico espresso come posizione relativa a un altro indirizzo noto.
La **rilocazione** può essere **statica** se a un processo sarà sempre assegnata la stessa partizione, in questo caso si può usare un caricatore (**loader)** rilocante. Al caricamento tutti i riferimenti a indirizzi relativi sono sostituiti da indirizzi assoluti, determinati dall'indirizzo base del processo caricato.

La rilocazione può essere invece **dinamica** se il processo può occupare differenti partizioni nel corso della sua vita (ad esempio partizioni variabili e caricamento dinamico), si ha chiaramente quando avviene lo swapping tra memoria principale e secondaria:

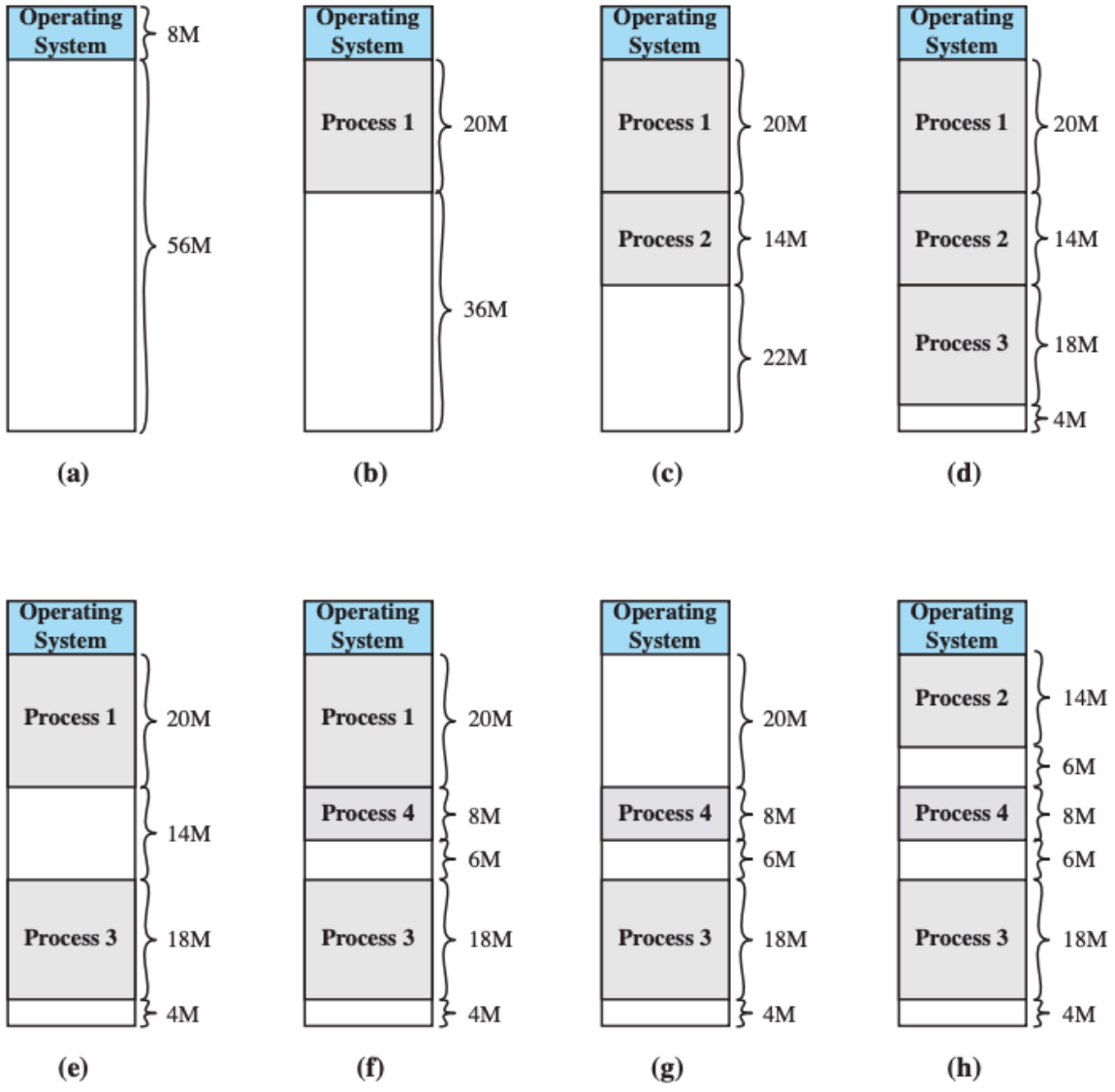
- Quando l'immagine del processo è creata, viene caricata in una partizione;
- Quando è trasferita in memoria secondaria e poi ricaricata in memoria, occuperà un'altra partizione;
- In caso di compattamento della memoria, la partizione sarà ancora modificata.
- Quindi le locazioni di memoria riferite dal programma variano ogni volta che il processo subisce uno swap.

È necessaria la rilocazione dinamica con supporto hardware.

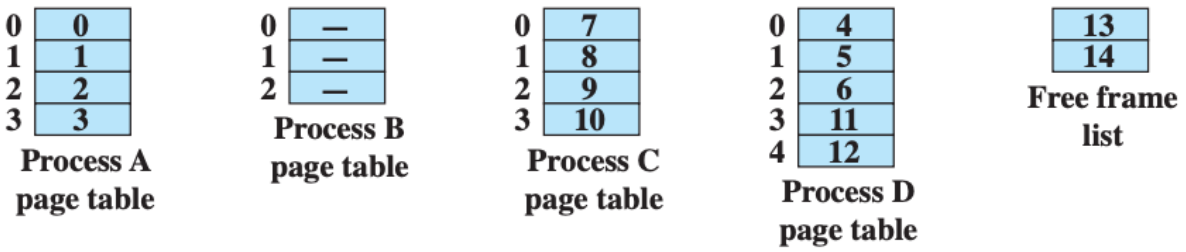


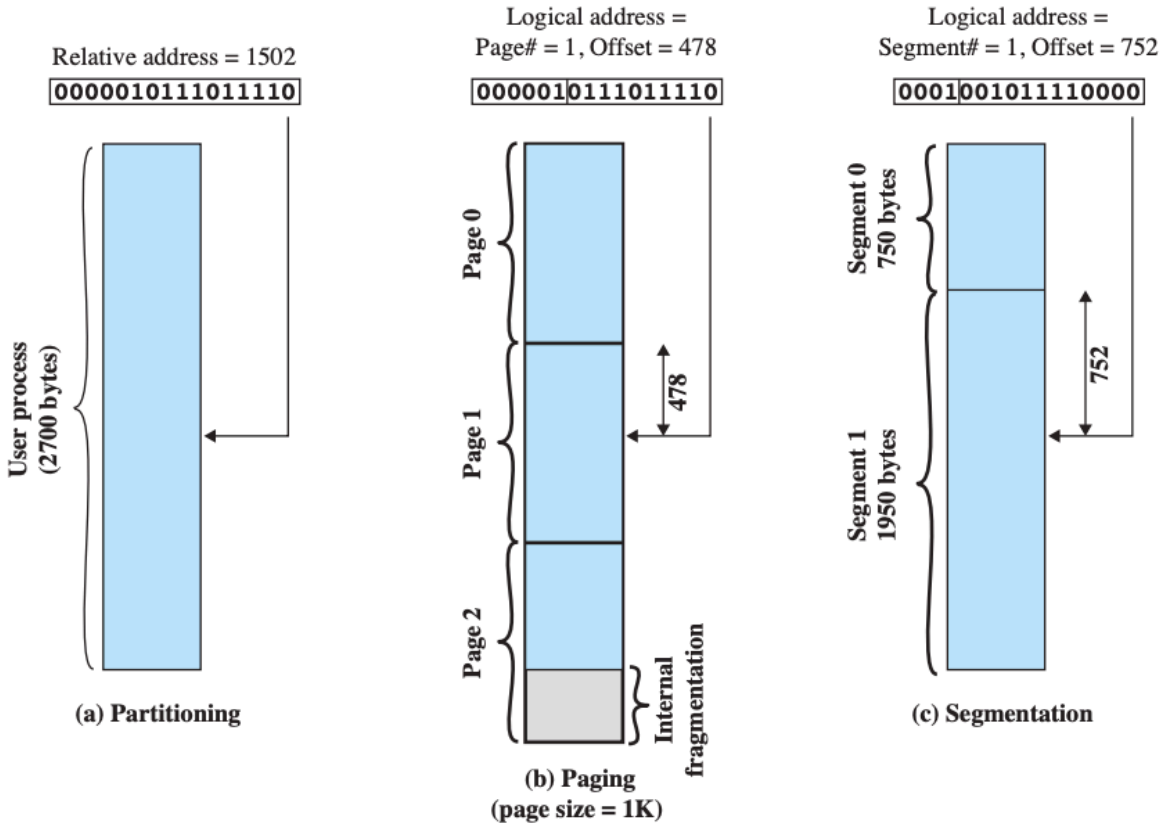
PAGINAZIONE

Nella paginazione la memoria principale è partizionata in blocchi di dimensione fissa detti **frame,** i programmi sono divisi in blocchi della stessa dimensione detti **pagine**, queste pagine sono poi caricate in frame grandi quanto la pagina.



Per ogni processo viene creata una tabella che contiene la posizione del frame per ogni pagina del processo, il processore accede alla tabella delle pagine del processo corrente per produrre l'indirizzo fisico.





Le 3 pagine sono lunghe ciascuna 1024 byte, l'ultima pagina ha uno spazio "sprecato" dovuto al fatto che il programma occupa 2700 byte, quindi all'ultima pagina se ne allocano 652 e il resto non servirà. L'indirizzo relativo 1502 rappresenta la casella indicata dalla freccia, l'indirizzo logico corrispondente è che, la casella appartiene alla pagina di numero 1, con offset = 478, ossia salterà i primi 1024 byte che appartengono alla prima pagina (la pagina 0) e partirà dal 1024 esimo byte applicandogli l'offset di 478 che ci permette di arrivare all'indirizzo 1502, {segmentazione la spiega la prox volta}.

SEGMENTAZIONE

Il caricamento dei programmi segmentati avviene tramite segmentazione dinamica. Però a ogni processo servono tante partizioni, una per ogni segmento. Quindi a ogni processo assegneremo tanti partizioni quanti sono i segmenti possibili. Cosa si è guadagnato cosi facendo? I segmenti sono più piccoli del programma intero, quindi tutti i fenomeni problematici che si avevano quando si aveva bisogno di una partizione molto grande, non vi è più, poiché i segmenti sono piccoli (ce ne sono quindi tanti). Il programma risulta strutturato e vi sono una serie di vantaggi. Il problema scaturisce dal fatto che perché tutto funzioni i

segmenti non possono essere troppo grandi. La dimensione dei segmenti dipende dal programmatore (non succede dunque che a frame più piccoli o più grandi deve variare la grandezza del programma). Quindi gli indirizzi sono fatti da coppie: indirizzo di segmento, offset. Traduzione degli indirizzi: come si fa? Vi è un indirizzo logico che è fatto da n bit, che è l'indirizzo di segmento, e un'altra parte che è l'offset. Nella figura (c) vi sono 4 bit per l'indice di segmento e 12 per l'offset. Dopodiché nel nostro sistema si ha una tabella dei segmenti (non più delle pagine) per cui ogni processo ha una tabella dei segmenti in cui vi è l'indirizzo fisico in cui comincia il segmento che mi interessa. A questo punto l'offset va sommato (addizione vera) all'indirizzo base, e trovo l'indirizzo al quale accedere. Posso accedere? Non ancora perché vi potrebbe essere un'eccezione di indirizzo. Si applica dunque la protezione di memoria, ovvero confronto il risultato della mia operazione con l'indirizzo limite, se è più grande il sistema operativo segnala un'eccezione. Nella paginazione non c'è questo problema, poiché la dimensione è fissa, il massimo valore che può assumere è la dimensione del frame.
Sommo l'offset con l'indirizzo base, il risultato è l'indirizzo fisico. L'accesso alla tabella dei segmenti non è frequente perché tipicamente un programma rimane nel segmento per tempi molto lunghi. L'aggiornamento di questi registri (base e limite) avviene di fatto una volta ogni tanto, non a ogni istruzione. Il discorso è diverso nel caso della paginazione, lì la divisione è puramente casuale.

MEMORIA VIRTUALE

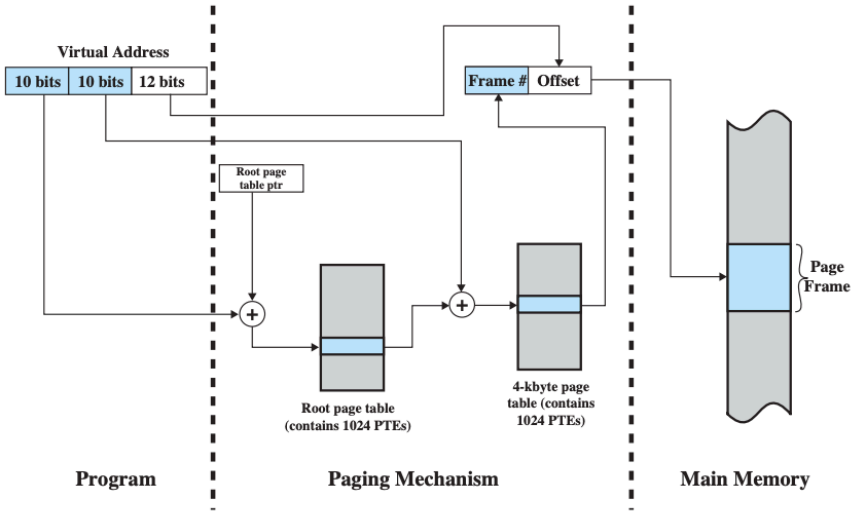
Il programma per essere eseguito deve stare in memoria, che però è limitata quindi ad esempio se nella memoria possono starvi 10 programmi vi saranno attivi solo 10 processi. I sistemi moderni possono però fare di meglio tramite il modello della memoria virtuale. Innanzitutto la rilocazione, ovvero la traduzione degli indirizzi, normalmente avviene a tempo di esecuzione (run-time). Inoltre il processo non ha necessariamente un modello continuo ma può essere paginato oppure segmentato, quindi in realtà il programma è diviso in pezzi. Il processo può avanzare anche solo se in memoria vi sono i pezzi che mi servono (cioè i processi) e non il programma intero. Dunque il punto chiave è che non è necessario che l'intero programma sia in memoria per poter

eseguire il processo. Quindi di volta in volta verrà caricata in memoria solo la parte di processo che ci serve in tale momento.
Il principio di località: per tempi d'esecuzione relativamente brevi sono istruzioni e dati necessari sono tutti concentrati in una piccola parte di programma, cioè osservando il programma per un piccolo periodo, ci si accorge che le istruzioni e i dati sono concentrati in una piccola parte. Effettivamente per periodi relativamente lunghi (si parla di frazioni di secondo) il processo utilizza in realtà aree di programma molto piccole. L'immagine del processo, cioè il programma con i dati, viene copiata nel disco rigido, dopodiché, pensando alla paginazione, il processo carica alcune pagine in memoria, quando viene richiesta una pagina che non vi è il sistema operativo sospende il processo e il processore è assegnato a un altro processo. Poi la pagina (o il segmento) contenente lìindirizzo logico richiesto viene caricata in memoria. Il trasferimento da disco a memoria principale avviene senza spessa di CPU. Al termine del caricamento avviene un'altra sospensione dal controllore, ovvero dalla DMA. In seguito all'interruzione il sistema operativo riattiva il processo, per cui il processore riprende da dove aveva sospeso.
Vantaggi derivano dal fatto che si incrementa la multiprogrammazione, cioè posso avere tanti processi perché ognuno di questi ha bisogno di un piccolo spazio di memoria principale, non deve quindi essere caricato interamente ma si necessita solo di una piccola porzione. Ciò garantisce l'efficientamento della gestione del processore, perché aumentano i processi pronti.
Non è più necessario l'overlay, si può avere in linea di principio un programma che richiede uno spazio molto maggiore di quello disponibile in sistema perché tanto non deve essere caricato interamente in memoria, ma solo le parti necessarie di volta in volta.
Vi sono due modelli di memoria: paginazione e segmentazione. Possibilità di caricamento semplice, ovvero segmentazione semplice e possibilità con memoria virtuale. Nella paginazione la memoria viene divisa in frames, nella segmentazione la memoria non è partizionata inizialmente, quindi avviene un partizionamento dinamico al caricamento. Il programma è tagliato in pagine nel caso di paginazione (tutte della stessa dimensione) e ciò è svolto dal sistema operativo, nel caso della segmentazione il programma è diviso in segmenti scelti dal programmatore. Spreco di memoria: con la segmentazione c'è

frammentazione interna, l'ultima pagina è minore. Con la paginazione c'è frammentazione esterna, certe pagine sono troppo piccole per caricarvi processi. La frammentazione interna rispetto al caricamento in partizioni fisse prevede partizioni fisse, al massimo si possono sprecare un frame -1byte nella nostra memoria, mentre con la segmentazione avendo segmenti che sono più piccoli di un programma con modello continuo, frammenti piccoli si riescono a utilizzare. Per la paginazione vi è la tabella delle pagine in cui l'informazione principale è il frame occupato da ogni pagina, mentre nella segmentazione si ha una tabella dei segmenti per ciascun processo che mostra l'indirizzo di caricamento e la lunghezza di ciascun segmento. L'indirizzo fisico nel caso della paginazione si ottiene sostituendo il numero di pagine con l'indice di frame, nel caso della segmentazione bisogna calcolare l'indirizzo fisico tramite il numero di segmento e l'offset.

Principio di località: i riferimenti a dati e codice di un processo si concentrano in zone ben precise del programma, quindi si concentrano in poche pagine e in pochi segmenti. Il principio di località assicura che di solito il trashing non avviene, ovvero lo stato in cui il sistema spende la maggior parte del suo tempo nel caricamento e scaricamento dalla memoria, anziché nell'esecuzione dei processi. Quando la memoria è piena scarico ciò che non mi servirà nel futuro, dunque è possibile fare ipotesi intelligenti su quali parti saranno necessarie in futuro.
Supporto HW e SW per la memoria virtuale: l'hw deve supportare la paginazione e la segmentazione, il sistema operativo inoltre deve includere il software per gestire il movimento delle pagine o dei segmenti fra memoria secondaria e memoria principale. In particolare l'hw deve disporre di meccanismi che permettano di accedere correttamente alla lettura della tabella delle pagine eccetera.
Nella tabella delle pagine, in paginazione dinamica, vi è un bit di presenze, per cui vero se la pagina è caricata in memoria e falso se non lo è, per cui il processore guarda per prima cosa quel primo bit. Poi c'è un bit modificato (dirty bit) che serve a indicare se tale pagina, nel momento in cui è stata realizzata è stata modificata.

L’indirizzo virtuale è quello che sta nel program counter. C’è un registro che contiene il puntatore alla tabella delle pagine in memoria principale, a questo puntatore si somma l’indice di pagina e si ottiene il puntatore al descrittore di pagina, cioè la posizione (l’indirizzo) del descrittore di pagina del record che mi interessa. A questo punto si recupera l’indice di frame e si va a sostituire all’indice di pagina, per cui otterrò un nuovo indirizzo, questa volta fisico, i bit più significativi sono gli indici di frame. Ci sarà un addizionatore dedicato che ci permetterà di trovare l’indirizzo del record che ci interessa e a questo punto si può accedere alla memoria di tale record. Ci manca la verifica che il bit di presenza del caricamento sia true o false.
Quando le pagine sono troppe può succedere che questa operazione diventi troppo grande per stare qui dentro, per cui si utilizza una tabella delle pagine a 2 livelli. Dunque ci sarà una tabella che mi dice in quali pagine è caricata la tabella delle pagine. Quindi per accedere alla tabella delle pagine, dovrà anzitutto interpretando i primi bit, cioè la parte più significativa dell’indirizzo, determinare l’indice che consente di accedere alla voce corrispondente nella tabella delle pagine.
Sommiamo i primi 10 bit al registro che contiene il puntatore alla pagina primaria (quindi indicano la pagina primaria), leggiamo la posizione della pagina in cui si trova il dato che ci interessa, poi all’interno di questa tabella i secondi 10 bit mi danno l’offset a cui si trova il dato.

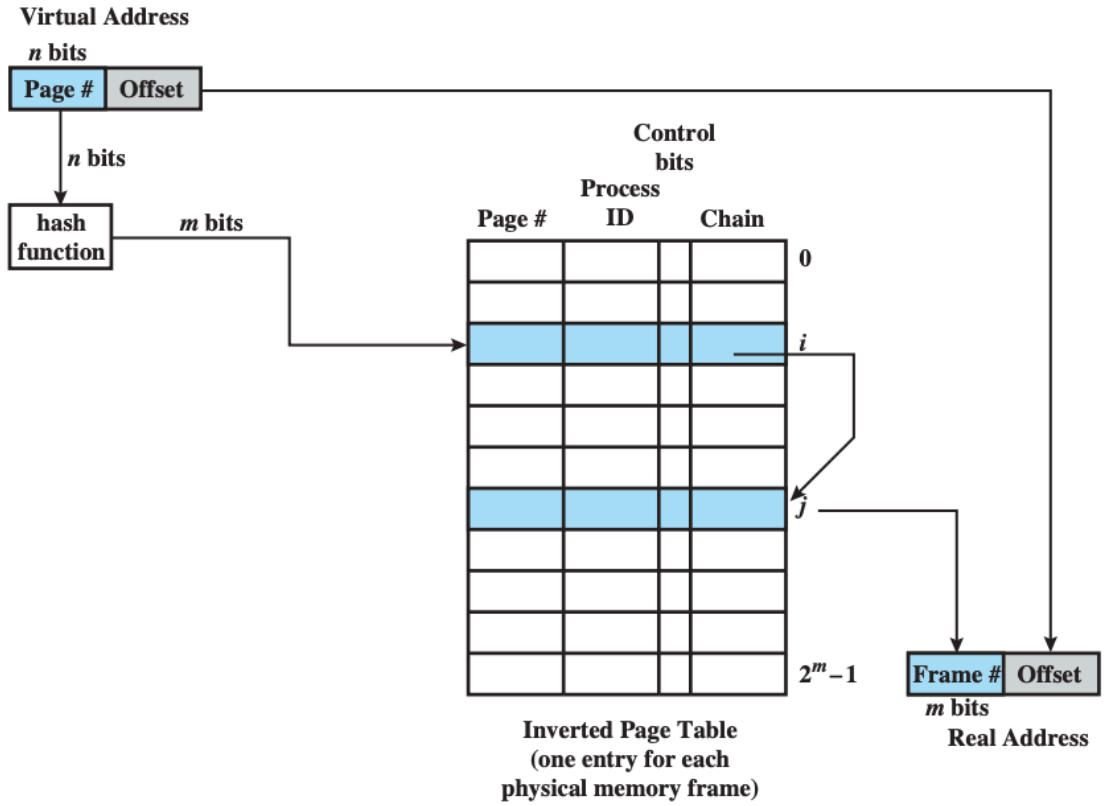


In alternativa si usa la **tabella delle pagine invertita**, poichè tabelle di pagine molto grandi dipendono dal fatto che i programmi sono molto grandi, ma non tutto il programma sta nella memoria principale, a noi ci basta avere i record delle pagine caricate effettivamente nella memoria

principale, quindi si usa questa tabella delle pagine invertita, un array indicizzato non in base alle tabelle nel nostro programma, ma un array in cui ci sono solo le pagine caricate in memoria.
Il numero di pagina è mappato in una tabella hash, la tabella è invertita nel senso che le entries sono ordinate in ordine di frame anzichè di pagina.
Campi della PTE (tabella invertita):

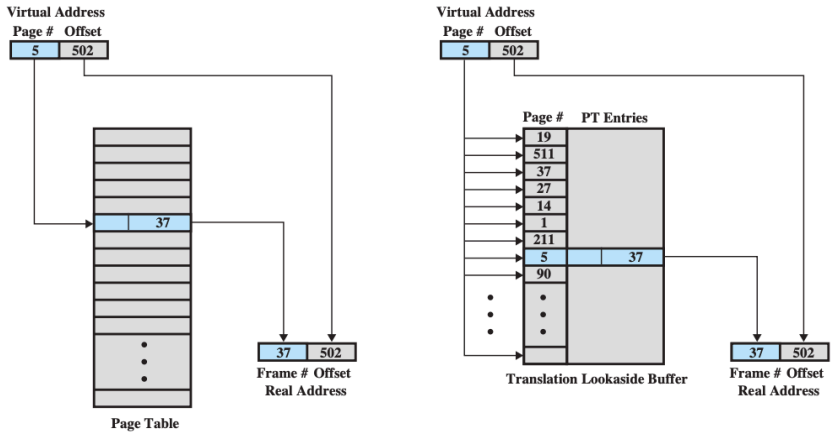
- Numero di pagina;
- Identificatore di processo;
- Bit di controllo;
- Puntatore alla entry successiva.

I primi n bit sono inseriti in una funzione di hash che ci consente di cercare il nostro frame nella tabella, trovato il numero di frame gli aggiungiamo poi l'offset contenuto nell'indirizzo.



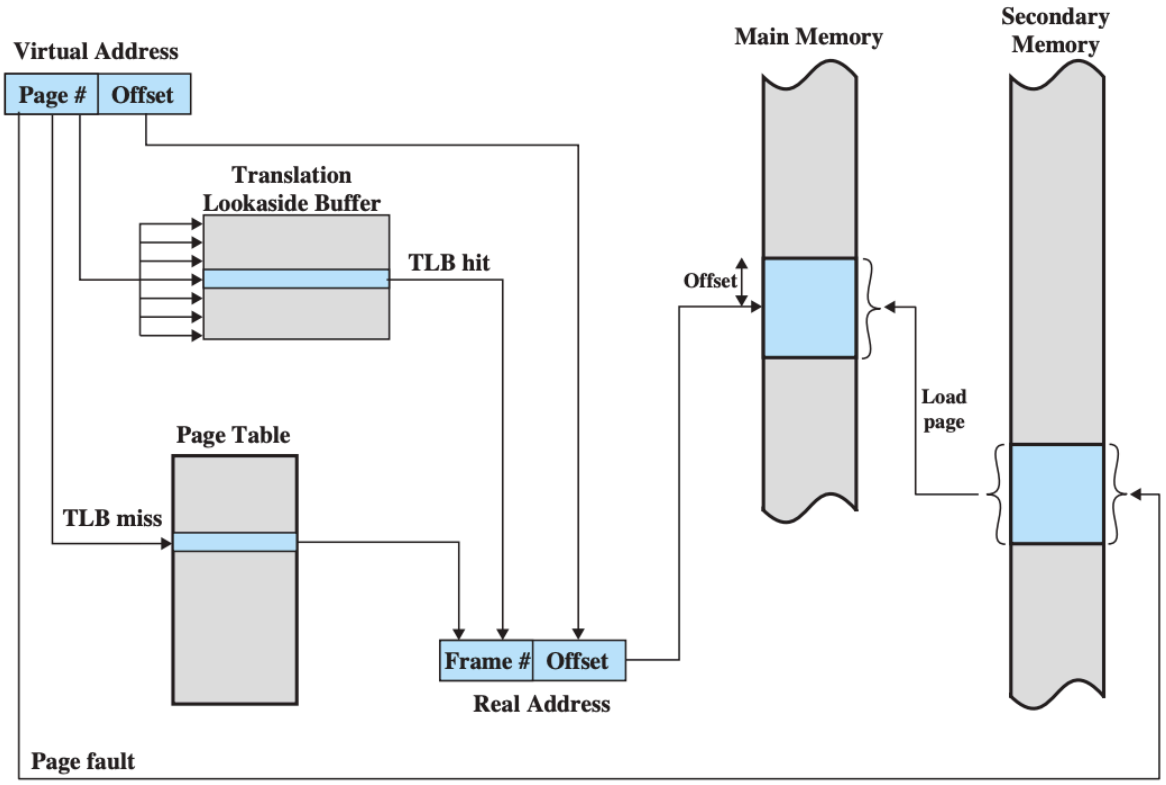
TRANSLATION LOOKASIDE BUFFER (TLB)
Quando devo tradurre un indirizzo logico ad un indirizzo fisico devo accedere alle tabelle delle pagine, nel caso in cui si ha un indirizzamento doppio si hanno quindi 2 accessi alla memoria, il **TLB** evita questo doppio accesso in memoria a ogni riferimento. È infatti una sorta di memoria veloce (**cache)** contenente le PTE (**Page Tables Entries,** campi delle tabelle di

pagine) usate di recente usate di recente, tramite una **mappa associativa,** si ricerca simultaneamente nel campo indice di pagina di tutte le mie PTE.
A sinistra nella foto abbiamo il metodo classico, a destra la ricerca viene eseguita in maniera parallela su tutti i campi, ovviamente il TLB non è la tabella delle intere pagine, ma solo di un piccolo pezzo.

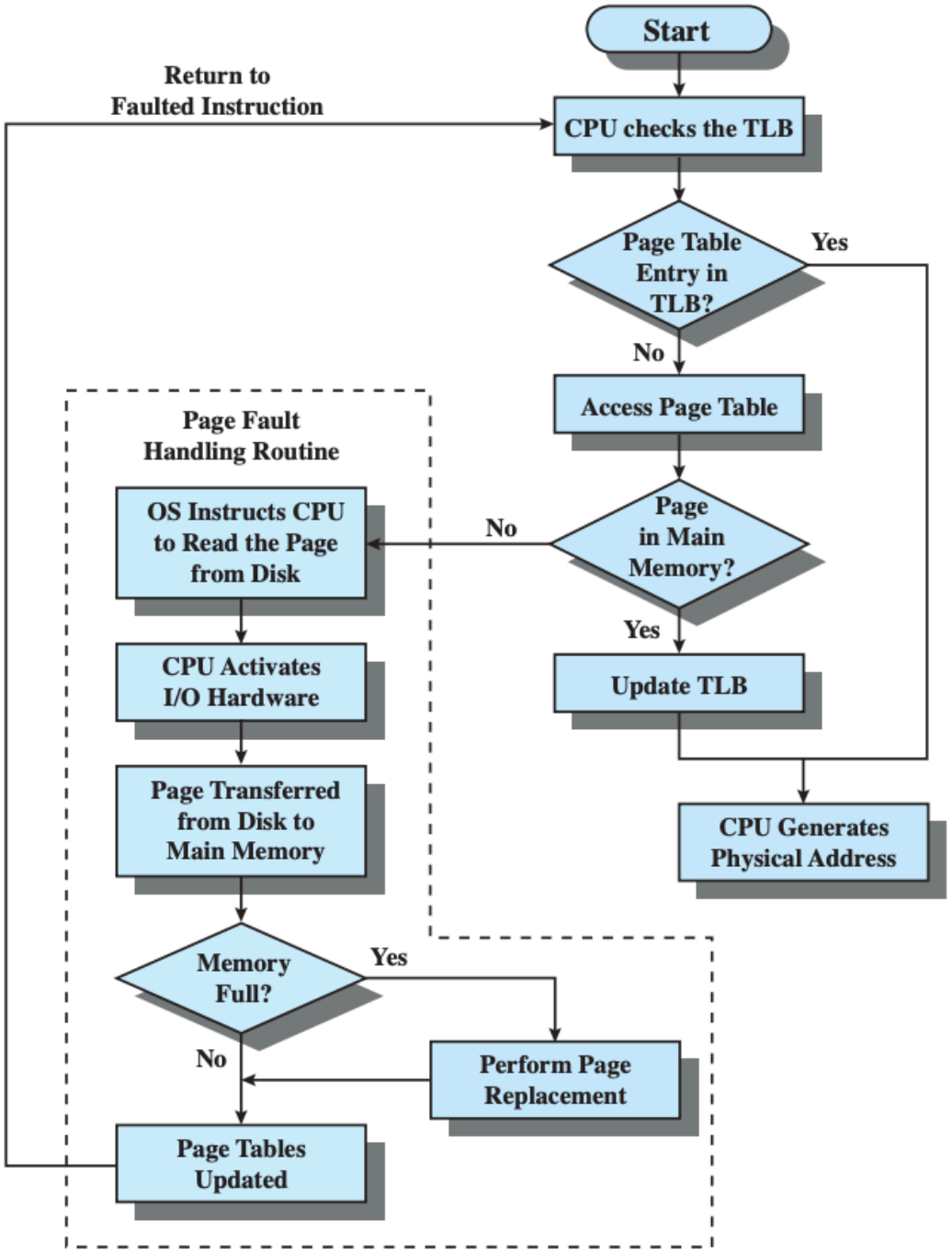


Rilocazione con TLB

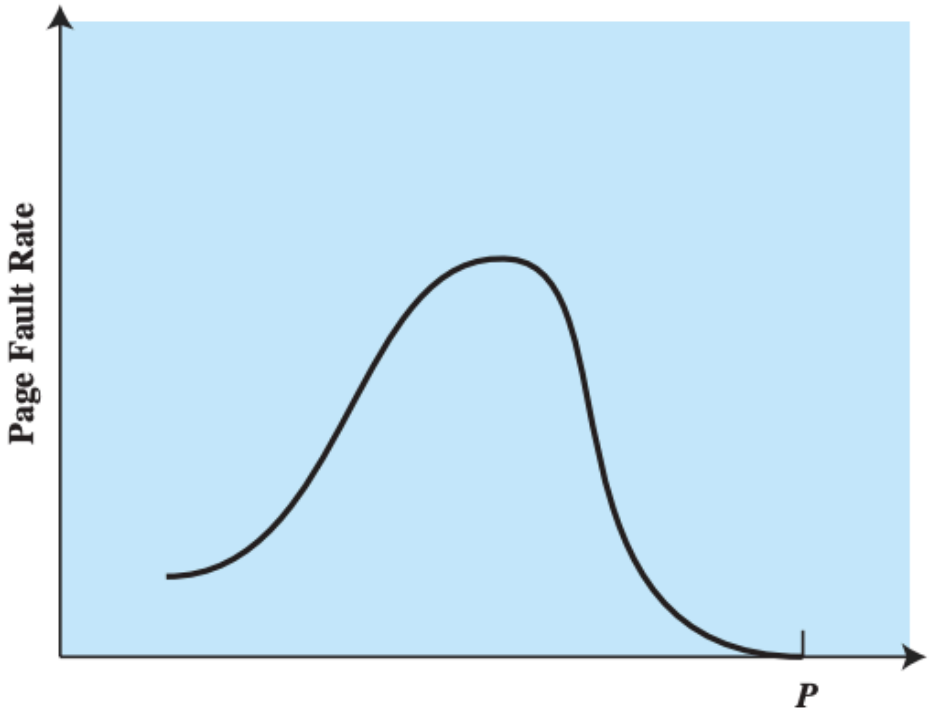
Dato quindi l'indirizzo virtuale, prima si controlla il TLB, se l'indirizzo c'è, prendiamo il frame e applichiamo l'offset per trovare l'indirizzo fisico, se non c'è nel TLB si fa il doppio accesso alla memoria, prima nella Page table e poi nella main memory, se la pagina non esiste, il processo viene interrotto con **page fault**



Questo quindi ci evita di accedere alla memoria nel caso in cui l'indirizzo sia presente nel TLB, facendo tutto nel processore, un buon risparmio di tempo, se la pagina non è neanche in memoria si inizializza la routine che gestisce l'assenza delle pagine, e la pagina viene trasferita dal disco alla memoria ovviamente se lo spazio c'è, se la memoria è piena si rimpiazza qualche vecchia pagina che probabilmente non ci serve più (le pagine da sostituire si individuano tramite un algoritmo specifico) con la nuova pagina che prima non era presente in memoria.



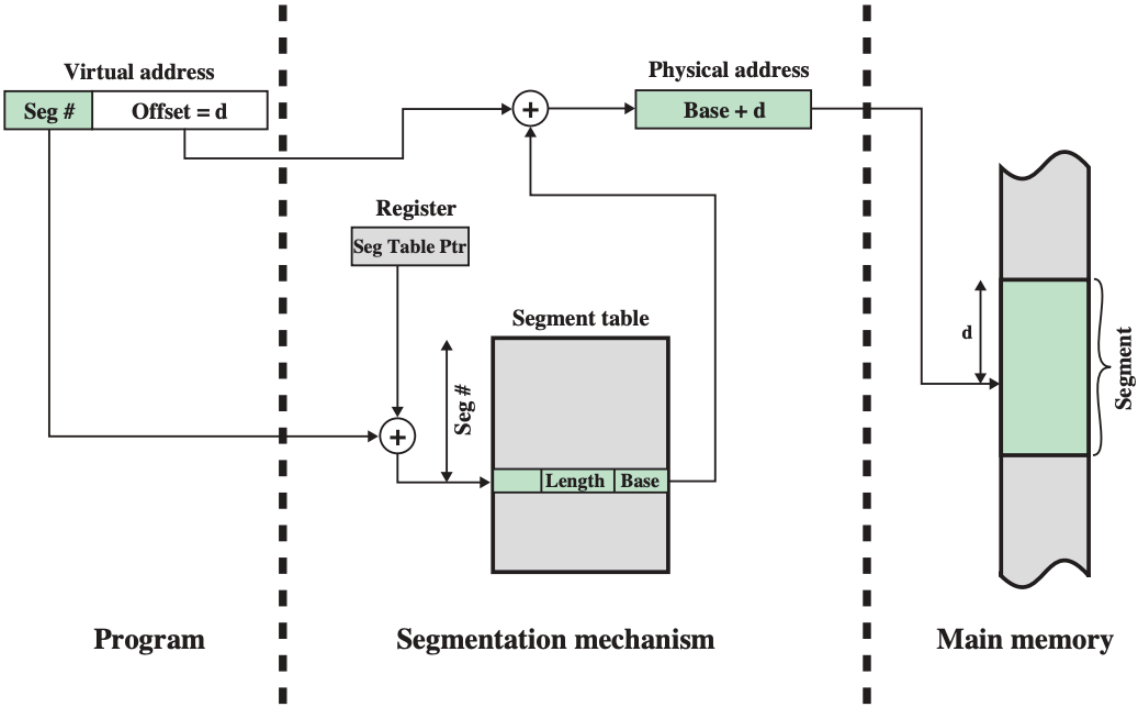
Come scegliamo la dimensione delle pagine?
Pagine troppo grandi creano frammentazione interna, pagine piccole riducono la frammentazione interna, ma aumentano la dimensione della tabella delle pagine e quindi potrebbe accadere che neanche la tabella delle pagine può stare in memoria principale.
Il trasferimento di informazioni da disco a memoria principale avviene a blocchi, quindi le cose vengono più semplici se la pagina è grande quanto un blocco, vengono peggio se è grande come mezzo blocco, ecc…
La dimensione delle pagine è importante poichè influisce sulla frequenza degli errori di pagina.



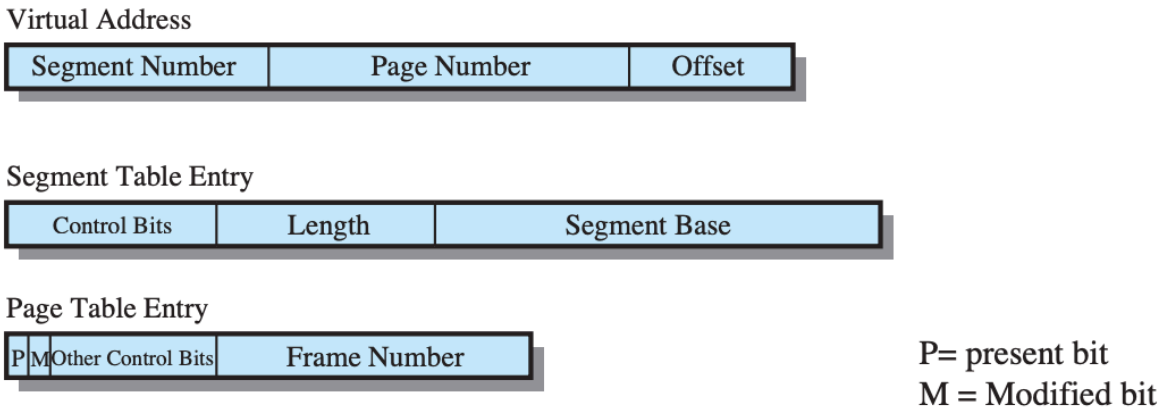
La scelta delle dimensioni è collegata alla dimensione della memoria principale, ma anche alla dimensione dei programmi (Se sono molto grandi è inutile avere pagine piccole).
Le tecniche di programmazione moderne riducono la località, questo è un problema per l'uso del TLB.
Con la **segmentazione** il programmatore vede la memoria composta da multipli spazi di indirizzamento (**segmenti**).
Si semplificano la gestione di strutture dati di dimensione variabile, si permette la modifica e ricompilazione di segmenti di programma, senza modificare il resto, si facilita la condivisione di dati fra processi e si facilita la protezione.
Per fare ciò si utilizza la **tabella dei segmenti**, ogni record della tabella dei segmenti contiene:

- L'indirizzo iniziale del corrispondente segmento in memoria;
- La lunghezza del segmento;

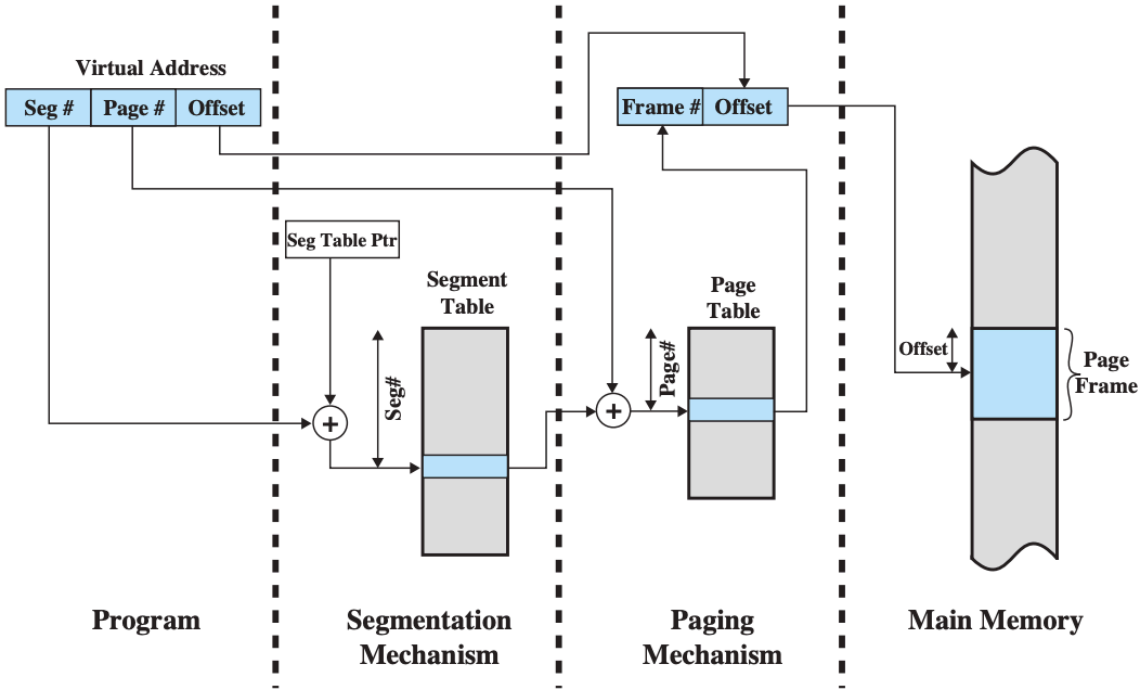
- Il flag ‘caricato’ o ‘modificato’.



Un metodo più evoluto è la **segmentazione paginata,** il programmatore realizza il suo programma con segmenti, i singoli segmenti sono divisi in pagine di dimensione fissa, uguale alla lunghezza dei frame della memoria principale.
La segmentazione era visibile al programmatore, la paginazione era trasparente e affidata al sistema operativo, con la segmentazione paginata l’indirizzo virtuale si divide in numero di segmento, numero di pagina e offset. Inoltre avremo in questo caso due strutture dati, la **Segment table e la Page table.**



Rilocazione con segmentazione paginata:



La segmentazione facilita l'implementazione della protezione e della condivisione:

- Ogni record della tabella dei segmenti contiene indirizzo base e lunghezza, per controllare accessi erronei;
- La condivisione è implementata inserendo un riferimento allo stesso segmento nella tabella dei segmenti di più processi.

MEMORIA VIRTUALE (POLITICHE DI GESTIONE)

La progettazione del gestore della memoria dipende da tre decisioni fondamentali:

- Usare o meno tecniche di memoria virtuale;
- Usare la paginazione o la segmentazione o la segmentazione paginata;
- Gli algoritmi utilizzati per i vari aspetti della gestione della memoria, ossia:

Strategia di caricamento, come caricare le pagine in memoria, **strategia di posizionamento**, dove caricare le pagine in memoria, **strategia di sostituzione**, come decido quali pagine sono sostituibili in caso di memoria piena, **gestione del resident set**, può avere dimensione fissa o variabile, l'ambito di sostituzione indica dove carico le nuove pagine dopo averne sostituite alcune vecchie.

Strategia di cleaning, A richiesta quando è necessario, oppure quando frame liberi sono vicini allo 0 faccio precleaning.

Controllo del caricamento, screening a lungo termo che influisce sul grado di multiprogrammazione.

Strategia di caricamento	A richiesta, Prepaginazione
Strategia di posizionamento	
Strategia di sostituzione	Algoritmi di base (ottimo, LRU, FIFO, orologio), Buffer per pagine
Gestione del resident set	Dimensione del resident set (fissa, variabile), Ambito di sostituzione (globale, locale)
Strategia di cleaning	a richiesta, precleaning
Controlllo del caricamento	Grado di multiprogrammazione

STRATEGIE DI CARICAMENTO

Determinano quando una pagina deve essere caricata in memoria.

A richiesta: la scelta più naturale, la pagina viene caricata, quando c'è un riferimento a quella pagina ed essa non è presente in memoria, i problemi possono essere molti errori di pagina all'avvio del processo (poichè nessuna pagina è presente all'inizio, errore di pagina = page fault = pagina non presente in memoria), il principio di località suggerisce che al crescere delle pagine caricate, gli errori di pagina si riducono a un livello molto basso.

Prepaginazione: Sono caricate altre pagine oltre a quelle che hanno determinato l'errore di pagina, sfrutta le caratteristiche dei dispositivi di memoria secondaria, ad esempio le pagine memorizzate in aree contigue di memoria secondaria sono caricate più velocemente in serie piuttosto che una alla volta, ma possono essere caricate pagine non riferite.

STRATEGIE DI POSIZIONAMENTO

Determina in quale parte della memoria principale la pagine o il segmento deve essere caricato, è importante solo per la segmentazione non paginata, può usare i criteri **First fit, best fit e next fit.**

Il posizionamento influisce sulle prestazioni anche con la paginazione nei sistemi distribuiti e i sistemi distribuiti NUMA.

STRATEGIE DI SOSTITUZIONE

Selezionano la pagina in memoria principale da sostituire quando è necessario caricare una nuova pagina, l'obiettivo ovviamente è quello di selezionare la pagina che più difficilmente sarà riferita nel futuro prossimo, bisogna però stare attenti poiché più complessa è la politica di sostituzione, maggiore è l'overhead per implementarla.

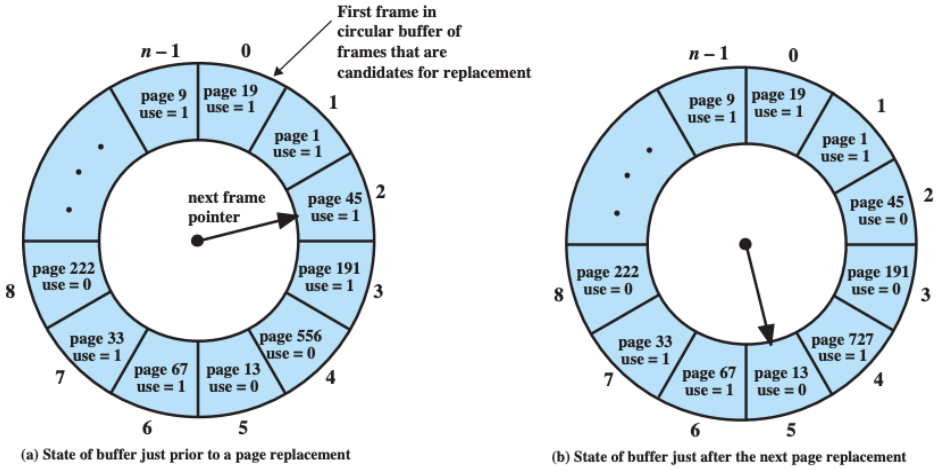
In alternativa si usano algoritmi non troppo complessi, ma che comunque danno un buon risultato.

Ci sono però dei frame che non possono essere sostituiti anche se inutilizzati, sono i frame **bloccati (locked)**, la pagina in quei frame non può essere sostituita, in frame bloccati sono caricati ad esempio il kernel del SO e le sue strutture di controllo principali, oppure Buffer I/O e aree critiche per interazioni real-time, il blocco si ottiene anche associando un 'lock bit' a ogni frame.
L'algoritmo ottimo di sostituzione sarebbe quello che sostituisce la pagina che, in futuro, sarà riferita per ultima, ciò però è ovviamente irrealizzabile, ma che può funzionare da **benchmark**, quindi come confronto di prestazioni.
LRU (Least Recently Used) è uno degli algoritmi, seleziona la pagina che da più tempo non è riferita, è ragionevole pensare che questa pagina anche in futuro non sarà riferita, il problema è che è difficile da implementare, dovrei marcare ogni pagina col tempo di ultimo riferimento oppure organizzare le pagine in una coda FIFO, ordinate quindi dalla più giovane alla più vecchia. Questo comporta un **grande overhead.**
Un'alternativa è la politica **FIFO,** le pagine vengono organizzate in un buffer circolare e vengono rimosse in ordine di caricamento, si sostituisce quindi la pagina che è stata più tempo in memoria, facile da implementare, ma il difetto sta nel fatto che ci sono parti di codice eseguite in continuazione, quindi prima o poi potrei sostituire parti di codice che si sono in memoria da molto tempo, ma vengono usate spesso.
Uno dei metodi più efficienti è la **strategia dell'orologio**, abbiamo un flag associato a ogni frame (bit di **utilizzo**), settato a 1 quando la pagina è caricata o ogni volta che la pagina è riferita.
L'algoritmo scandisce circolarmente i frame:

- Se il bit di utilizzo è 1, viene settato a 0;
- Se il bit è 0, il frame viene selezionato per la sostituzione.

Quindi il puntatore gira, dà tempo ai frame di essere utilizzati girando tutti i frame.
Una strategia più complessa distingue tra frame modificati e non modificati.
Nell'esempio pag. 556 ha il flag a 0 quindi verrà scaricata e viene inserita la pag. 727. Una pagina non verrà scaricata se nel frattempo tale pagina viene

modificata.



In questa foto dunque si vede un confronto tra i diversi algoritmi visti finora, nell'algoritmo ottimo (OPT) dovremmo andare a leggere nel futuro quale delle pagine 2,3,1 non verrà più utilizzata o comunque verrà utilizzata più in là delle altre, 1 non sarà più usata dunque sarà la pag.1 ad essere scaricata per prima. Nell'algoritmo LRU (Least recently used) guardo quella usata meno recentemente, in questo caso la pagina 3. Nel FIFO tolgo la prima pagina caricata (quella che vi è da più tempo).

Page address stream	2	3	2	1	5	2	4	5	3	2	5	2
OPT	2	2 3	2 3	2 3 1	2 3 5 F	2 3 5	4 3 5 F	4 3 5	4 3 5	2 3 5 F	2 3 5	2 3 5
LRU	2	2 3	2 3	2 3 1	2 5 1 F	2 5 1	2 5 4 F	2 5 4	3 5 4 F	3 5 2 F	3 5 2	3 5 2
FIFO	2	2 3	2 3	2 3 1	5 3 1 F	5 2 1 F	5 2 4 F	5 2 4	3 2 4 F	3 2 4	3 5 4 F	3 5 2 F
CLOCK	2*	2* 3*	2* 3*	2* 3* 1*	5* 3 1 F	5* 2* 1 F	5* 2* 4* F	5* 2* 4*	3* 2 4 F	3* 2* 4	3* 2 5* F	3* 2* 5*

BUFFER PER PAGINA

Migliora la gestione della memoria virtuale, l'idea è quella di utilizzare il **buffer per pagine**, per cui le pagine scaricate vengono inserite in due liste, **pagine libere e pagine modificate**. Le pagine libere possono essere immediatamente sovrascritte. Mentre queste pagine sono dentro le liste, di fatto sono ancora presenti dentro al sistema, per cui se arriva una richiesta le pagine possono essere recuperate rapidamente senza accedere al disco, migliorando le prestazioni.
Questo meccanismo è utile perché evita di ricaricare pagine che erano state appena rimosse, riducendo così il numero di page-fault e di operazioni di I/O costose. Le pagine modificate sono salvate a **gruppi**.

RESIDENT SET

Il SO decide quante pagine di ogni processo tenere in memoria principale. Sarebbe utile infatti decidere a priori per ogni processo quante pagine tenere in memoria, ciò è il **resident set**.
Come selezioniamo la dimensione del resident set? Avere tanti frame significa avere meno errori di pagina, però se vogliamo tanti processi chiaramente la RAM è limitata, per cui tanti frame potrebbero non decrementare gli errori di pagina.
Ci sono due possibilità:

- **Allocazione fissa**, per cui il processo ha un numero fisso di frame in cui essere eseguito, e a ogni errore di pagina si scarica una pagina del processo che lo ha determinato;
- **Allocazione variabile**, per cui il numero dei frame allocati a un processo può variare nel tempo. Quando il sistema si accorge che un processo fa tanti errori di pagina, decide di aumentargli il resident set. Se invece non fa errori di pagina, riduce il resident set per dar spazio agli altri processi.

Si hanno varie possibilità di scelta per determinare come modificare il resident set. Una prima opzione è la sostituzione in **ambito locale**, in cui ogni processo sceglie la pagina da sostituire esclusivamente tra le proprie pagine residenti. In questo modo, solo il processo che ha causato l'errore di pagina viene sospeso, e si ha un rallentamento localizzato al singolo processo coinvolto. In alternativa, con la sostituzione in **ambito globale** si considerano tutte le pagine non bloccate presenti nella memoria principale, indipendentemente dal processo a cui appartengono. In questo caso, il sistema ha maggiore libertà nella scelta delle pagine da sostituire, ma il rischio è che processi non direttamente coinvolti vengano penalizzati. A fronte di ciò, se il numero di processi attivi è limitato, si riduce anche la competizione per la memoria, migliorando l'efficienza complessiva del sistema.

Allocazione variabile, ambito locale: quando un nuovo processo è caricato, gli vengono allocati un certo numero di frames. La pagina da scaricare di solito sarà una pagina scelta fra i tanti processi, per cui più errori di pagina si fanno più lo spazio assegnato al processo aumenta. Quando c'è un errore di pagina, viene sostituita una pagina dell'insieme residente del processo che ha determinato l'errore. Periodicamente, si rivaluta lo spazio assegnato, incrementandolo o decrementandolo, per migliorare l'efficienza.
Allocazione variabile, ambito globale: come nell'allocazione fissa, al momento della creazione del processo, viene assegnato un numero di frame da assegnare al processo… con l'andare del tempo se si verifica che un processo determina troppi errori di pagina lo spazio assegnato aumenta, se invece si verifica il contrario lo spazio usato diminuisce.

WORKING SET

È **l'insieme delle pagine a cui un processo ha fatto riferimento durante un intervallo di tempo virtuale**, ovvero un intervallo espresso non in un tempo reale ma in numero di riferimenti a memoria. Esso rappresenta le pagine attivamente utilizzate da un processo in un determinato periodo, e serve a stimare quanta memoria è effettivamente necessaria affinché il processo possa continuare a eseguire senza incorrere frequentemente in errori di pagina.
Un processo con un working set ampio dovrebbe avere un resident set ampio. Il pallino indica che il working set non si è modificato rispetto al passo precedente. Man mano che si aumenta la dimensione del delta, i casi in cui il working set non si modifica aumentano.
Una strategia possibile per il working set: controllare periodicamente il working set di ogni processo, rimuovere dall'insieme residente le pagine che non sono nel working set, un processo può essere eseguito solo se il suo working è interamente in memoria, implementazione difficile.
Un'alternativa è agire sulla frequenza degli errori di pagina: quando gli errori di pagina diventano troppo ravvicinati, si aggiunge una pagina all'insieme resident, altrimenti si scaricano tutte le pagine non utilizzate dall'ultimo errore di pagina, e tutte le altre diventano candidate alla prossima.
Page fault frequency: quando c'è un errore di pagina se il tempo virtuale dall'errore di pagina precedente è inferiore a una soglia F, si aggiunge una pagina all'insieme residente, altrimenti si scaricano tutte le pagine con il bit di utilizzo 0 e si azzera il bit di utilizzo delle rimanenti.

Variable-Interval Sampled Working Set (VSWS)

- Valuta il working set a intervalli di campionamento basati sul tempo virtuale
- Diretto da tre parametri
  - Durata minima dell'intervallo di campionamento
  - Durata massima dell'intervallo di campionamento
  - Numero degli errori di pagina consentiti tra gli intervalli di campionamento

POLITICA DI CLEANING

Cleaning su richiesta

La pagina è salvata in memoria secondaria solo quando deve essere sostituita

Precleaning

Permette il salvataggio di pagine a lotti

POLITICA DI CARICAMENTO

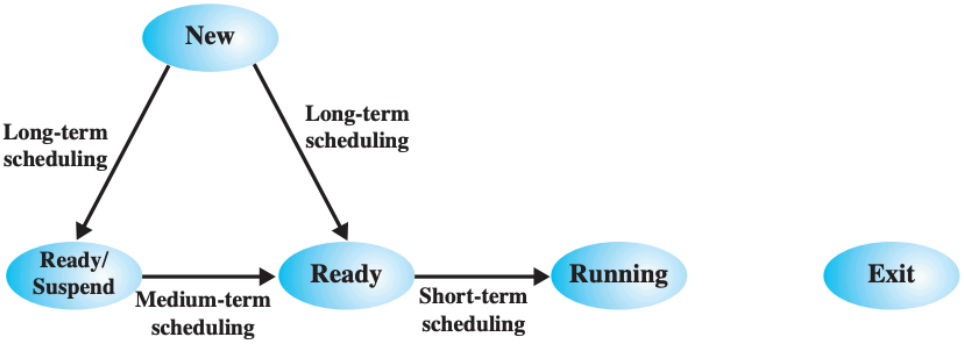
- Determina il numero dei processi residenti in memoria principale
  - grado di multiprogrammazione
  - Pochi processi = molte occasioni con tutti i processi bloccati
  - Troppi processi = trashing
- Per ridurre il grado di multiprogrammazione può essere necessario scaricare uno o più processi.
- Sei possibili politiche
  - processo con priorità minima
  - processo che provoca errori di pagina
  - ultimo processo attivato
  - processo con l'insieme residente più piccolo
  - processo più grande
  - processo col massimo tempo di esecuzione rimanente

SCHEDULING

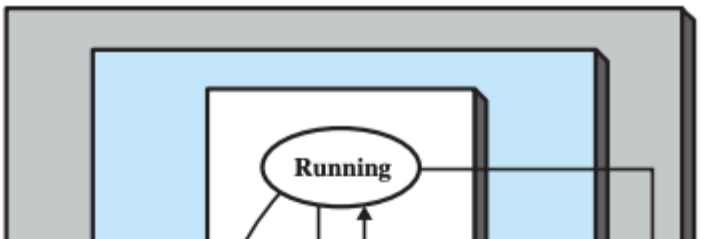
Lo scheduling deve essere il più efficiente possibile:

Lungo termine	Inserimento nella lista dei processi da eseguire
Medio termine	Aggiunta ai processi caricati (parzialmente o totalmente) in memoria principale
Breve termine	Selezione, tra i processi pronti, del processo da eseguire
Scheduling I/O	Seleziona il processo in attesa di I/O da assegnare a un dispositivo disponibile

Lo scopo della schedulazione è quello di assegnare al processore i processi da eseguire in modo da realizzare gli obiettivi del sistema, per esempio il **tempo di risposta del processore**, ci sono processi interattivi, in cui l'utente interagisce con il processo e quello che vorremmo da questi processi è una risposta immediata, un altro aspetto è il **throughput**, quanti lavori riusciamo a fare nel minor periodo di tempo e l'**efficienza del processore,** cioè si deve sempre utilizzare quanto più processore possibile.
Un processo passa attraverso nuovi stati, da processo **New** a processo **Ready** agisce la schedulazione a lungo termine, che agisce anche per i processi **Ready/Suspend,** poi lo scheduling a medio termine deciderà quale di questi processi potrà passare in stato di **Ready,** quello short-term decide quale dei processi verrà messo in esecuzione.
Lo scheduling a medio termine gestisce anche i processi Bloccati o sospesi.

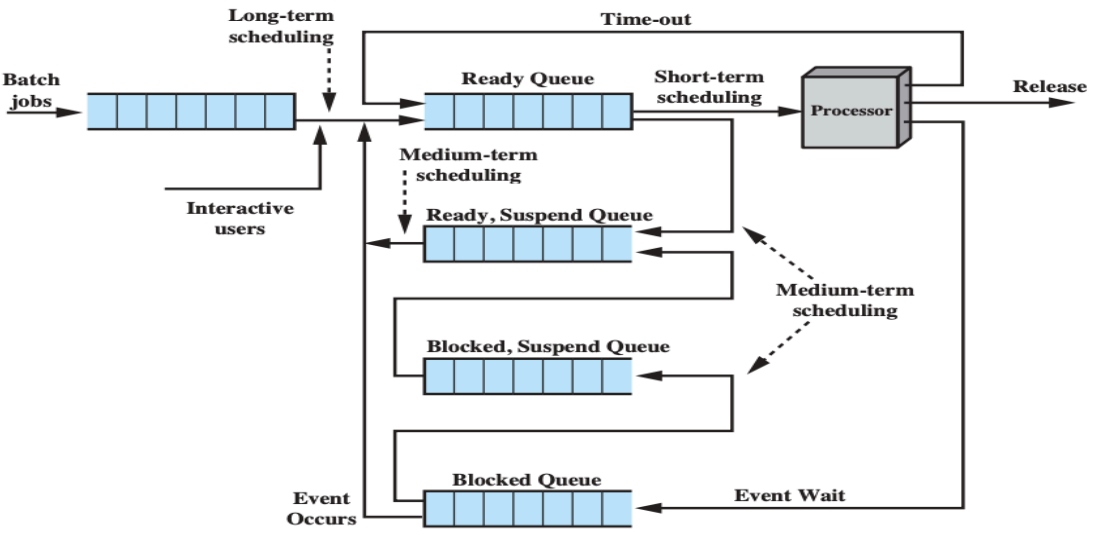






Tutti gli stati possibili di un processo:

- Ready;
- Running;
- Blocked;
- Blocked/Suspend;
- Ready/Suspend;
- New;
- Exit.



SCHEDULATORE A LUNGO TERMINE

Determina quali programmi sono ammessi nel sistema per l'esecuzione.

Controlla il grado di multiprogrammazione:

- Più processi creati = più piccole frazioni di tempo di esecuzione assegnate ai processi, i processi quindi avanzano molto lentamente;
- Per fornire servizi soddisfacenti all'insieme corrente di processi, è opportuno limitarne il numero.

Deve quindi adottare delle politiche per creare un processo dalla coda dei nuovi job, si deve decidere:

- Quando il SO può accettare uno o più processi;
- Quali job trasformare in processi, la scelta può adottare ad esempio una politica First come, first served (FCFS, oppure FIFO), una politica di priorità, può calcolare una stima del tempo di esecuzione atteso, oppure può considerare i requisiti I/O.

SCHEDULATORE A MEDIO TERMINE

È parte della funzione di swapping, le decisioni di caricamento/scaricamento sono correlate al **controllo del grado di multiprogrammazione** come per lo schedulatore a lungo termine, ma anche ai **requisiti di memoria dei processi**.

DISPATCHER (SCHEDULATORE A BREVE TERMINE)

Il **dispatcher** esegue la schedulazione a breve termine, è un programma che viene eseguito con grande frequenza (ogni volta che un processo deve essere assegnato), decide quale tra i processi pronti deve essere eseguito.

È attivato da:

- Interruzioni di clock (il tempo assegnato a un processo è stato raggiunto e quindi il processo deve lasciare il processore per altri processi);
- Interruzioni I/O (il processo deve interagire con un dispositivo quindi sarà interrotto per diverso tempo);
- Chiamate di sistema (un processo esegue una system call, si sospende);
- Segnali (come semafori)

Lo scheduling a breve termine è ciò che ci interessa di più, è un po' la chiave di volta di un sistema operativo, poichè essendo chiamata molto frequentemente deve usare algoritmi rapidi e semplici, non si possono adottare politiche molto complesse, d'altra parte però se non funziona viene danneggiato tutto il sistema.

Si devono usare quindi dei criteri che vengono stabiliti dal progettista che

permettano di allocare la CPU in modo da ottimizzare alcuni aspetti del comportamento del sistema.

Criteri orientati all'utente

Sono quei criteri relativi alla percezione del comportamento del sistema da parte degli utenti (o dei processi), ad esempio il tempo di risposta nei sistemi interattivi, questi criteri sono importanti in tutti i sistemi.

Criteri orientati al sistema

Sono relativi all'efficienza dell'utilizzazione del processore, ad esempio la frequenza del completamento dei processi, sono trascurabili nei sistemi a singolo utente.

Criteri relativi alle prestazioni

Sono quantitativi e facili da misurare (es: tempo di risposta)

Criteri non relativi alle prestazioni

Sono qualitativi e difficili da misurare (es: predicibilità)

CRITERI ORIENTATI ALL'UTENTE, relativi alle prestazioni

Tempo di risposta:

Per un processo interattivo, questo è il **tempo che trascorre dall'invio di una richiesta fino a quando la risposta non comincia ad essere ricevuta**. Spesso un processo può cominciare a produrre qualche output per l'utente. mentre continua l'esecuzione. Perciò, dal punto di vista dell'utente, questa è una misura migliore del tempo di turnaround. La disciplina di scheduling dovrebbe tentare di raggiungere bassi tempi di risposta e di massimizzare il numero di utenti interattivi che hanno un tempo di risposta accettabile.

Tempo di turnaround:

È l'intervallo di tempo tra l'invio di un processo e il suo completamento. Comprende il tempo di esecuzione e il tempo speso in attesa delle risorse, compreso il processore. Questa è una misura appropriata per i job batch.

Scadenze:

Quando si può specificare il termine di completamento di un processo, la disciplina di scheduling può subordinare altri obiettivi oltre a quello di massimizzare la percentuale di scadenze raggiunta.

Altri criteri orientati all'utente non relativo alle prestazioni è la **prevedibilità**, un dato job può essere eseguito nello stesso tempo e allo stesso costo, indipendentemente dal carico del sistema. Un'ampia variazione nel tempo di risposta o nel tempo di turnaround è fastidiosa per l'utente: potrebbe segnalare un'ampia oscillazione nel carico di lavoro del sistema oppure la necessità di adattare il sistema per evitare l'instabilità

CRITERI ORIENTATI AL SISTEMA, relativi alle prestazioni

Throughput:

La strategia di scheduling dovrebbe tentare di massimizzare il numero di processi completati per unità di tempo. Così si misura la quantità di lavoro eseguito. che dipende chiaramente dalla lunghezza media di un processo, ma che è anche influenzata dalla strategia di scheduling che può avere effetti sull'utilizzazione.

Utilizzazione del processore:

Questa è la percentuale del tempo in cui il processore è occupato. Per un sistema costoso e condiviso, questo è un criterio significativo, mentre in sistemi a singolo utente e in alcuni altri sistemi, come i sistemi a tempo reale, questo criterio è meno importante.

Fairness:

In assenza di indicazioni da parte dell'utente, oppure del sistema, i processi devono essere trattati allo stesso modo, e nessun processo deve subire starvation.

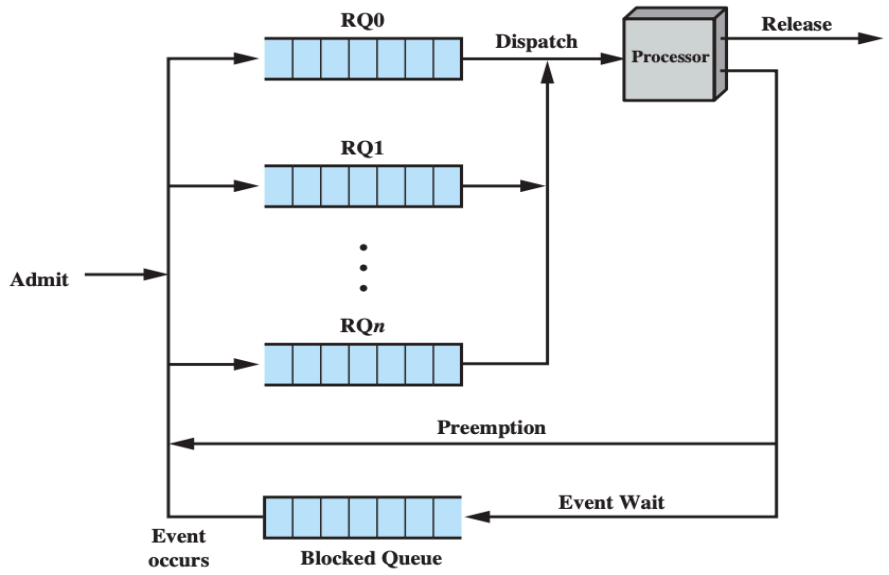
Priorità:

Quando si assegna una priorità ai processi, la strategia di scheduling deve favorire i processi a più alta priorità.

Bilanciamento delle risorse:

La strategia di scheduling deve tenere impegnate le risorse del sistema, favorendo i processi che sotto utilizzano le risorse più impegnate. Questo criterio coinvolge anche lo scheduling a medio e lungo temine.

Gestione a priorità



FUNZIONE DI SELEZIONE

Determina quale processo, tra quelli pronti, è selezionato per l'esecuzione.

- Può essere basata su
  - priorità,
  - esigenza di risorse,
  - caratteristiche di esecuzione
- Nel terzo caso, si tiene conto dei parametri
  - w = tempo di attesa
  - e = tempo di esecuzione
  - s = tempo stimato di esecuzione (compreso e)

MODO DI DECISIONE

Specifica il momento in cui è eseguita la decisione:

- Senza prerilascio, il processo in esecuzione continua fino al termine o fino a un'interruzione per I/O;
- Con prerilascio, il processo in esecuzione può essere interrotto dal SO e posto in stato di pronto, il prerilascio può avvenire:
  1. Quando arriva un nuovo processo;
  2. Quando un'interruzione riattiva un processo sospeso;
  3. Periodicamente, in base a un clock.

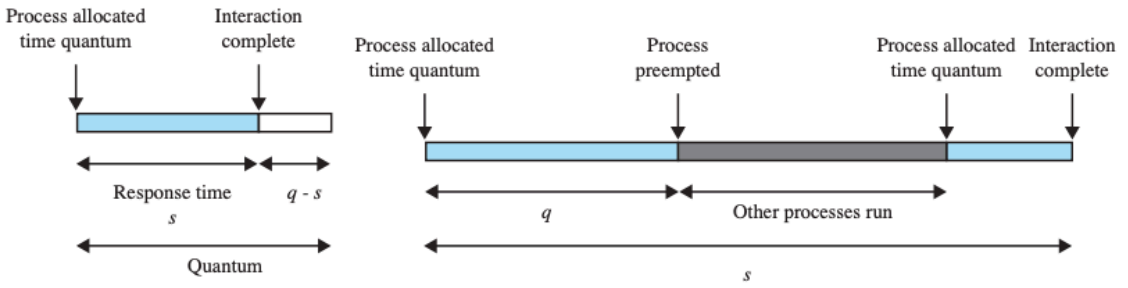
FIRST COME FIRST SERVED (FCFS)

È la gestione più semplice (FIFO), ogni processo pronto è inserito nella coda pronti, quando il processo in esecuzione termina o è sospeso, viene selezionato per l'esecuzione il processo che è stato più a lungo nella coda pronti, favorisce i processi lunghi rispetto a quelli brevi e i processi **processor-bound (non interattivi, processi batch di solito)** rispetto a quelli **I/O** bound.

ROUND ROBIN

Politica molto usata soprattutto su sistemi a multi-utente.
C'è un clock che definisce il **time-slice,** una fetta di tempo assegnata a ciascun processo, quando ad un processo viene assegnato il processore, se il processo si sospende, tornerà all'ultimo posto della coda, se non si sospende entro il time-slice il processo viene pre rilasciato.

Il principale aspetto progettuale è la durata del **quanto** di tempo, ogni processo interattivo dovrebbe riuscire a fare i propri compiti nel quanto.
È molto efficiente nei sistemi time-sharing e orientati alle transazioni.
Favorisce i processi processor-bound rispetto a quelli I/O bound.
L'alternativa è quella del **VRR (virtual round robin),** dove i processori che non utilizzano tutto il quanto di tempo sono premiati alla successiva esecuzione.



SHORTEST PROCESS NEXT (SPN)
Politica che massimizza il throughput, seleziona il processo col minimo tempo stimato di esecuzione, non è previsto il prerilascio.
Un processo breve quindi può saltare in testa alla coda pronti, si massimizza quindi il numero di processi finiti in un determinato lasso di tempo.
C'è però un problema, cioè la possibile starvation per processi lunghi.
È inadatto nei sistemi time-sharing proprio per l'assenza di prerilascio.
Inoltre è difficile conoscere o stimare il tempo di esecuzione, ci si potrebbe basare sulle esecuzioni precedenti:

- **Media aritmetica della durata delle esecuzioni precedenti:**

$$S_{n+1} = \frac{1}{n}\sum_{i=1}^{n} T_i = \frac{T_n + (n-1)S_{n-1}}{n}$$

- **Media esponenziale:**

$$S_{n+1} = \alpha \sum_{i=1}^{n} (1-\alpha)^{n-i} T_i + (1-\alpha)^n S_1 = \alpha T_n + (1-\alpha) S_n$$

$0 < \alpha < 1.$

Se il tempo di esecuzione supera la stima del programmatore, il programma può essere terminato.

SHORTEST REMAINING TIME (SRT)
Sostanzialmente è un SPN, ma con prerilascio, se c'è un processo che ha un tempo di esecuzione stimato minore del tempo rimanente al processo in esecuzione il processo in esecuzione viene prerilasciato e parte il nuovo processo.
I problemi però sono sempre possibile starvation per processi lunghi, tuttavia si migliora il tempo di turnaround rispetto a SPN, poichè i job brevi hanno precedenza immediata sui job lunghi.
Evita i prerilasci addizionali di Round Robin e non favorisce i lavori lunghi come FCFS.
Un altro problema però è l'overhead per la stima del tempo di esecuzione.

HIGHEST RESPONSE RATIO NEXT (HRRN)

$$R = \frac{w + s}{s}$$

ove w = tempo di attesa e s = tempo stimato di esecuzione.

Cerca di eliminare il problema della starvation tenendo conto anche del tempo di attesa.
Seleziona il processo con il più alto rapporto R di risposta (**tempo di turnaround normalizzato),** questa è più alta per i processi brevi poiché se s è piccola la frazione è molto alta, MA processi che sono in attesa da molto tempo avranno una w molto grande e quindi finiranno per essere eseguiti**.**
Quindi i processi brevi sono comunque favoriti, ma si evita la starvation.

CONFRONTO TRA LE DIVERSE POLITICHE

	FCFS	Round robin	SPN	SRT	HRRN	Feedback
Selection function	$\max w$	constant	$\min s$	$\min(s-e)$	$\max \frac{w+s}{s}$	$\cdots$
Decision mode	Non-preemptive	Preemptive (at time quantum)	Non-preemptive	Preemptive (at arrival)	Non-preemptive	Preemptive (at time quantum)
ThroughPut	Not emphasized	May be low if quantum is too small	High	High	High	Not emphasized
Response time	May be high, especially if there is a large variance in process execution times	Provides good response time for short processes	Provides good response time for short processes	Provides good response time	Provides good response time	Not emphasized
Overhead	Minimum	Minimum	Can be high	Can be high	Can be high	Can be high
Effect on processes	Penalizes short processes; penalizes I/O bound processes	Fair treatment	Penalizes long processes	Penalizes long processes	Good balance	May favor I/O bound processes
Starvation	No	No	Possible	Possible	No	Possible

Cosa fare se abbiamo un sistema multi-thread?

Dobbiamo tenere conto di due aspetti, a quali processi assegnare la CPU, ma anche a quale utilizzatore assegnare il processore.

SCHEDULAZIONE FAIR-SHARE

Usa politiche di schedulazione basate su insiemi di processi.

Ogni utilizzatore ha assegnata una ‘frazione’ del processore

L’obiettivo è monitorare l’uso per dare meno risorse a chi ne ha avute più della frazione spettante e più risorse a chi ne ha avute di meno.

Si può assegnare una priorità determinata con:

$$CPU_j(i) = \frac{CPU_j(i-1)}{2}, \qquad GCPU_k(i) = \frac{CPU_k(i-1)}{2},$$

$$P_j(i) = Base_j + \frac{CPU_j(i)}{2} + \frac{GCPU_k(i)}{4 \cdot W_k}$$

ove
$CPU_j(i) =$ uso del processore del processo j nell'intervallo i
$GCPU_k(i) =$ uso del processore del gruppo k nell'intervallo i
$P_j(i) =$ priorità del processo j all'inizio dell'intervallo i
$Base_j =$ priorità iniziale del processo j
$W_k =$ peso assegnato al gruppo k $\left(\sum_k W_k = 1\right)$

SCHEDULAZIONE UNIX TRADIZIONALE
Ottimizzato per sistemi time-sharing.
Si ha un buon tempo di risposta per i processi interattivi, evitando starvation dei processi in background.
Si ha un feedback multilivello con round-robin in ogni coda di priorità con prerilascio dopo 1 secondo.
La priorità è basata sul tipo di processo e storia dell'esecuzione:

$$CPU_j(i) = \frac{CPU_j(i-1)}{2},$$

$$P_j(i) = Base_j + \frac{CPU_j(i)}{2} + nice_i$$

ove
$CPU_j(i) =$ uso del processore del processo j nell'intervallo i
$P_j(i) =$ priorità del processo j all'inizio dell'intervallo i
$Base_j =$ priorità iniziale del processo j
$nice_j =$ fattore di aggiustamento controllabile dall'utilizzatore

La principale funzione della priorità è quella di tenere separati i processi di gestione delle risorse, i pesi di CPU e nice sono limitati in modo da tenere separate le seguenti fasce di processi:

- **Swapper;**
- **Controllo dei dispositivi I/O a blocchi;**
- **Manipolazione di file;**
- **Controllo dei dispositivi I/O a caratteri;**
- **Processi utente.**

INPUT/OUTPUT

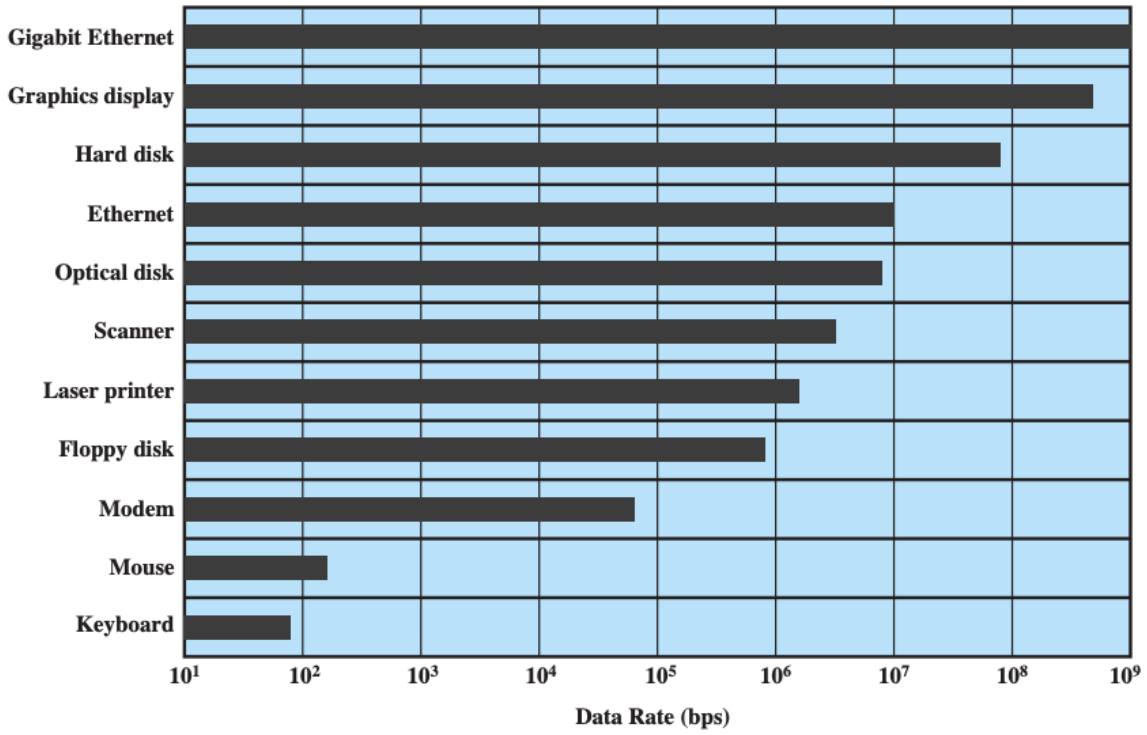
Sono molto più lenti del sistema centrale, li possiamo dividere in 3 categorie:

- Dispositivi che servono per comunicare con l'utilizzatore (**leggibili dall'uomo**), stampanti, terminali, video display, tastiera, mouse.
- Dispositivi leggibili dalla macchina, che servono per comunicare con l'apparecchiatura elettronica, ad esempio i drive di dischi e nastri, chiavette USB, sensori, attuatori.
- Dispositivi di comunicazione usati per comunicare con dispositivi remoti, come i modem.

Le principali caratteristiche da esaminare dei dispositivi I/O sono:

- **Velocità del trasferimento:** sono più lenti del sistema e la velocità dipende dal dispositivo, possono esserci grandi differenze tra un dispositivo e un altro;
- **Applicazioni,** l'uso di un particolare dispositivo influenza il software;
- **Complessità di controllo,** differenti dispositivi richiedono differenti software. La complessità è parzialmente filtrata dalla complessità del modulo I/O che controlla il dispositivo.
- **Unità di trasferimento,** possono esserci dispositivi a caratteri (tastiera o stampante), che trasmettono un carattere alla volta o a blocchi (il disco rigido), trasferisce un blocco di qualche kilobyte alla volta;
- **Rappresentazione dei dati,** dispositivi differenti usano codifiche differenti.
- **Condizioni di errore,** I tipi di errore e il modo in cui sono segnalati differiscono da un dispositivo all'altro.

Velocità di trasferimento:



Funzioni I/O

Ci sono 2 modi per gestire l'input/output:

I/O PROGRAMMATO

Il processore genera un comando di I/O per conto di un processo che rimane in attesa attiva durante il completamento dell'operazione.

I/O GUIDATO DALLE INTERRUZIONI

Il processore genera un comando di I/O per conto di un processo:

- **non-bloccante**: il processore continua a eseguire istruzioni del processo che ha eseguito il comando I/O;
- **bloccante**: il controllo passa al SO che sospende il processo corrente e ne schedula un altro.

È possibile inoltre usare un **DMA**, un modulo che controlla lo scambio di dati tra memoria principale e dispositivo I/O.

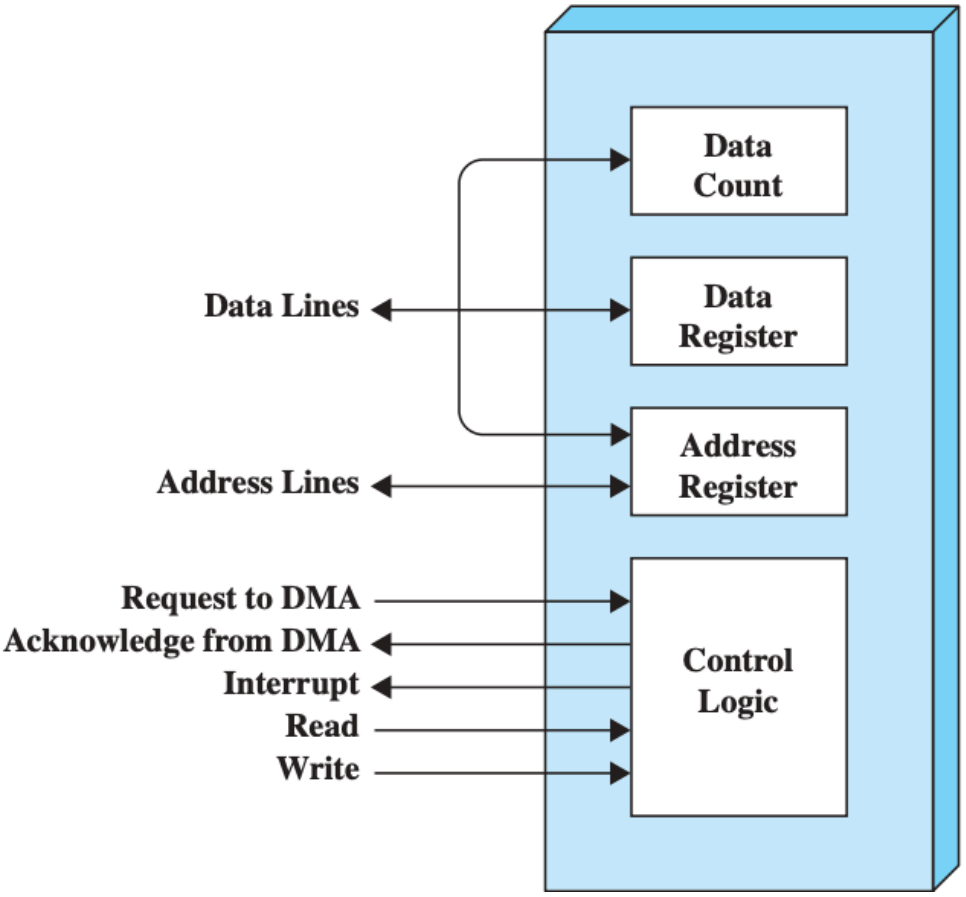
Evoluzione della funzione I/O

1. Il processore controlla direttamente il dispositivo periferico
2. Viene aggiunto un controllore o modulo I/O;
3. Al controllore vengono introdotte le interruzioni.

4. Col DMA, il modulo I/O ha controllo diretto sulla memoria;
5. Il modulo I/O diventa un processore separato con un suo instruction set specializzato (**canale di I/O)**.
6. Il modulo I/O viene dotato di memoria locale e diventa un computer vero e proprio (**processore di I/O).**

Via via, il processore è stato coinvolto sempre meno nella gestione input output.

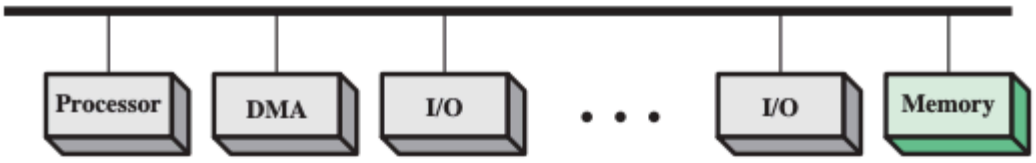
DMA



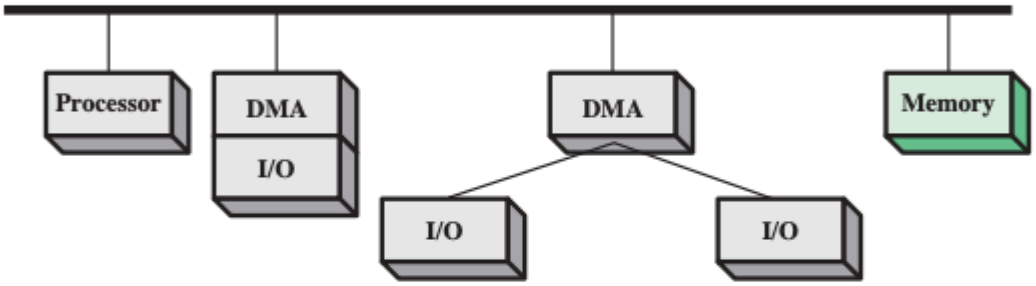
CONFIGURAZIONI DI DMA

Come organizziamo il nostro DMA, ci sono diverse possibilità, la più semplice è la **(a)**, un singolo bus a cui è connesso tutto, la memoria, il processore, il DMA e i dispositivi I/O, il vantaggio è la semplicità, ma il difetto sta nel fatto che sulla stessa linea devono passare i dati dal dispositivo al DMA, dal DMA alla memoria e i dati fra lo scambio di dati e processore, allora può essere preferibile il modello **(b)** in cui c'è ancora un singolo bus, ma i dispositivi sono ancora connessi, ognuno con il proprio DMA, la terza possibilità **(c)** è creare due bus, uno a cui sono collegati tutti i dispositivi I/O che è collegato al DMA e l'altro bus,

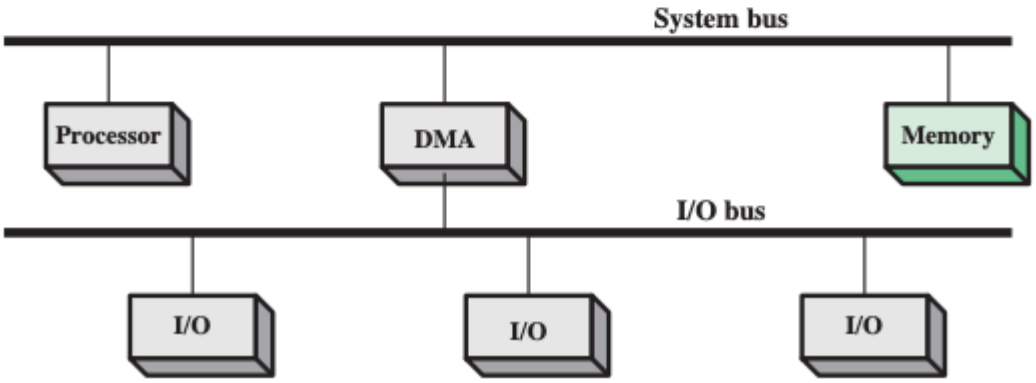
collegato agli elementi del sistema.



(a) Single-bus, detached DMA



(b) Single-bus, Integrated DMA-I/O



OBIETTIVI DI PROGETTAZIONE

Efficienza:

È il maggiore obiettivo poichè le operazioni di I/O costituiscono un bottleneck poichè i dispositivi I/O sono molto lenti rispetto a RAM e processore, soprattutto per I/O dal disco.

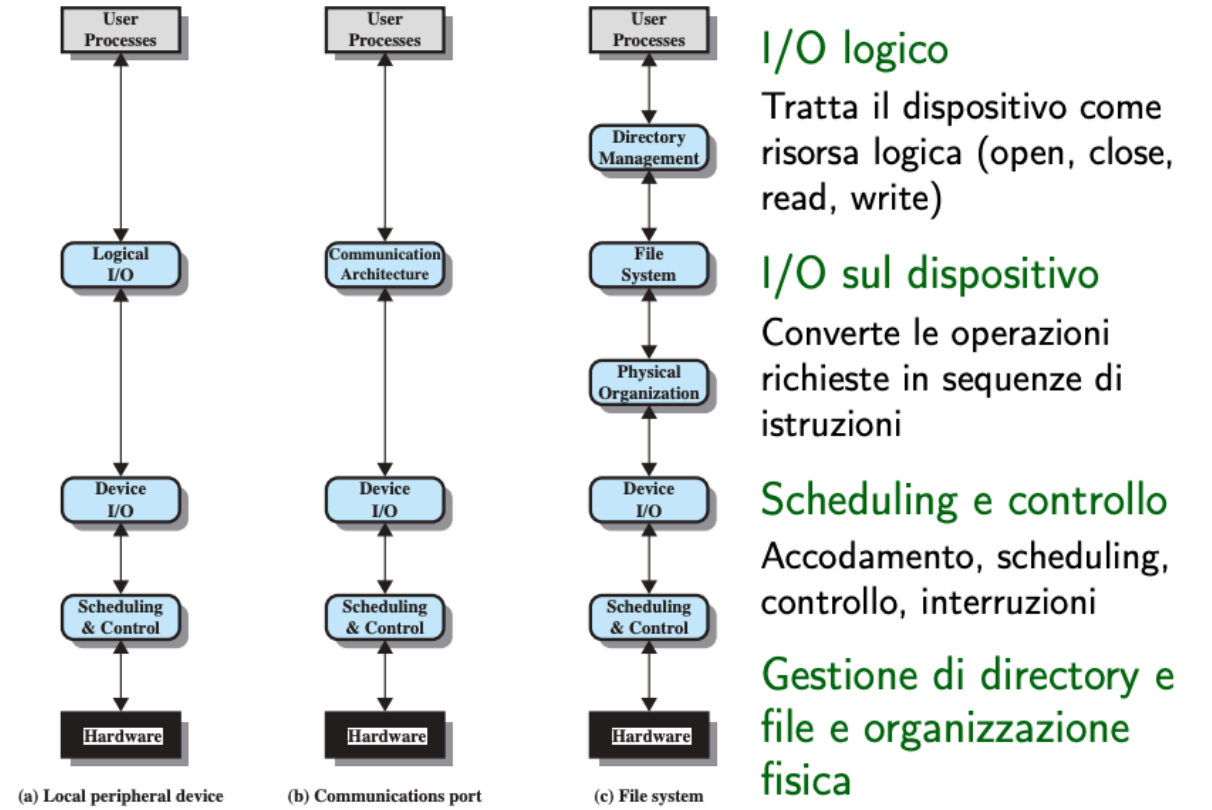
Generalità:

Si ricerca un'uniformità nella gestione dei dispositivi:

- nell'interfaccia verso i progressi;
- nella gestione dei dispositivi da parte del SO e le relative operazioni.

Questo però è difficile da realizzare per le diversità tecniche dei dispositivi, si cerca di realizzare un approccio gerarchico per nascondere i dettagli dei dispositivi nelle routine di basso livello.

STRUTTURA LOGICA



/output Sistemi operativi 2023/24

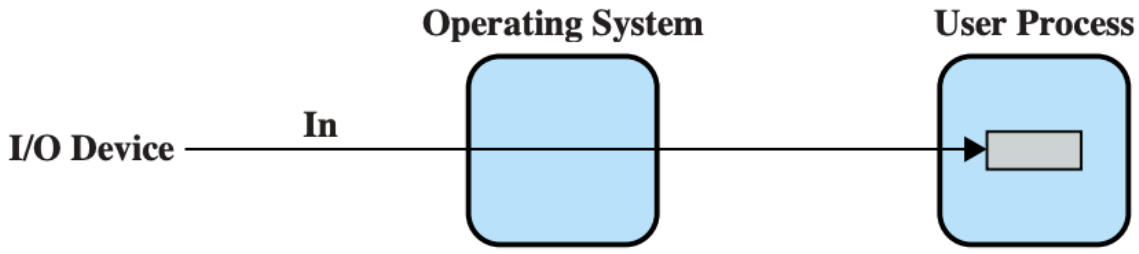
BUFFERING

Soprattutto quando abbiamo dispositivi a blocchi può essere utile eseguire trasferimenti in anticipo sulla richiesta e trasferimenti in output in ritardo rispetto alla richiesta, si ipotizza che alcune informazioni saranno richieste e perciò si cerca di anticipare.
Quindi quando si scrive ad esempio su dispositivi a blocchi, si può scrivere una parte dei dati in una parte riservata della memoria principale (**cache del disco**) e stanno lì finché non verranno caricati sul disco al momento opportuno.
(nei dispositivi a blocchi l'informazione è memorizzata in blocchi di dimensione fissa e si trasferisce un blocco alla volta, nei dispositivi

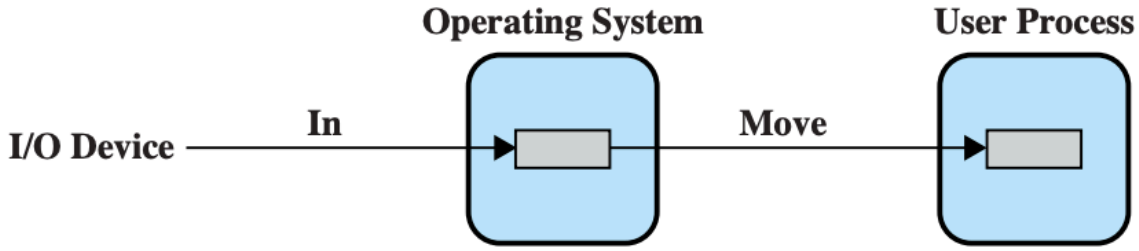
a flusso l'informazione non è strutturata in blocchi e il trasferimento dei dati avviene come un flusso di bytes).

Buffer singolo

Col buffering i dati vengono quindi trasferiti prima in quest'area di memoria principale e poi vengono lette dal buffer al processo.



(a) No buffering



Il buffer singolo **per un dispositivo a blocchi** è il sistema di supporto più semplice, quando il processo fa una richiesta di input/output, il SO assegna un buffer all'operazione (nello spazio del sistema), nelle operazioni di input, i dati sono trasferiti nel buffer.

Input anticipato:

- in previsione che il blocco sia richiesto;
- terminato il trasferimento, il blocco viene trasferito dal buffer allo spazio utente e ne viene immediatamente richiesto un altro.

Velocizza il sistema:

- Mentre il processo elabora un blocco di dati, il blocco successivo viene caricato nel processo;
- Il SO può scaricare il processo perchè il trasferimento avviene nello spazio del sistema.

Svantaggi:

- Maggior complessità nella programmazione del SO;
- Influenza la gestione dello swapping.

Per i dispositivi a flusso a una linea alla volta oppure un byte alla volta, prendiamo ad esempio una stampante a linee o i terminali in modo scroll, oppure i terminali in modo form, l'**input** è diviso in linee che terminano con **invio o byte**:

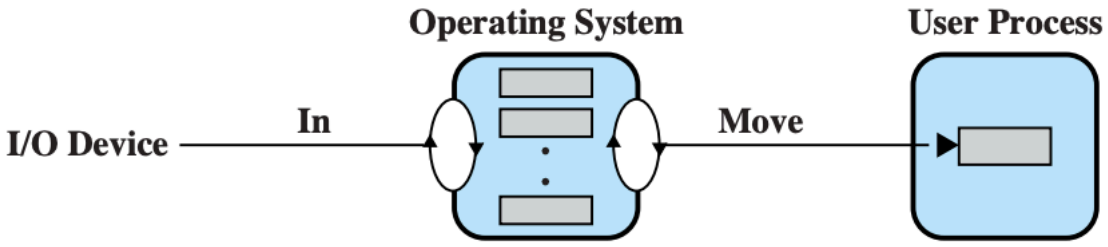
- Il processo è sospeso in attesa della linea di input;
- La linea di output è trasferita nel buffer e il processo può continuare l'elaborazione.

DOPPIO BUFFER

Alternativamente si possono usare due buffer per l'operazione di I/O, uno in cui il processo trasferisce i dati e l'altro che il SO svuota o riempie, questo ci permette di agire ancora più velocemente poichè non si deve attendere che il buffer si sia svuotato.

BUFFER CIRCOLARE

Ci sono più di due buffer assegnati all'operazione I/O caricati e scaricati circolarmente.

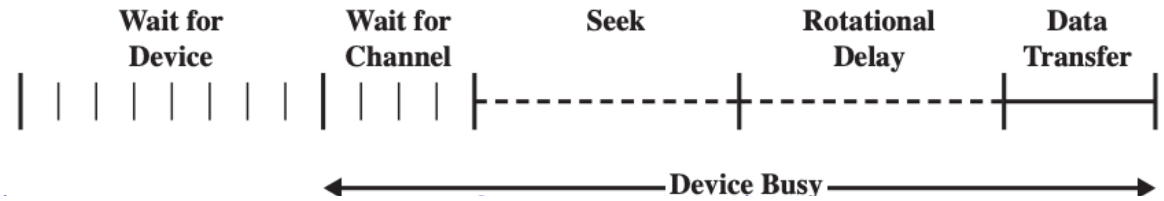


(d) Circular buffering

PARAMETRI DI PRESTAZIONE DEL DISCO

Il disco è diviso in tracce, cerchi concentrici che contengono il materiale ferro-magnetico usato per memorizzare le cose, su ogni disco c'è una testina che scorre sulla traccia, per certi dischi c'è una testina per ogni traccia. Essa serve per la lettura/scrittura e deve essere posizionata all'inizio del **settore** richiesto nella particolare **traccia** richiesta, nei dischi **a teste fisse**, la traccia si ottiene selezionando elettronicamente la relativa testina, nei dischi a **teste mobili** è necessario spostare la

testina sulla relativa traccia e sull'inizio del relativo **settore.** (**tempo di ricerca)**.
Una volta avviato il sistema il disco comincia a ruotare con velocità costante.
Il tempo necessario al settore per raggiungere la testina è detto **ritardo rotazionale,** la somma di tempo di ricerca e ritardo rotazionale costituisce il **tempo di accesso,** dopo il quale siamo all'inizio del settore che ci interessa e la testina può leggere o scrivere i dati che ci interessano**.**



TEMPO DI ACCESSO

Tempo di ricerca:
Tempo necessario a muovere la testina del disco sulla traccia richiesta, consiste di:

- tempo di avvio;
- tempo necessario per attraversare i cilindri (comprensivo di **settling time)**.

Più piccolo è il disco, minore è il tempo di ricerca (oggi, < 10 ms)
Ritardo rotazionale:
Il tempo necessario al settore per raggiungere la testina, più il disco gira velocemente, minor tempo impiegherà il settore ad arrivare, anche qui siamo nell'ordine dei millisecondi (può essere a 3600 rpm, in media 8.3 millisecondi, oppure a 15000 rpm, in media 2 millisecondi).
Tempo di trasferimento:
Lo possiamo calcolare con $T = \frac{b}{rN}$, con b = bit da trasferire, N = numero di bit per traccia, r = velocità di rotazione in rps.

Tempo di accesso medio:

$$T_a = T_s + \frac{1}{2r} + \frac{b}{rN}$$

ove
b = numero di bit da trasferire,
N = numero di bit per traccia,
r = velocità di rotazione in rps,
T_s = tempo di ricerca medio

Esempio:

Un disco con tempo di ricerca medio 4ms, velocità rotazionale 7500 rpm, 500 settori per traccia, ciascuno da 512 byte.

Un file occupa tutti i settori su 5 tracce adiacenti. (organizzazione sequenziale)

Il tempo per la lettura della prima traccia sarà

tempo di ricerca medio	4 ms
ritardo rotazionale	4 ms
lettura di 500 settori	8 ms
	16 ms

Per le tracce successive, il tempo di ricerca si riduce quasi a 0, per cui il tempo totale necessario è

$$(16 + 12 + 12 + 12)\,\text{ms} = 64\,\text{ms}.$$

Se i settori sono distribuiti a caso sul disco, il tempo di accesso a ogni settore è

tempo di ricerca medio	4 ms
ritardo rotazionale	4 ms
lettura di un settore	0.016 ms
	8.016 ms

Per cui il tempo totale necessario risulta, mediamente,

$$5 \cdot 500 \cdot 8.016\,\text{ms} = 20.04\,\text{secondi}.$$

È molto più inefficiente se i settori non sono distribuiti in maniera sequenziale.

È dunque opportuno scegliere la migliore **politica di schedulazione del disco.**

POLITICHE DI SCHEDULAZIONE DEL DISCO

L'obiettivo di queste politiche è quello di ridurre il tempo medio di ricerca.

FIFO

Le richieste sono **processate nel loro ordine di arrivo**, è semplice da implementare ed è equa per tutti i processi, in presenza di molti processi, approssima il tempo impiegato dalla schedulazione random.

Gestione Prioritaria

La gestione a priorità è chiaramente eseguita al di fuori del software di controllo del disco, l'obiettivo non è quindi ottimizzare l'utilizzo del disco, ma ottimizzare il comportamento globale.
La priorità verrebbe data ai **lavori brevi** o ai **processi interattivi**, per premiare l'utente che utilizza il sistema.
Si hanno **buoni tempi di risposta per i processi a cui la priorità è data, ma lunghe attese per i lavori che in generale hanno bassa priorità** (un problema potrebbe essere la starvation dei lavori con bassa priorità), come ad esempio i lavori batch, in un sistema condiviso, l'utente può però spezzare il lavoro in piccoli segmenti per "ingannare" il sistema.
È inoltre inadatta per i sistemi di basi di dati.

SSTF

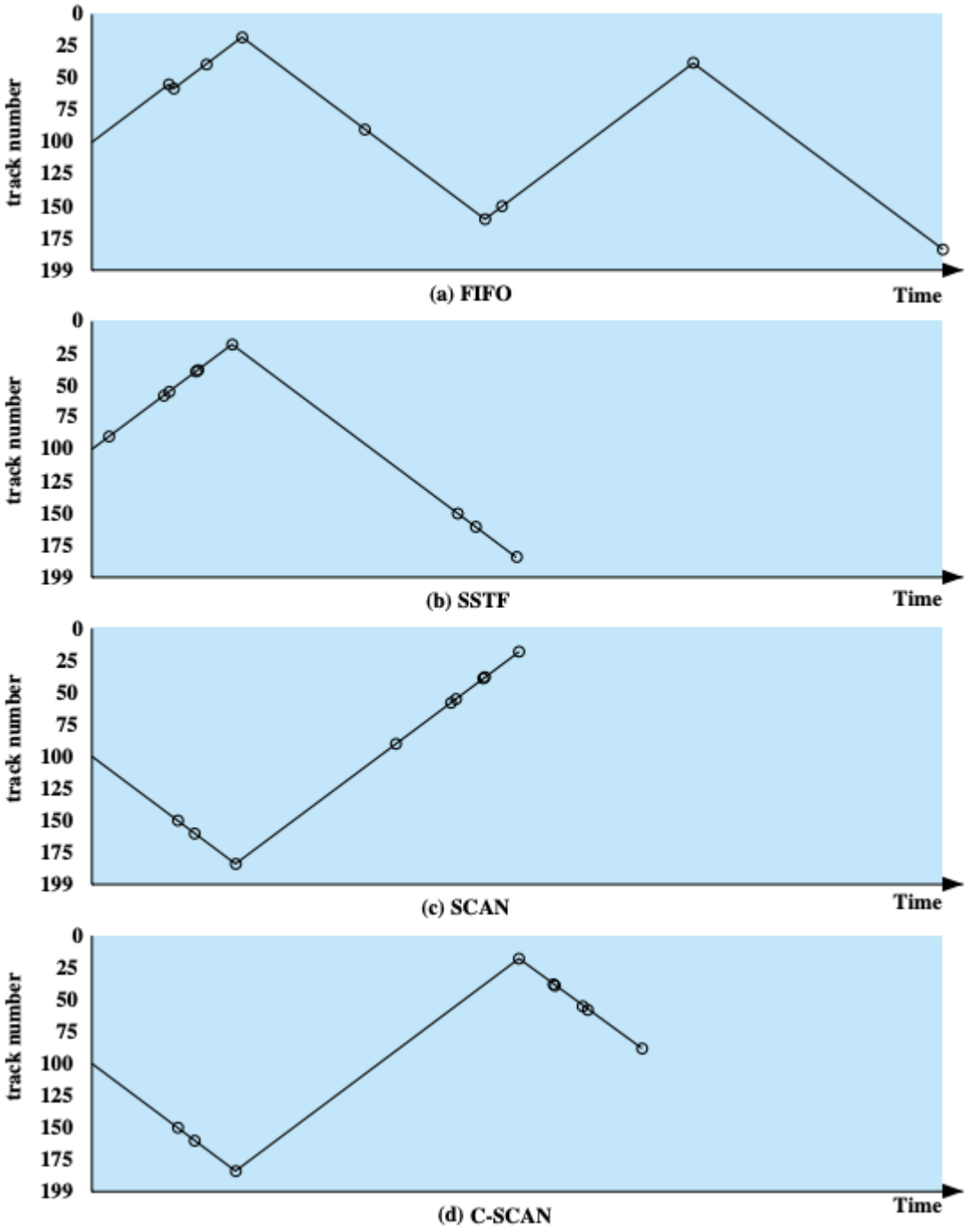
Si dà precedenza alla richiesta con tempo di servizio minore (**shortest service first**), si seleziona quindi la richiesta che richiede il minimo movimento del braccio del disco dalla posizione corrente.
Questo minimizza il tempo di richiesta, ma anche qui c'è la possibilità di starvation per lavori che richiedono tempo di servizio più alto.

SCAN
Detto anche l'**algoritmo dell'ascensore,** come per gli ascensori c'è una fase di salita e una di discesa, nella fase di salita il disco si ferma su un cilindro e fa tutti i lavori su quel cilindro, poi passa al prossimo cilindro superiore se c'è richiesta di altri lavori su cilindri superiori a quello precedente, quando a salire non ci sono altre richieste comincia la fase di discesa.
Il braccio quindi si muove sempre in una direzione e soddisfa tutte le richieste su ciascuna traccia finché non raggiunge l'ultima traccia, poi inverte la direzione.
Tutto sommato si ha una buona performance e si ha una soluzione al problema dell'attesa indefinita poiché prima o poi tutte le richieste vengono completate, vengono favoriti i settori centrali poichè vengono incontrati sia in discesa che in salita.

C-SCAN
Restringe la scansione a un'unica direzione, poi raggiunta l'ultima traccia, il braccio ritorna alla prima traccia e ricomincia la scansione nella medesima direzione.

N-Step-SCAN
Evita attese troppo lunghe quando nuovi lavori arrivano sulla medesima traccia.
Le richieste sono inserite in sotto-code di lunghezza N che sono processate una alla volta con SCAN.
Le nuove richieste non possono essere inserite nella coda attualmente processata.

FSCAN
Si hanno due sottocode, all'inizio della scansione una coda è vuota, durante la scansione sono eseguite le richieste della coda inizialmente non vuota e tutte le nuove richieste sono inserite nell'altra.


track number
0
25
50
75
100
125
150
175
199
Time
(a) FIFO
track number
0
25
50
75
100
125
150
175
199
Time
(b) SSTF
track number
0
25
50
75
100
125
150
175
199
Time
(c) SCAN
track number
0
25
50
75
100
125
150
175
199
Time
(d) C-SCAN

(a) FIFO (starting at track 100)		(b) SSTF (starting at track 100)		(c) SCAN (starting at track 100 in the direction of increasing track number)		(d) C-SCAN (starting at track 100 in the direction of increasing track number)	
Next track accessed	Number of tracks traversed	Next track accessed	Number of tracks traversed	Next track accessed	Number of tracks traversed	Next track accessed	Number of tracks traversed
55	45	90	10	150	50	150	50
58	3	58	32	160	10	160	10
39	19	55	3	184	24	184	24
18	21	39	16	90	94	18	166
90	72	38	1	58	32	38	20
160	70	18	20	55	3	39	1
150	10	150	132	39	16	55	16
38	112	160	10	38	1	58	3
184	146	184	24	18	20	90	32
Average seek length	55.3	Average seek length	27.5	Average seek length	27.8	Average seek length	35.8

Confronto tra politiche:

Name	Description	Remarks
	Selection according to requestor	
RSS	Random scheduling	For analysis and simulation
FIFO	First-in-first-out	Fairest of them all
PRI	Priority by process	Control outside of disk queue management
LIFO	Last in first out	Maximize locality and resource utilization
	Selection according to requested item	
SSTF	Shortest-service-time first	High utilization, small queues
SCAN	Back and forth over disk	Better service distribution
C-SCAN	One way with fast return	Lower service variability
N-step-SCAN	SCAN of N records at a time	Service guarantee
FSCAN	N-step-SCAN with N = queue size at beginning of SCAN cycle	Load sensitive

GESTIONE DEI FILE (14/05)

Tutti i dati sono contenuti nei file, una collezione di dati che il sistema ci permette di interpretare grazie ad una struttura logica. È considerata fra le parti più importanti del Sistema Operativo lato utente.

Proprietà dei file

- Persistenza: i file sono memorizzati e non scompaiono. Quando spegniamo il PC, se tutto funziona come dovrebbe, il file rimane la.
- Condivisione: i file hanno un nome e dei permessi, (interessante soprattutto nei sistemi condivisi perché utenti differenti possono essere autorizzati o meno a fare uso dei vari file).
- Struttura: un aspetto fondamentali dei file è che sono organizzati in strutture gerarchiche.

File system

Per poter organizzare i nostri file abbiamo bisogno di una parte del sistema operativo detto “gestore del file system”.
Questo fornisce una serie di operazioni che permettono di operare sui file e anche di gestire gli attributi associati ai file (per esempio i permessi).
Le principali operazione che il file system ci deve assicurare: creare i file, cancellazione dei file, apertura (per scrivere o leggere il file), chiusura, lettura (di vecchie informazioni) e scrittura (di nuove informazioni)

Sistema di Gestione dei File

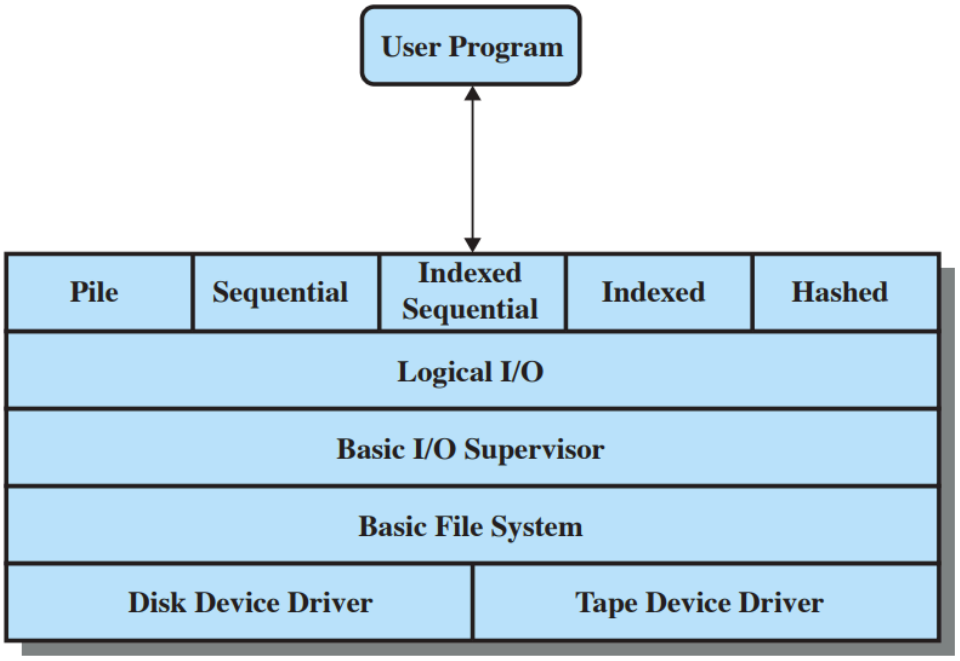
Abbiamo quindi la necessità di un sistema di gestione dei file.
Questo fornisce agli utenti quelle operazioni che possiamo utilizzare nei file.
Obiettivi:

- Incontrare le esigenze degli utenti (i principali metodi di accesso ai file usando windows sono quelli che utilizzano l’interfaccia grafica, molto utile per gli utenti meno esperti)
- Permettono di modificare i dati in maniera veloce.
- Garantire la validità dei dati (le operazioni che coinvolgono i dispositivi sono quelle che danno i maggiori problemi, a causa delle parti meccaniche, fili… quindi bisogna verificare che le

operazioni siano state portate a termine senza errori oppure segnalare gli errori).
- Provvedere all'input/output dei dispositivi (disco fisso, disco ottico, nastro, penna USB…)
- Minimizzare il rischio di perdita o distruzione dei dati (un backup del contenuto del disco è sempre utile).
- Fornire un'interfaccia standardizzata, uniforme.
- Nei sistemi multiutente deve essere previsto il supporto input/output.

Esigenze degli utenti:
- Creare, cancellare, leggere, scrivere, modificare i file
- Nei sistemi multiutente l'accesso ai file deve essere controllato
- Permesso di esecuzione (di file eseguibili)
- Fornire la possibilità di spostare i dati fra i file
- Meccanismo di backup con relativa possibilità di ripristino del file in caso di danneggiamento (capire come ripristinare i file dal backup).
- Fondamentale: l'accesso ai file per nome

Architettura del sistema di gestione dei file



Il sistema di gestione dei file viene organizzato a strati.

Partendo dal livello più basso, troviamo **l'hardware** dei dispositivi di memorizzazione (dischi, nastri, chiavette, CD, ecc.). I **Driver dei Dispositivi** comunicano direttamente con queste periferiche e gestiscono le operazioni di input/output, sia il loro avvio che il loro completamento.

Il **Basic File System** agisce come interfaccia elementare con l'ambiente esterno (I/O fisico).
È responsabile del posizionamento dei blocchi di dati all'interno della memoria secondaria. Ad esempio, i caratteri di un file sono distribuiti in blocchi sul dispositivo, che non sono necessariamente consecutivi, e il Basic File System tiene traccia di dove questi blocchi si trovano. Un'altra sua operazione è l'organizzazione dei buffer nella memoria principale (soprattutto per le operazioni di scrittura). A questo livello, non viene ancora gestita la struttura logica dei file.

Ancora più in alto, il **Basic I/O Supervisor** (o supervisore dell'I/O base) è responsabile dell'avvio, della terminazione e della gestione del trasferimento effettivo dei dati per tutte le operazioni di input/output relative ai file. Gestisce le strutture di controllo per i dispositivi, implementa le politiche di schedulazione degli accessi ai dispositivi e cura lo stato dei file. Inoltre, seleziona il dispositivo appropriato su cui eseguire un'operazione di I/O.

Al di sopra, il **Gestore Logico di Input/Output (Logical I/O)** permette di vedere un file come una collezione di record logici di lunghezza fissa o variabile (ad esempio, in un file sequenziale semplice, ogni record potrebbe essere un singolo carattere). Abilita gli utenti e le applicazioni ad accedere ai file basandosi su questa struttura logica, interpretando la sequenza binaria memorizzata fisicamente come una collezione organizzata di record, realizzando così l'aspetto logico dell'accesso ai file.

Infine, al livello più alto, i **Metodi di Accesso** rappresentano l'interfaccia più prossima all'utente e alle applicazioni. Il sistema operativo deve fornire questi diversi metodi per accedere ai file, in base alla loro organizzazione logica. Questo strato realizza un'interfaccia uniforme e standardizzata.
Differenti metodi di accesso sono dovuti alle diverse strutture logiche con cui i file possono essere organizzati.

Directory

Il file system utilizza le directory per organizzare i file.
Per l'utente i file hanno un nome e un utilizzatore associato.
Immaginiamo di avere dei cassetti, all'interno dei quali ci sono altri cassetti e poi ci sono i file.
Noi abbiamo la possibilità di navigare fra le directory.
Operazioni possibili sulla directory:
Ricerca, crea file, cancella file, listare il contenuto della directory, aggiornare la directory

Tabella con le informazioni principali che caratterizzano i file.

	Basic Information
File Name	Name as chosen by creator (user or program). Must be unique within a specific directory.
File Type	For example: text, binary, load module, etc.
File Organization	For systems that support different organizations
	Address Information
Volume	Indicates device on which file is stored
Starting Address	Starting physical address on secondary storage (e.g., cylinder, track, and block number on disk)
Size Used	Current size of the file in bytes, words, or blocks
Size Allocated	The maximum size of the file
	Access Control Information
Owner	User who is assigned control of this file. The owner may be able to grant/deny access to other users and to change these privileges.
Access Information	A simple version of this element would include the user's name and password for each authorized user.
Permitted Actions	Controls reading, writing, executing, transmitting over a network
	Usage Information
Date Created	When file was first placed in directory
Identity of Creator	Usually but not necessarily the current owner
Date Last Read Access	Date of the last time a record was read
Identity of Last Reader	User who did the reading
Date Last Modified	Date of the last update, insertion, or deletion
Identity of Last Modifier	User who did the modifying
Date of Last Backup	Date of the last time the file was backed up on another storage medium
Current Usage	Information about current activity on the file, such as process or processes that have the file open, whether it is locked by a process, and whether the file has been updated in main memory

Schema a due livelli
C’è una directory principale e una directory per ogni utente.
Dalla directory principale possiamo raggiungere le directory degli utenti.
Il SO si occupa delle restrizioni di accesso.
Si utilizza lo schema ad albero.

Organizzazione dei file

- Il meccanismo più semplice per organizzare i file è il meccanismo a mucchio: i dati vengono registrati in ordine di arrivo. Ogni nuovo record viene attaccato in coda al file. Sono però difficili da leggere, infatti per trovare un record bisogna leggere tutto il file.
- Il file sequenziale: i record hanno formato fisso, ogni record ha gli stessi campi, c’è un campo chiave che serve ad identificare il record, l’ordine è basato su una certa logica che permette poi di ricercare in maniera efficiente la chiave. Il file delle transazione contiene i nuovi record (aggiunti successivamente).
- File sequenziale indicizzato: c’è un indice per accesso random che rende l’accesso ai record è accelerato. Le modifiche ai file vengono inserite in un file di overflow. Nel file di overflow ci sono dei puntatori ai nuovi record.
- File indicizzato: grazie agli indici si accede rapidamente ai file
- File diretto o a hash: sono ancora migliori. L’hash della chiave permette di accedere direttamente al blocco (disco/supporto sul quale i dati sono salvati è diviso in blocchi, unità che contengono una quantità di dati, che vengono trasferite in un'unica operazione dal dispositivo alla memoria e viceversa) del record. Con l’hash, non c’è bisogno di organizzare i record in maniera sequenziale.

Allocazione dei file
Nella memoria secondario il file è una collezione di blocchi.
I file sono organizzati in record.
L'approccio che noi scegliamo per allocare i file è fondamentale per l'efficienza del sistema. È fondamentale selezionare la politica di allocazione dei blocchi nel modo migliore.
Le cose funzionano bene se noi allocazione lo spazio in porzioni (blocchi contigui, uno dietro l'altro)

La struttura che tiene traccia dei blocchi allocati ai file viene chiamata FAT. Ci sono 2 possibilità per allocare:

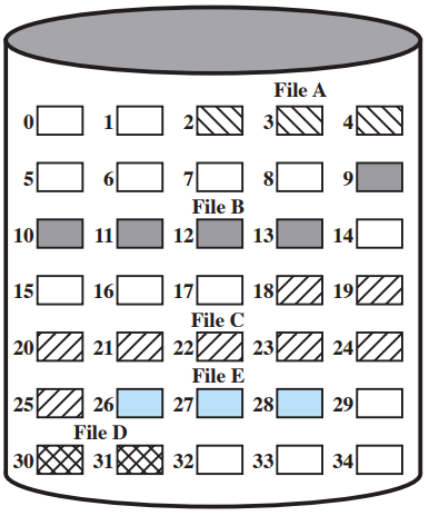
- preallocazione: creo il mio file, dichiaro quanti blocchi sono necessari, il sistema seleziona i (per esempio) 100 blocchi che mi servono facendo si che siano contigui e quindi che vi si possa accedere rapidamente. Problema: la stima dello spazio necessario è difficile, si tende a sovrastimare. Però comunque mi permette di allocare gli spazi in maniera intelligente perchè lo faccio prima
- allocazione dinamica: se serve un nuovo blocco per dei nuovi record, si alloca un nuovo blocco.

Dimensione delle porzioni
2 esigenze in contrasto:
da una parte l'opportunità di avere blocchi contigui (in certi casi è fondamentale), porzioni grandi e la dimensione della FAT diminuisce. Inoltre con le porzioni di taglia fissa si facilita l'allocazione dello spazio.
Lo svantaggio è che c'è lo spreco di spazio.

Invece, se usiamo come unità di allocazione i blocchi siamo più flessibili, ma perdiamo la contiguità e le tabelle di allocazioni sono grandi. I blocchi vengono assegnati dinamicamente (è inutile preallocare se perdiamo la contiguità).

Nell'allocazione contigua potremmo dover ricorrere alla compattazione per avere dei blocchi contigui liberi.

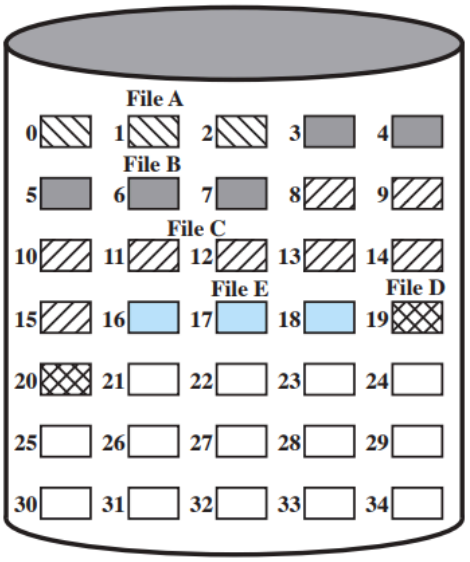
Allocazione contigua



File Allocation Table

File Name	Start Block	Length
File A	2	3
File B	9	5
File C	18	8
File D	30	2
File E	26	3

Allocazione contigua (dopo compattazione)

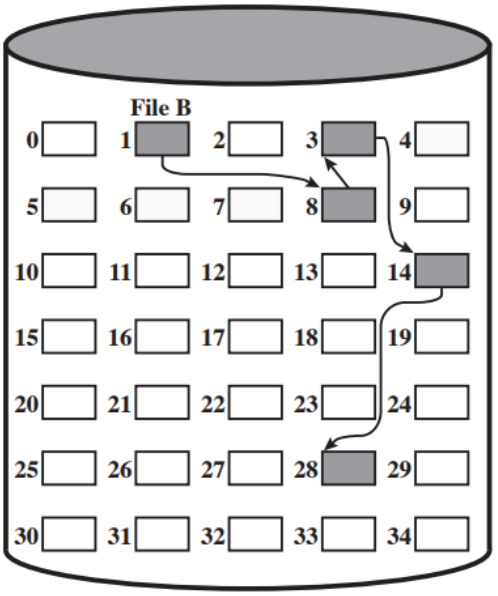


File Allocation Table

File Name	Start Block	Length
File A	0	3
File B	3	5
File C	8	8
File D	19	2
File E	16	3

Nell'allocazione a catena si perde la continuità, ogni blocco contiene un puntatore al blocco successivo: non abbiamo problemi di compattazione.

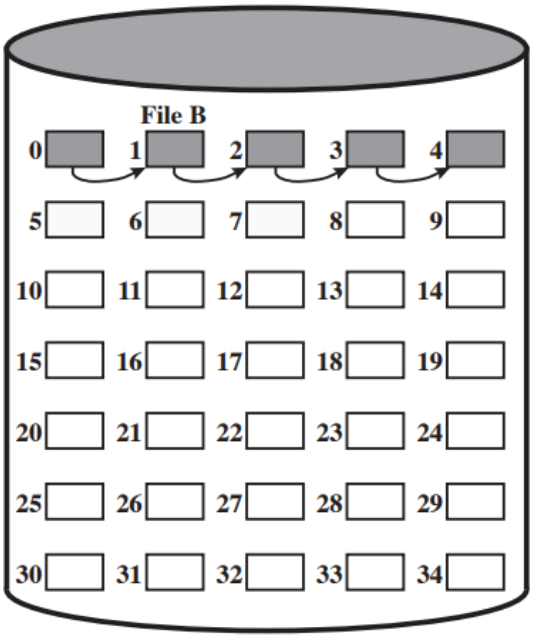
Allocazione a catena



File Allocation Table

File Name	Start Block	Length
• • •	• • •	• • •
File B	1	5
• • •	• • •	• • •

Allocazione a catena (dopo compattazione)

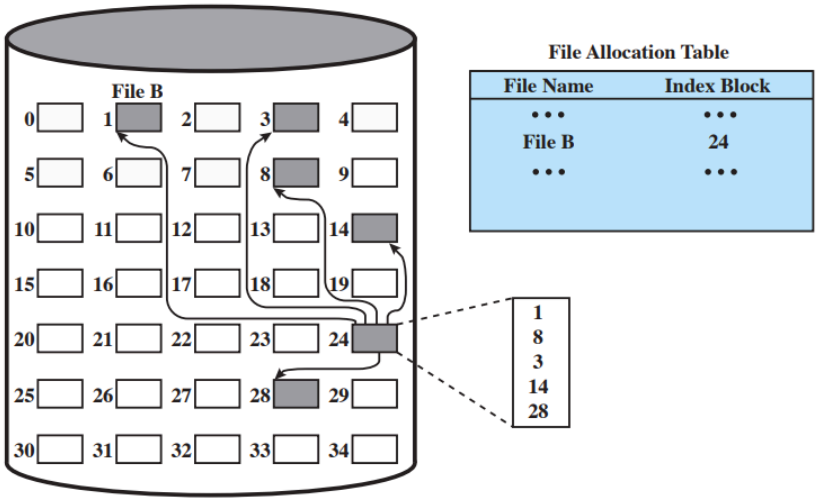


File Allocation Table

File Name	Start Block	Length
• • •	• • •	• • •
File B	0	5
• • •	• • •	• • •

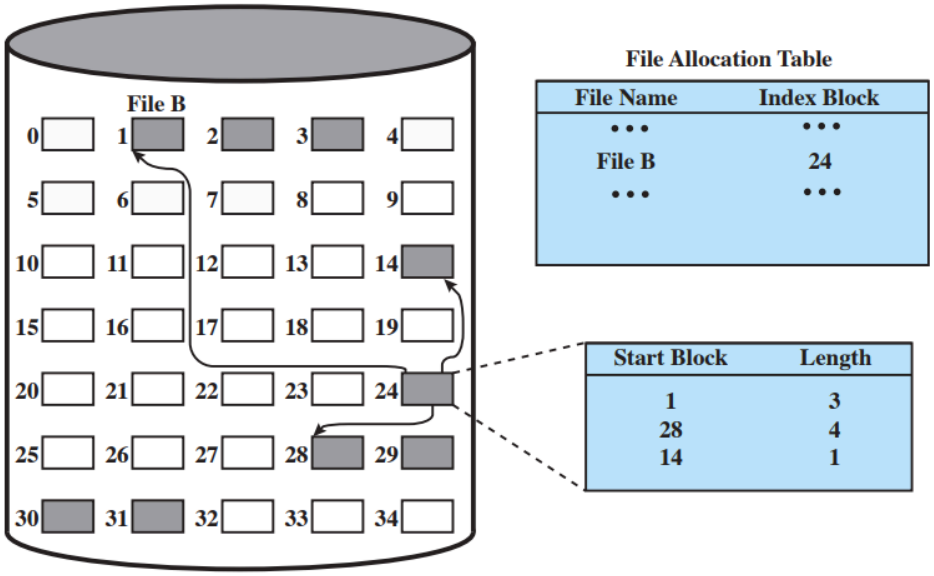
Allocazione indicizzata con porzioni a blocchi: il primo blocco contiene tutti i puntatori verso gli blocchi. È così molto rapido per accedere magari all'ultimo blocco.

Allocazione indicizzata con porzioni a blocchi



Allocazione indicizzata con porzioni a lunghezza variabile: la differenza è che qua nei limiti del possibile i blocchi solo allocati in porzioni di lunghezza variabile. Nel primo blocco ci sono i puntatori ai primi blocchi della porzione. Il vantaggio è che almeno parzialmente i blocchi sono contigui.

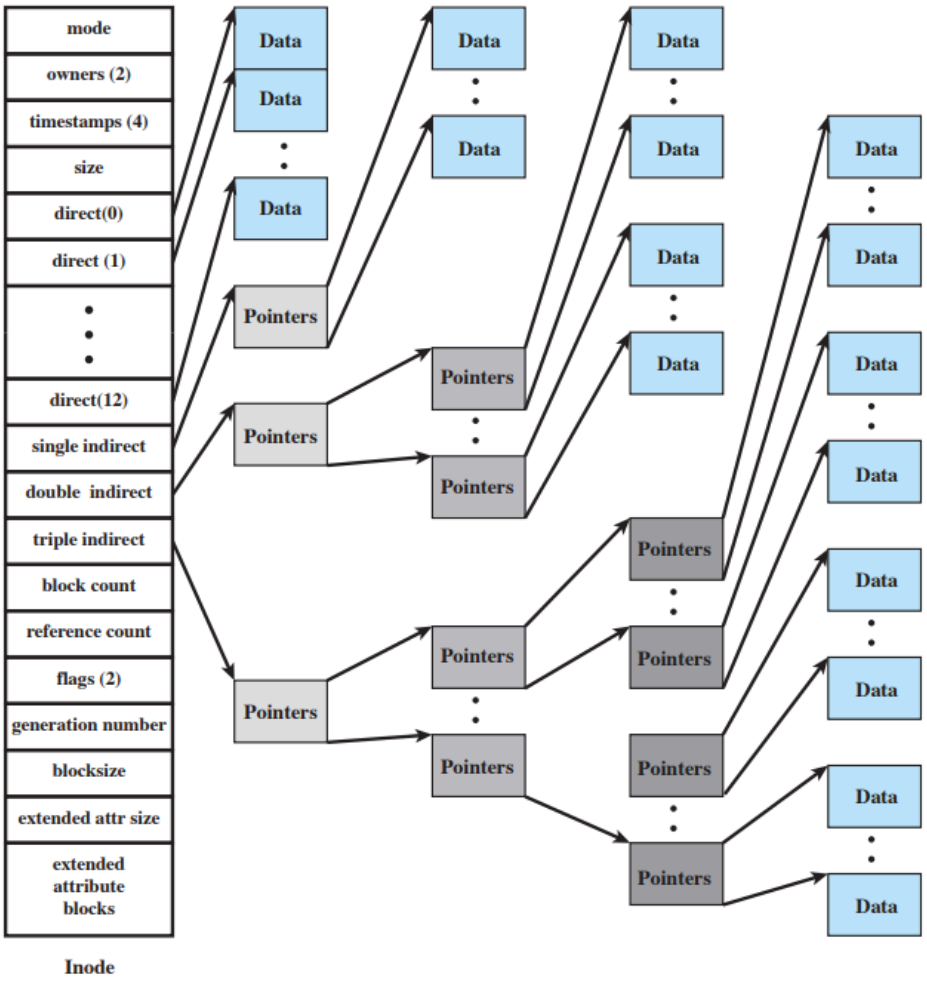
Allocazione indicizzata con porzioni a lunghezza variabile



Gestione degli spazi liberi
Come assegnamo i blocchi che non sono stati ancora assegnati?
Con la tabella di allocazione del disco troviamo i blocchi ancora disponibili.
Vari meccanismi:
La più semplice è la bitmap: ho un milione di blocchi, utilizzo un vettore con un milione di bit, uno per ogni blocco.
Porzioni libere concatenate: non abbiamo spreco di spazio
Indicizzazione:

Gestione file in UNIX
6 tipi di file:
ordinari, directory (contiene una lista di nomi di file + il puntatore ai relativi i-nodi, cioè la struttura dati che serve a gestire il file), file speciali (usati per accedere ai dispositivi periferici), pipe (servono ai processi per comunicare fra di loro), link (un nome alternativo ai file già esistenti), link simbolico (meccanismo differente per accedere allo stesso file).

I-node
La struttura chiave per la gestione dei file sotto UNIX è la i-node. Contiene tutte le informazioni che servono al sistema per gestire i file. Ogni file ha il suo i-node, corrispondenza 1:1), ma più file possono essere associati allo stesso i-node.



Struttura del volume

Come è fatto un disco? (stiamo parlando di dischi logici)
Un disco può essere partizionato e quindi apparire come più volumi.
Si può fare anche il contrario.

Com'è fatto il volume? può contenere:
un boot block (sicuramente lo contiene il disco principale): serve per caricare il sistema operativo.
Poi c'è il super block, contiene tutte le informazioni e gli attributi del file system: dirà la dimensione del volume dei blocchi, come sono organizzati i file.
Poi c'è la tabella degli i-node ed infine i blocchi dati.